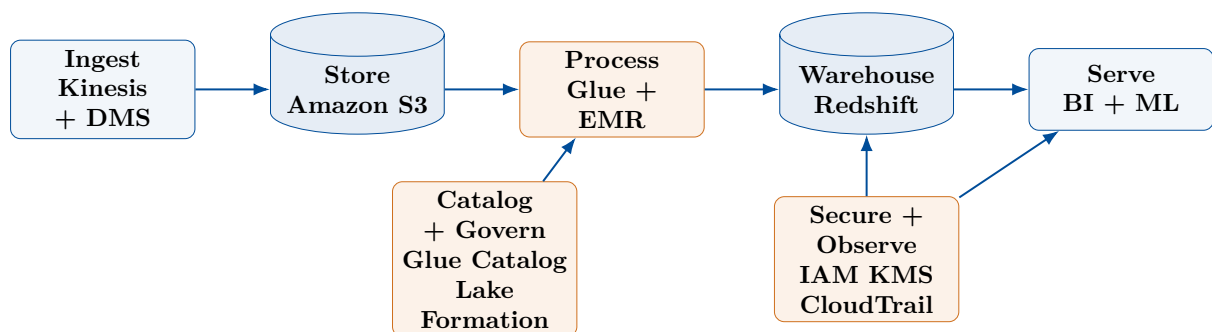
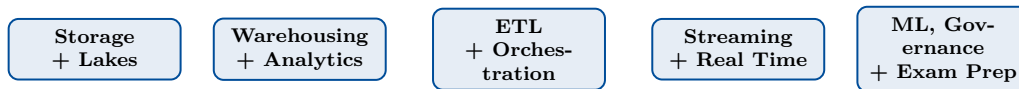

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

AWS Master Data Engineer

Data Engineering on Amazon Web Services



Inside this book



From durable object storage to governed lakehouse, streaming, analytics, and machine-learning data platforms.

PhD Aned Esquerra Arguelles

A 24-week, milestone-driven academic and hands-on AWS data engineering program

July 31, 2026

Preface

Welcome to *AWS Master Data Engineer*. This text is designed as an academic, hands-on, progressive program for learning how modern data platforms are built on Amazon Web Services. By working through the architecture discussions, service trade-offs, and weekly labs, you will move from foundational object storage to secure, governed, production-grade data lakehouse and analytics systems.

This book is the theory-and-architecture companion to a 24-week AWS Data Engineering training program. Where the weekly practice workspace asks you to *deploy* services and build projects, this book explains *why* each service exists, what operational failure mode it addresses, and how a data engineer reasons about scale, security, governance, cost, and reliability.

Who This Book Is For

This book is written for aspiring and practicing data engineers, cloud engineers, analytics engineers, database developers, and technical leads who want a rigorous path into AWS data engineering. It assumes basic SQL, Python, command-line comfort, and familiarity with data formats such as CSV, JSON, and Parquet. Prior AWS experience helps, but the first part establishes the storage, identity, networking, and lake foundations used throughout the rest of the program.

How This Book Is Organized

The book follows six Parts, matching six curriculum milestones, and 24 chapters, matching 24 training weeks:

- **Part I: Core Data Infrastructure and Storage** builds the base: Amazon S3, IAM and networking concepts, data lake layout, and Athena query patterns.
- **Part II: Data Warehousing and Analytics** develops Amazon Redshift, performance design, semantic analytics, QuickSight, and cost-aware analytical platforms.
- **Part III: Batch Processing and ETL Orchestration** covers the AWS Glue Data Catalog, Glue ETL with Spark, Amazon EMR, Step Functions, and managed orchestration patterns.
- **Part IV: Streaming and Real-Time Analytics** introduces Kinesis, Firehose, Amazon MSK, Lambda stream processing, and real-time architecture trade-offs.
- **Part V: Machine Learning and MLOps** connects data engineering to SageMaker, feature stores, ML data pipelines, model monitoring, and operational ML workflows.
- **Part VI: Security, Governance, and Exam Prep** consolidates Lake Formation, compliance, observability, architecture review, capstone work, and certification-oriented re-

view.

Appendices provide selected command references, design checklists, troubleshooting playbooks, cost-estimation patterns, and worked solutions for selected exercises.

Reading Paths

- **New to AWS data engineering:** read Parts I–III in order before attempting streaming, MLOps, or governance-heavy architectures.
- **Already using AWS analytics:** skim Chapter 1 for terminology, then focus on Redshift, Glue, EMR, and Kinesis chapters relevant to your current platform.
- **Building a portfolio:** complete each hands-on project and keep architecture diagrams, CLI commands, failure notes, and cost estimates as evidence.
- **Preparing for interviews or certification:** work every Key Takeaway, Interview Q&A, and Exercise section; practice explaining trade-offs rather than memorizing service names.
- **Operating a regulated platform:** read the security and governance material alongside every technical chapter, not only at the end.

Conventions

Code identifiers, AWS CLI commands, bucket names, IAM actions, and file names appear in **monospace**. Python listings use the shared style defined in the preamble; shell commands and JSON policies use listing captions to state their purpose. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Every architecture discussion treats cost, reliability, security, and governance as first-class design dimensions.

Contents

0	Getting Started: Your AWS Free Tier Account and Environment	1
0.1	Why Chapter 0 Exists	1
0.2	Creating an AWS Account on the Free Tier	2
0.2.1	What You Need Before Sign-up	2
0.2.2	Understanding Free Tier Categories	2
0.2.3	Sign-up Walkthrough	3
0.3	Securing the Account	3
0.3.1	Root User: Owner, Not Operator	3
0.3.2	Enable MFA on the Root User	4
0.3.3	Create an Administrative Identity	4
0.3.4	Password Policy and Account Hygiene	4
0.4	Installing and Configuring the Tooling	5
0.4.1	AWS CLI v2 on Windows	5
0.4.2	Configuration Option A: IAM Identity Center	5
0.4.3	Configuration Option B: Access Keys	5
0.4.4	Profiles, Regions, and Output Formats	6
0.4.5	AWS CloudShell as a No-install Alternative	6
0.5	Cost Guardrails to Stay Free	7
0.5.1	Turn on Billing Visibility	7
0.5.2	Create a Zero-Spend or Low-Spend Budget	7
0.5.3	Free Tier Usage Alerts and Service-specific Controls	9
0.6	Choosing a Region and Naming Conventions	9
0.6.1	Default Region for the Book	9
0.6.2	Naming Convention	9
0.6.3	Tagging Convention	10
0.7	Setup Flow Across the Six Platform Books	10
0.8	Verification: Hello World	11
0.8.1	Check the Caller Identity	11
0.8.2	Create, List, and Delete a Small S3 Bucket	11
0.8.3	Optional Setup Flow Diagram	12
0.9	Further Reading	12
0.10	Exercises	12
I	Core Data Infrastructure and Storage	14
1	Amazon S3 and Storage Foundations	15
1.1	Week 1 Roadmap and Mental Model	16
1.2	Monday: S3 Architecture and Storage Classes	17
1.2.1	Object Storage, Not a Shared File System	17

1.2.2	Regional Buckets and Global Names	17
1.2.3	Durability and Availability	18
1.2.4	Consistency Model	18
1.3	Choosing a Storage Class for Data Engineering	19
1.4	Tuesday: Lifecycle, Versioning, Object Lock, and Replication	19
1.4.1	Lifecycle Policies	19
1.4.2	Versioning	19
1.4.3	Object Lock: Governance, Compliance, and Legal Hold	19
1.4.4	S3 Replication: SRR and CRR	20
1.5	Wednesday: Migration and Hybrid Storage	20
1.5.1	AWS DataSync	20
1.5.2	AWS Storage Gateway	21
1.5.3	AWS Snow Family	21
1.6	Thursday Hands-on Project: Bucket, Lifecycle, and Events	22
1.6.1	Boto3 Verification and Event Reader	24
1.7	Friday Use Case: Healthcare Disaster-Recovery Storage Architecture	25
1.7.1	Requirements and Assumptions	25
1.7.2	Architecture Decisions	25
1.8	Interview Q&A	26
1.8.1	Concepts and Trade-offs	26
1.9	Further Reading	27
1.10	Exercises	27
1.11	Capstone Project: A Compliant Multi-Region Medical Image Data Lake on S3	28
2	Relational Databases: RDS and Aurora	38
2.1	Week 2 Roadmap and Mental Model	39
2.2	Monday: Amazon RDS Architecture, Multi-AZ, and Read Replicas	40
2.2.1	Managed RDS Architecture	40
2.2.2	Backups, Maintenance, and Observability	40
2.2.3	Multi-AZ Deployments	40
2.2.4	Read Replicas	41
2.3	Tuesday: Amazon Aurora Cloud-Native Relational Architecture	42
2.3.1	Distributed Storage and Compute Separation	42
2.3.2	Aurora Global Database	42
2.3.3	Aurora Serverless v2	43
2.4	Wednesday: AWS DMS and Schema Conversion Tool	43
2.4.1	Migration Phases	43
2.4.2	Replication Instances, Endpoints, and Tasks	43
2.4.3	Validation and Cutover	43
2.5	Thursday Hands-on Project: Aurora Serverless v2 and Regional Failover	44
2.5.1	Boto3 Verification	45
2.6	Friday Use Case: Zero-Downtime Oracle ERP Migration to Aurora PostgreSQL	46
2.6.1	Architecture Narrative	46
2.6.2	Operational Decisions	47
2.7	Interview Q&A	47
2.7.1	Concepts and Trade-offs	47
2.8	Further Reading	48
2.9	Exercises	48
2.10	Capstone Project: Migrating and Operating a Cross-Region Aurora PostgreSQL Fleet for Oracle ERP Cutover	49
3	High-Scale NoSQL: Amazon DynamoDB	56

3.1	Week 3 Roadmap and Mental Model	57
3.2	Monday: DynamoDB Core Concepts, Partitions, and Capacity	58
3.2.1	Tables, Items, and Physical Partitions	58
3.2.2	Read and Write Capacity	58
3.2.3	Provisioned Versus On-Demand Capacity	59
3.3	Tuesday: Data Modeling with Primary Keys, Sort Keys, LSI, and GSI	59
3.3.1	Access-Pattern-First Modeling	59
3.3.2	Primary Keys and Sort-Key Encoding	59
3.3.3	Local and Global Secondary Indexes	60
3.4	Wednesday: Streams, TTL, and Global Tables	60
3.4.1	DynamoDB Streams	60
3.4.2	Time to Live	60
3.4.3	Global Tables	61
3.5	Thursday Hands-on Project: Forum Single-Table Schema and GSI	61
3.6	Friday Use Case: Global Gaming Leaderboard at One Million Reads and Writes Per Second	62
3.6.1	Architecture Narrative	63
3.6.2	Design Decisions	63
3.7	Interview Q&A	64
3.7.1	Concepts and Trade-offs	64
3.8	Further Reading	64
3.9	Exercises	65
3.10	Capstone Project: Building a Global Gaming-Leaderboard Data Layer	65
4	Hybrid Ingestion and Caching	71
4.1	Week 4 Roadmap and Mental Model	72
4.2	Monday: Amazon ElastiCache for Database Acceleration	73
4.2.1	Redis, Valkey, and Memcached in the Data Path	73
4.2.2	Cache-Aside, Write-Through, TTL, and Invalidation	73
4.2.3	Operational Design	74
4.3	Tuesday: DynamoDB Accelerator	74
4.3.1	Where DAX Fits	74
4.3.2	DAX Versus ElastiCache	74
4.3.3	DAX Operations	75
4.4	Wednesday: AppSync and API Gateway Data Ingestion	75
4.4.1	Managed API Front Doors	75
4.4.2	Choosing AppSync or API Gateway	76
4.4.3	Hybrid Ingestion Patterns	76
4.5	Thursday Hands-on Project: Redis Cache-Aside for RDS Queries	76
4.6	Friday Use Case: Mobile Sports API with Extreme Traffic Spikes	78
4.6.1	Architecture Narrative	78
4.6.2	Design Decisions	79
4.7	Interview Q&A	79
4.7.1	Concepts and Trade-offs	79
4.8	Further Reading	80
4.9	Exercises	80
4.10	Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend	81
A	Cross-Cloud Service Equivalence	87
A.1	Object and Data-Lake Storage	87
A.2	Managed Relational Databases	88
A.3	Elastic Compute and Analytical Warehouses	88

A.4 Data Organization and the Medallion Pattern	88
A.5 Document Data Modeling	89
A.6 Globally Distributed and NoSQL Databases	89
A.7 Managed Apache Spark	90
A.8 Query Acceleration and Materialized Views	90
A.9 In-Memory Caching and Hybrid Ingestion	90
A.10 Wide-Column and Key-Value NoSQL	91
A.11 Data Protection: Cloning, Time Travel, and Migration	91
Glossary	92

List of Figures

1	Chapter 0 setup flow: account, secure identity, tooling, cost guardrail, and verification.	12
1.1	A storage-class lifecycle path for analytical and archival objects. Actual thresholds must be justified by access data, compliance needs, and retrieval objectives.	19
1.2	A multi-region compliant disaster-recovery storage architecture for 10 PB of healthcare imaging data. Replication protects regional availability; Object Lock, KMS, IAM, Lake Formation, and CloudTrail support compliance controls.	25
1.3	Capstone reference architecture for a compliant multi-region medical-image data lake on S3. The primary bucket receives encrypted, versioned, immutable objects; lifecycle rules manage cost; CRR creates a disaster-recovery copy; S3 event notifications feed SQS for downstream processing.	30
2.1	RDS Multi-AZ protects availability by synchronously maintaining a standby in another Availability Zone and moving the endpoint during failover. It is not a read-scaling feature in the traditional DB instance model.	41
2.2	Aurora separates compute instances from a distributed cluster volume. Writer and reader endpoints route application traffic while the storage service handles replication and durability.	42
2.3	Zero-downtime Oracle ERP migration pattern using SCT for schema conversion, DMS for full load and CDC, validation for business reconciliation, and Aurora PostgreSQL with cross-region recovery for the target.	47
2.4	Capstone reference architecture for an Oracle ERP migration to Aurora PostgreSQL with DMS CDC, SCT conversion, high availability, cross-region recovery, observability, and analytics offload.	51
3.1	DynamoDB hashes the partition key to route items and requests to physical partitions. Sort keys organize related items after the partition key has chosen the item collection.	58
3.2	Global gaming leaderboard architecture with regional writes to DynamoDB Global Tables, streams for derived top-N views and lake ingestion, and cache or DAX layers for hot read serving.	63
3.3	Capstone reference architecture for a global gaming-leaderboard data layer with regional APIs, DynamoDB Global Tables, Streams processors, top-N materialized items, cache acceleration, S3 export, and observability.	67
4.1	Cache-aside flow between an application, ElastiCache Redis, and an RDS or Aurora source of truth.	73
4.2	Spiky-traffic mobile sports API using API Gateway, AppSync, ElastiCache, DAX, DynamoDB, Aurora or RDS, and streaming ingestion.	79

- 4.3 Milestone 1 e-commerce reference architecture integrating Aurora Serverless v2, DynamoDB, S3, ElastiCache, DAX, AWS DMS, and observability across all tiers. 83

List of Tables

1	Minimum training tags used throughout this book.	10
1.1	Storage-class choice as a workload and risk decision, not merely a price decision.	18
1.2	S3 management features mapped to operational review questions.	20
1.3	Migration and hybrid-storage options for AWS data engineering foundations.	21
2.1	RDS deployment patterns mapped to common data engineering decisions.	41
3.1	Example single-table access patterns for a forum workload.	59
4.1	DAX and ElastiCache solve different cache problems.	75

Getting Started: Your AWS Free Tier Account and Environment

Learning Objectives

- Create an AWS account, distinguish 12-month Free Tier offers from Always-Free offers and short trials, and identify the inputs required during sign-up.
- Secure the account by protecting the root user with MFA, avoiding root for daily work, and creating a separate administrative identity.
- Install and validate AWS CLI v2 on Windows PowerShell, and compare long-lived access-key configuration with IAM Identity Center sign-in.
- Configure profiles, default Regions, and AWS CloudShell so examples in later chapters can run reproducibly.
- Establish cost guardrails with billing alerts, Free Tier usage alerts, AWS Budgets, Cost Explorer, and routine Billing dashboard checks.
- Apply the Region, naming, and tagging conventions used throughout this book.
- Verify the environment with AWS STS and a short S3 create, list, and cleanup workflow.

Key Terms

Root user: the account owner identity created with the account email address. **IAM Identity Center:** AWS's workforce sign-in service, formerly AWS Single Sign-On. **Profile:** a named set of AWS CLI credentials and configuration. **Region:** a geographic AWS service location such as `us-east-1`. **Budget:** a billing guardrail that sends alerts when actual or forecasted spend crosses thresholds.

0.1 Why Chapter 0 Exists

Before creating buckets, Glue jobs, streams, warehouses, or lakehouse tables, you need a safe sandbox. This chapter builds that sandbox deliberately: account, identity, command-line tooling, budget alarms, naming rules, and a tiny verification exercise. The goal is not to become an AWS administrator in one sitting. The goal is to remove the avoidable problems that derail hands-on learning: using the root user every day, working in the wrong Region, losing track of profiles, creating resources without tags, or discovering charges after a lab was supposed to be cleaned up.

This book assumes a Windows workstation and PowerShell. Most AWS commands are the

same on macOS, Linux, AWS CloudShell, Git Bash, and Windows Terminal, but file paths, environment variables, quoting, and here-documents differ. Where the shell matters, listings use native PowerShell. Later chapters may show Bash-style commands when that is shorter for cloud shells or Linux-based services; when you run them on Windows, translate variable assignment and file creation to PowerShell.

Note

The examples in this chapter use a personal learning account or an organization-approved sandbox account. Do not run training commands in a production account unless your employer has explicitly approved the account, Region, budget, and cleanup process.

0.2 Creating an AWS Account on the Free Tier

0.2.1 What You Need Before Sign-up

To create a standard AWS account, prepare four items: a unique email address, a strong password, a credit or debit card for verification and payment, and a phone number that can receive verification. The email address becomes the root user sign-in name. Use an email address you can keep long term, because it is also the recovery channel for account ownership.

The payment card is required even if you plan to stay within free offers. AWS uses it for identity and billing verification, and charges can occur if you exceed Free Tier limits or create resources that are not free. The phone verification step proves that you control a reachable phone number. Depending on country, AWS may also ask for a billing address, tax information, or an additional verification step.

Definition

AWS Free Tier is a collection of offers, not a single unlimited free account. Some offers last for 12 months after account creation, some are Always Free within monthly limits, and some are short service-specific trials.

0.2.2 Understanding Free Tier Categories

AWS groups free usage into three practical categories:

- **12-month Free Tier:** introductory monthly allowances that expire 12 months after the account creation date. Common examples include limited compute, storage, database, and data-transfer allowances. When the 12-month period ends, normal pay-as-you-go pricing applies.
- **Always-Free offers:** recurring monthly allowances that do not expire as long as AWS continues the offer and you stay within limits. Examples vary by service and Region, so always check the service's current Free Tier page before a lab.
- **Trials:** service-specific promotions for a fixed time, fixed usage amount, or fixed credit. Trials may start when you first activate the service, not necessarily when you create the account.

Free Tier is measured by usage dimensions: GB-months stored, requests made, compute hours, data processed, messages sent, log ingestion, or data transferred out. A service can have one free dimension but charge for another. For example, a storage allowance may be free while requests, retrieval, replication, public IPv4 addresses, NAT gateways, or cross-Region transfer are billable.

Common Pitfall

Assuming Free Tier means no bill. Free Tier is a discount schedule with limits. Resources outside the offer, usage above the allowance, unsupported Regions, idle billable infrastructure, data egress, and forgotten resources can still create charges.

0.2.3 Sign-up Walkthrough

Start at the AWS sign-up page and choose **Create an AWS account**. Enter the email address that will own the account, an account name, and a strong password. Use an account name that is human-readable but not sensitive, such as **data-engineering-sandbox**. Avoid patient names, internal project codenames, confidential client names, or anything that would be embarrassing in a bill, screenshot, or support case.

Next, choose the account type. Individual learners usually choose a personal account. Company learners should follow their employer's cloud onboarding process instead, because many organizations require AWS Organizations, centralized identity, service control policies, centralized logging, and approved billing. Enter contact information, payment information, and the verification code sent through the phone flow. Choose the Basic support plan unless your organization requires a different plan.

After sign-up, wait for account activation email. Then sign in as the root user only long enough to secure the account and create an administrative identity. Record the account ID from the console menu; later commands and budget policies may need it. Do not create training resources yet. Security and cost guardrails come first.

Tip

Use a password manager from the beginning. Store the root email, password, account ID, MFA recovery codes or device recovery notes, and the name of the administrative profile you create later. Do not store access keys in notes, screenshots, or source files.

0.3 Securing the Account**0.3.1 Root User: Owner, Not Operator**

The root user has unrestricted authority over the account. It can change billing settings, close the account, alter account-level security settings, recover access, and perform actions that ordinary IAM users or roles may not need. That power makes root valuable for rare account ownership operations and dangerous for daily work.

Daily work should use federated identities, IAM Identity Center permission sets, IAM roles, or in a small learning account, an IAM administrator user. This separation gives you better audit trails, lets you rotate or revoke operational credentials without changing account ownership, and reduces blast radius if a laptop, browser session, or access key is compromised.

Common Pitfall

Leaving root unprotected. An AWS account with root sign-in and no MFA is a high-risk target. If an attacker obtains the root password, they can create users, disable controls, delete resources, change billing settings, and lock you out. Enable MFA on root before building anything.

0.3.2 Enable MFA on the Root User

Sign in to the AWS Management Console as the root user. Open the account menu, choose **Security credentials**, and find the MFA section. Add a virtual authenticator app, hardware security key, or another supported MFA device. Scan the QR code or register the key, enter consecutive MFA codes when prompted, and verify that the device appears as active.

A virtual authenticator is acceptable for a personal lab, but a hardware security key is stronger. If you use a phone app, think about device replacement: keep recovery information in a secure place and avoid a single point of failure. Do not share the root MFA device casually. For organizations, root MFA ownership should follow a documented break-glass process.

Note

After MFA is enabled, sign out and sign back in once to confirm that root sign-in requires the second factor. Then stop using root for routine labs.

0.3.3 Create an Administrative Identity

AWS has two common paths for a learning account. The preferred modern path is **IAM Identity Center**. The smaller, legacy-compatible path is an **IAM admin user and group**. Use IAM Identity Center if you can; use an IAM admin user only if your account or course environment does not support the Identity Center flow.

Preferred: IAM Identity Center

Open **IAM Identity Center** in the console. If prompted, enable it for the account. Create a user with your learner email address, assign that user to the AWS account, and attach an administrative permission set for the sandbox account. Accept the invitation email, set a password, register MFA for the workforce user if prompted, and sign in through the AWS access portal URL.

From the access portal, you can open the console or copy CLI configuration values. This flow avoids creating long-lived access keys on your laptop. It also resembles how many companies grant cloud access: users authenticate to a workforce identity system and receive temporary credentials for a role.

Alternative: IAM Admin User and Group

If IAM Identity Center is unavailable, create an IAM group such as **Administrators**, attach the AWS-managed **AdministratorAccess** policy, then create a user for daily work and add the user to that group. Give the user a console password only if you need console sign-in. Create access keys only for CLI work, rotate them periodically, and delete unused keys.

For a personal learning account, an admin user is acceptable as a temporary bootstrap identity. For team, enterprise, or production use, prefer federation and roles. Never create access keys for the root user.

0.3.4 Password Policy and Account Hygiene

In IAM account settings, configure a strong password policy: minimum length of at least 14 characters, uppercase, lowercase, number, symbol, expiration or rotation according to your organization's policy, and prevention of password reuse. Require MFA for privileged users. Add an alternate billing contact and security contact if the account is important enough that another human must receive urgent notices.

Finally, review the account’s security credential report or IAM access analyzer findings periodically. Delete unused users, deactivate unused access keys, and avoid embedding credentials in scripts. In later chapters, the safest pattern is to use short-lived role credentials from IAM Identity Center or CloudShell instead of long-lived local keys.

0.4 Installing and Configuring the Tooling

0.4.1 AWS CLI v2 on Windows

AWS CLI v2 is the command-line interface used throughout the book. On Windows, install the official MSI package from AWS, or install through an approved package manager if your organization manages software that way. Open a new PowerShell window after installation so `aws.exe` is on PATH. Verify the version before configuring credentials.

```
1 aws --version
2
3 # Expected shape; exact patch versions vary.
4 # aws-cli/2.x.x Python/3.x Windows/10 exe/AMD64 prompt/off
```

Listing 1: Verifying AWS CLI v2 from Windows PowerShell.

On macOS, AWS provides a signed installer package. On Linux, AWS provides a bundled installer archive. The commands in this book focus on AWS CLI behavior, not installer mechanics; use your operating system’s official AWS CLI v2 instructions when in doubt.

0.4.2 Configuration Option A: IAM Identity Center

If you created an IAM Identity Center user, configure the CLI with SSO. The command opens a browser so you can authorize the device. The profile name should be explicit because later labs may use multiple accounts or Regions.

```
1 aws configure sso --profile aws-book-admin
2
3 # Typical prompts:
4 # SSO session name: aws-book
5 # SSO start URL: https://your-start-url.awsapps.com/start
6 # SSO region: us-east-1
7 # CLI default client Region: us-east-1
8 # CLI default output format: json
9
10 aws sso login --profile aws-book-admin
11 aws sts get-caller-identity --profile aws-book-admin
```

Listing 2: Configuring an AWS CLI profile with IAM Identity Center.

SSO profiles use temporary credentials cached locally after login. When the session expires, run `aws sso login --profile aws-book-admin` again. This is the recommended path for the book because it reduces the risk of leaked static access keys.

0.4.3 Configuration Option B: Access Keys

If you use an IAM admin user, create an access key for that user and configure a named profile. Keep the secret access key private; AWS shows it only once. Do not paste it into documents, screenshots, notebooks, Git repositories, or chat messages.

```
1 aws configure --profile aws-book-admin
2
3 # AWS Access Key ID [None]: AKIA...
4 # AWS Secret Access Key [None]: paste-secret-here
```

```

5 # Default region name [None]: us-east-1
6 # Default output format [None]: json
7
8 aws sts get-caller-identity --profile aws-book-admin

```

Listing 3: Configuring an AWS CLI profile with access keys.

The command writes configuration under `%USERPROFILE%\.aws`, including the `config` and `credentials` files. Treat the credentials file as sensitive. If a key is exposed, deactivate it immediately in IAM and create a replacement only if still needed.

Common Pitfall

Using the default profile accidentally. If one terminal uses `default` and another uses `aws-book-admin`, commands may run in different accounts or Regions. Use `-profile` in examples until you are confident, or set `$Env:AWS_PROFILE = 'aws-book-admin'` for the current PowerShell session.

0.4.4 Profiles, Regions, and Output Formats

A profile combines credentials with defaults such as Region and output format. This book uses `aws-book-admin` as the learning profile and `us-east-1` as the default Region unless a chapter says otherwise. JSON output is preferred because it can be queried, stored, and compared in tests.

```

1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3 $Env:AWS_DEFAULT_REGION = 'us-east-1'
4
5 aws configure list
6 aws sts get-caller-identity

```

Listing 4: Selecting the book profile and Region for the current PowerShell session.

PowerShell environment variables set this way affect only the current terminal session. That is useful: you can keep one terminal pointed at a sandbox and avoid changing global settings. If you want persistent configuration, use `aws configure set region us-east-1 -profile aws-book-admin`.

0.4.5 AWS CloudShell as a No-install Alternative

AWS CloudShell is a browser-based shell available from the AWS console in supported Regions. It includes AWS CLI, credentials from your console session, persistent home storage within service limits, and common command-line tools. CloudShell is excellent when you cannot install software locally, need a quick verification, or want to avoid local credential files.

CloudShell is still tied to your signed-in identity and selected Region. Before running a lab, check the Region selector and run `aws sts get-caller-identity`. Store only disposable training files in CloudShell home storage. Do not treat it as a permanent development workstation.

Tip

CloudShell removes local installation friction, not security responsibility. You still need MFA, least privilege, budget alerts, cleanup, and careful Region selection.

0.5 Cost Guardrails to Stay Free

0.5.1 Turn on Billing Visibility

Open the Billing and Cost Management console as root or an identity with billing permissions. Activate IAM access to billing if needed, then use the administrative identity for routine billing checks. Enable Free Tier usage alerts where available and confirm the email address that receives notices. Billing alerts do not prevent spending; they shorten the time between activity and awareness.

Cost Explorer helps you see cost trends by service, Region, usage type, linked account, and tag. It may need to be enabled and can take time to populate. For a learning account, the most important dashboard questions are simple: which services have charges, in which Regions, and from which resources?

Tip

Measure it. At the start and end of every hands-on session, open the Billing dashboard and Cost Explorer. Record current month-to-date spend and the services with nonzero cost. If the number changed unexpectedly, stop and investigate before starting the next chapter.

0.5.2 Create a Zero-Spend or Low-Spend Budget

AWS Budgets can send email alerts when actual or forecasted spend crosses thresholds. A zero-spend budget is useful in a new account because any charge matters. A low-spend budget, such as \$5 or \$10 per month, is practical if you expect tiny unavoidable charges while learning. The budget is a notification guardrail, not a hard spending cap.

Create at least two alerts: one for actual spend and one for forecasted spend. Actual spend tells you that charges have occurred. Forecasted spend warns that current usage trends may exceed your limit by the end of the month. Use an email address you actively read.

```

1 {
2   "BudgetName": "aws-book-low-spend",
3   "BudgetLimit": {
4     "Amount": "5.00",
5     "Unit": "USD"
6   },
7   "TimeUnit": "MONTHLY",
8   "BudgetType": "COST",
9   "CostFilters": {},
10  "CostTypes": {
11    "IncludeTax": true,
12    "IncludeSubscription": true,
13    "UseBlended": false,
14    "IncludeRefund": false,
15    "IncludeCredit": true,
16    "IncludeUpfront": true,
17    "IncludeRecurring": true,
18    "IncludeOtherSubscription": true,
19    "IncludeSupport": true,
20    "IncludeDiscount": true,
21    "UseAmortized": false
22  }
23 }
```

Listing 5: Minimal monthly AWS budget definition for a low-spend learning guardrail.

The following PowerShell example creates the JSON files and calls AWS Budgets. Replace the email address and account ID. Budgets is a global billing service, but the CLI still needs a configured profile.

```

1 $AccountId = '123456789012'
2 $Email = 'you@example.com'
3
4 @'
5 {
6   "BudgetName": "aws-book-low-spend",
7   "BudgetLimit": { "Amount": "5.00", "Unit": "USD" },
8   "TimeUnit": "MONTHLY",
9   "BudgetType": "COST",
10  "CostTypes": {
11    "IncludeTax": true,
12    "IncludeSubscription": true,
13    "UseBlended": false,
14    "IncludeRefund": false,
15    "IncludeCredit": true,
16    "IncludeUpfront": true,
17    "IncludeRecurring": true,
18    "IncludeOtherSubscription": true,
19    "IncludeSupport": true,
20    "IncludeDiscount": true,
21    "UseAmortized": false
22  }
23 }
24 '@ | Set-Content -Path .\budget.json -Encoding utf8
25
26 @"
27 [
28   {
29     "Notification": {
30       "NotificationType": "ACTUAL",
31       "ComparisonOperator": "GREATER_THAN",
32       "Threshold": 1,
33       "ThresholdType": "PERCENTAGE"
34     },
35     "Subscribers": [
36       { "SubscriptionType": "EMAIL", "Address": "$Email" }
37     ]
38   },
39   {
40     "Notification": {
41       "NotificationType": "FORECASTED",
42       "ComparisonOperator": "GREATER_THAN",
43       "Threshold": 80,
44       "ThresholdType": "PERCENTAGE"
45     },
46     "Subscribers": [
47       { "SubscriptionType": "EMAIL", "Address": "$Email" }
48     ]
49   }
50 ]
51 "@ | Set-Content -Path .\notifications.json -Encoding utf8
52
53 aws budgets create-budget '
54   --account-id $AccountId '
55   --budget file://budget.json '
56   --notifications-with-subscribers file://notifications.json

```

Listing 6: Creating an AWS Budget with actual and forecasted email alerts.

After creating the budget, confirm the subscription email if AWS sends a confirmation message.

Test that the budget appears in the console. If you later close or replace the account, recreate budgets in the new account; budgets are not portable across accounts automatically.

0.5.3 Free Tier Usage Alerts and Service-specific Controls

Free Tier usage alerts warn when usage approaches Free Tier limits for eligible services. Enable them from the Billing preferences page. They complement budgets because some Free Tier dimensions are usage-based rather than cost-based. For example, a service may remain at \$0 until the allowance is exceeded, then begin charging.

Some services also have their own limits, alarms, or quotas. S3 lifecycle expiration can delete temporary objects. CloudWatch alarms can detect unexpected traffic. Service Quotas can request or view quotas, but quotas are not a substitute for cleanup. For this book, the default policy is: create only what the lab requires, tag it, verify it, and delete it when the exercise says to delete it.

Common Pitfall

Relying on budgets as a kill switch. AWS Budgets sends notifications and can trigger actions in some scenarios, but it does not universally stop resource creation or halt all charges. Design labs so cleanup is explicit and cheap resources do not depend on expensive hidden infrastructure.

0.6 Choosing a Region and Naming Conventions

0.6.1 Default Region for the Book

Unless a chapter states otherwise, use `us-east-1`. It supports broad service coverage, keeps examples consistent, and is commonly used in public documentation. If your organization mandates another Region, use that Region consistently and change names accordingly. Be aware that service availability, pricing, quotas, and Free Tier eligibility can vary by Region.

A Region is not just a performance choice. It affects data residency, regulatory posture, disaster recovery design, KMS key location, networking, and service integration. For personal learning, consistency matters more than geographic perfection. For professional environments, Region selection should be a documented architecture decision.

0.6.2 Naming Convention

Resource names should be readable, unique where required, and safe to show in screenshots. This book uses the pattern:

`aws-book-<service>-<purpose>-<region>-<suffix>`

For globally unique names such as S3 buckets, append a short suffix such as your account ID fragment, date, or random string. Avoid uppercase letters in S3 bucket names. Avoid names that reveal sensitive data. Use hyphens rather than underscores for resource names unless the service requires otherwise.

Examples:

- `aws-book-s3-landing-us-east-1-1234`
- `aws-book-glue-demo-us-east-1`
- `aws-book-budget-low-spend`

0.6.3 Tagging Convention

Tags help you find, filter, allocate cost, and clean up resources. Not every AWS resource supports tags, but when tags are available, apply a consistent minimum set:

Tag key	Example value	Purpose
Project	AWSBook	Groups all book resources for search and cost review.
Environment	Training	Distinguishes labs from production, development, or shared sandboxes.
Owner	learner	Identifies who should clean up the resource. Do not put sensitive personal data here.
Chapter	Chapter0	Makes exercise resources easy to find and delete.
DeleteAfter	2026-08-31	Documents intended cleanup date. It is not automatic unless automation enforces it.

Table 1: Minimum training tags used throughout this book.

Tip

Use tags in the Billing dashboard and Cost Explorer. If a charge appears and the resource has no project or chapter tag, you have already found a process gap.

0.7 Setup Flow Across the Six Platform Books

Cross-Cloud Equivalence

The first hour in every cloud book follows the same pattern: understand the free offer, secure the administrative identity, install or open the shell, set cost guardrails, and verify the environment.

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (aws configure)	Azure CLI (az login)	gcloud CLI (gcloud init)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console

The three non-object-store books differ: Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

0.8 Verification: Hello World

0.8.1 Check the Caller Identity

The safest first command is read-only: `aws sts get-caller-identity`. It returns the AWS account, user or role ARN, and user ID for the active credentials. Run it before every lab, especially if you work with multiple profiles.

```

1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3
4 aws sts get-caller-identity
5
6 # Expected output shape:
7 # {
8 #     "UserId": "AROEXAMPLEID:learner",
9 #     "Account": "123456789012",
10 #     "Arn": "arn:aws:sts::123456789012:assumed-role/
11     AWSReservedSSO_AdministratorAccess_abc123/learner"
12 # }
```

Listing 7: Confirming which AWS identity the CLI is using.

If the account ID is not the account you expect, stop. Do not continue a hands-on lab in the wrong account. If the ARN shows root, stop and switch to your administrative identity.

0.8.2 Create, List, and Delete a Small S3 Bucket

This smoke test proves that credentials, Region, permissions, and cleanup all work. It creates an empty S3 bucket, lists it, and deletes it. S3 bucket names are globally unique, so the example constructs a suffix from your account ID and the current date. If the name already exists, change the suffix.

```

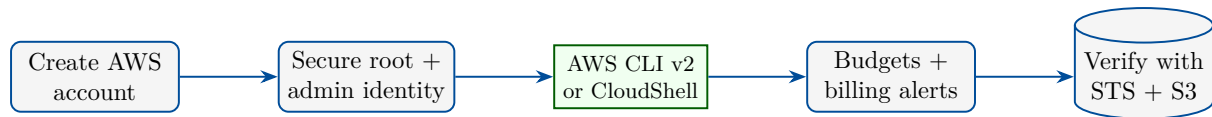
1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3 $AccountId = (aws sts get-caller-identity --query Account --output text)
4 $Suffix = Get-Date -Format 'yyyyMMddHHmmss'
5 $Bucket = "aws-book-chapter0-$AccountId-$Suffix"
6
7 aws s3api create-bucket --bucket $Bucket --region $Env:AWS_REGION
8 aws s3api put-bucket-tagging '
9     --bucket $Bucket '
10     --tagging 'TagSet=[{Key=Project,Value=AWSBook},{Key=Environment,Value=
11     Training},{Key=Chapter,Value=Chapter0}]'
12
13 aws s3 ls "s3://$Bucket"
14
15 aws s3api list-buckets --query "Buckets[?Name=='$Bucket'].Name" --output text
16
17 aws s3api delete-bucket --bucket $Bucket
18
19 aws s3api list-buckets --query "Buckets[?Name=='$Bucket'].Name" --output text
```

Listing 8: S3 smoke test with cleanup from Windows PowerShell.

Expected behavior: `create-bucket` succeeds, `aws s3 ls` returns no objects because the bucket is empty, `list-buckets` prints the bucket name before deletion, and the final `list-buckets` prints nothing after deletion. If deletion fails with `BucketNotEmpty`, remove objects and versions first; this smoke test should not create any objects.

Note

S3's `create-bucket` syntax differs for `us-east-1` and other Regions. In `us-east-1`, the simple command above is valid. In most other Regions, include `-create-bucket-configuration LocationConstraint=<region>`.

0.8.3 Optional Setup Flow Diagram

Build guardrails before creating data-engineering resources.

Figure 1: Chapter 0 setup flow: account, secure identity, tooling, cost guardrail, and verification.

0.9 Further Reading

Review the AWS account creation, Free Tier, IAM, IAM Identity Center, AWS CLI v2, CloudShell, Billing, Budgets, and Cost Explorer documentation from AWS. Keep those pages close while working through the first labs because console navigation and Free Tier offers can change. For security posture, also read AWS guidance on the root user, MFA, least privilege, and the shared-responsibility model. For cost control, read the current pricing page for each service before running a chapter that creates resources.

Key Takeaways

- Free Tier is a set of limited offers; it is not a guarantee that all account activity is free.
- Protect the root user with MFA and use a separate administrative identity for daily labs.
- IAM Identity Center and `aws configure sso` are preferred because they use temporary credentials.
- Named profiles and a consistent Region prevent many accidental-account and wrong-Region mistakes.
- Budgets, Free Tier alerts, Cost Explorer, and routine Billing dashboard checks are required learning-account hygiene.
- Tags, naming conventions, and cleanup commands make hands-on work auditable and reversible.
- A successful `sts get-caller-identity` plus S3 create, list, and delete test proves that Chapter 1 can start safely.

0.10 Exercises

1. **(Checklist)** Complete this environment setup checklist before Chapter 1:
 - AWS account created and account ID recorded.
 - Root user password stored securely and root MFA enabled.

- Daily administrative identity created through IAM Identity Center or an IAM admin user and group.
 - Administrative identity can sign in to the console without using root.
 - AWS CLI v2 installed locally or AWS CloudShell selected as the working shell.
 - Named profile `aws-book-admin` configured, or CloudShell identity verified.
 - Default Region set to `us-east-1` unless your organization requires another Region.
 - Billing access verified, Free Tier usage alerts enabled where available, and an AWS Budget created.
 - Cost Explorer enabled or checked for availability.
 - `aws sts get-caller-identity` returns the expected account and non-root identity.
 - S3 smoke-test bucket created, listed, deleted, and confirmed absent.
2. **(Conceptual)** Explain the difference between the root user, an IAM user, an IAM role, and an IAM Identity Center user. Which should you use for daily labs and why?
 3. **(Cost)** Create a monthly budget with actual and forecasted alerts. Record the budget name, threshold, email recipient, and the date you verified it in the console.
 4. **(CLI)** Run `aws configure list` and `aws sts get-caller-identity`. Save the account ID and ARN in your private lab notes, not in a public repository.
 5. **(Region)** Choose a Region other than `us-east-1`. Identify one service from later chapters that is available there and one pricing or availability difference you would check before using it.
 6. **(Governance)** Propose a tag policy for a shared training account. Include owner, project, environment, chapter, and cleanup-date fields.
 7. **(Cleanup)** Search the console for resources tagged `Project=AWSBook`. Confirm that no Chapter 0 resources remain after the S3 smoke test.

Part I

Core Data Infrastructure and Storage

Amazon S3 and Storage Foundations

Learning Objectives

- Explain Amazon S3’s object-storage architecture, namespace model, durability model, availability model, and strong consistency guarantees.
- Compare S3 Standard, Standard-IA, One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive for data engineering workloads.
- Design lifecycle, versioning, Object Lock, legal hold, and replication policies that match retention, recovery, and compliance requirements.
- Choose among AWS DataSync, Storage Gateway, and the AWS Snow Family for online, hybrid, and offline migration scenarios.
- Deploy a bucket with the AWS CLI, configure a 30-day transition to Glacier Flexible Retrieval, and connect S3 event notifications to Amazon SQS.
- Design a multi-region healthcare disaster-recovery storage architecture for 10 PB of medical-imaging data under HIPAA-style constraints.

Cross-Cloud Equivalence

Although this book teaches **AWS**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool / Cold	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication (GRS)	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider’s object store, and **MongoDB’s** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

1.1 Week 1 Roadmap and Mental Model

Amazon Simple Storage Service (Amazon S3) is the base layer of most AWS data engineering architectures. It is the landing zone for raw extracts, the durable system of record for data lakes, the staging layer for Redshift and EMR, the archive tier for infrequently accessed records, and the integration point for event-driven pipelines. The service looks simple because its core API is small: create buckets, put objects, get objects, list keys, attach metadata, and apply policies. The engineering challenge is not memorizing API names; it is matching S3’s storage semantics to data lifecycle, governance, cost, and reliability goals [10, 2].

A useful starting model is that S3 separates the concerns usually bundled inside a file system. The *bucket* is a regional administrative container. The *object key* is a string used for lookup and organization. The *object body* is immutable for a given version. Metadata, tags, access policies, encryption settings, lifecycle rules, event notifications, and replication rules sit around those objects as managed controls. Data engineers use those controls to turn inexpensive object storage into a governed analytical platform.

Definition

Amazon S3 is a regional object storage service. It stores data as objects inside buckets, exposes API-based access rather than block-device semantics, and provides features for access control, encryption, lifecycle management, replication, retention, and event notification.

Key Terms

Object: bytes plus metadata, tags, and a key. **Bucket:** regional namespace and policy boundary. **Prefix:** leading portion of a key used for organization and request distribution. **Version:** immutable historical instance of an object when versioning is enabled. **Storage class:** economic and operational placement for an object.

1.2 Monday: S3 Architecture and Storage Classes

1.2.1 Object Storage, Not a Shared File System

S3 is not POSIX storage. It does not expose in-place mutation, directory locks, or append semantics. A key such as `raw/xray/2026/07/30/study-001.dcm` may look like a path, but S3 treats it as a key string. Console folders are a user-interface convention based on delimiters. This distinction matters for Spark, Glue, Athena, and Redshift Spectrum because those engines discover data by listing prefixes and reading objects rather than traversing a local file system.

For data engineering, the non-POSIX model is a strength. It allows S3 to store very large numbers of objects, serve high request rates, decouple compute from storage, and become the durable substrate for a lakehouse architecture [30]. It also means writers must use commit protocols and table formats carefully. A failed writer cannot rely on renaming a temporary directory atomically in the way a local file system might.

Common Pitfall

Treating S3 like a local file system. Directory renames, small-file-heavy workloads, and append-in-place assumptions can create slow listings, expensive request patterns, and broken table commits. Design for object immutability, partition-aware prefixes, and idempotent writers.

1.2.2 Regional Buckets and Global Names

Each S3 bucket has a globally unique name and belongs to one AWS Region. Objects stay in that Region unless replication, transfer, or client code moves them. This regional boundary is a compliance and architecture decision: it affects latency, data residency, KMS key choice, disaster recovery, and downstream service placement. A bucket name should be stable and descriptive, but not leak sensitive data; for example, use `acme-health-imaging-prod-us-east-1` rather than patient or project names.

Note

Strong service durability does not remove the need for architecture-level recovery design. Region choice, replication, versioning, backup policy, and access controls are still your responsibility under the AWS shared-responsibility model.

1.2.3 Durability and Availability

AWS designs S3 for extremely high durability by redundantly storing objects across multiple Availability Zones in a Region for most storage classes [10]. Durability is the probability that stored data is not lost. Availability is the probability that a request can be served when needed. These are related but different: an object can remain durably stored even during an availability impairment that prevents a client from reading it immediately.

Definition

Durability measures resistance to data loss over time. **Availability** measures readiness to serve requests at a point in time. S3 storage-class selection must consider both, plus retrieval latency, minimum storage duration, retrieval fees, and resilience to Availability Zone loss.

Storage class	Primary use	Access pattern	Design caution
S3 Standard	Active data lakes, landing zones, analytics staging	Frequent, low-latency access	Highest storage price among listed classes; still usually right for hot data
S3 Standard-IA	Durable multi-AZ storage for infrequent access	Millisecond access, lower storage cost	Retrieval fee and minimum storage duration make it poor for short-lived data
S3 One Zone-IA	Re-creatable infrequent data	Millisecond access in one AZ	Lower resilience; do not use for the only copy of critical data
S3 Glacier Instant Retrieval	Rarely accessed archive needing milliseconds	Millisecond archive access	Retrieval pricing and minimum duration require careful modeling
S3 Glacier Flexible Retrieval	Long-lived archive with minutes-to-hours retrieval	Expedited, standard, or bulk retrieval	Not appropriate for interactive analytics
S3 Glacier Deep Archive	Lowest-cost deep archive	Hours-scale retrieval	Retrieval planning and restore operations must be operationalized

Table 1.1: Storage-class choice as a workload and risk decision, not merely a price decision.

1.2.4 Consistency Model

Modern S3 provides strong read-after-write consistency for puts and deletes of objects, and strong consistency for list operations after writes [1]. If a successful PUT returns, a subsequent GET or LIST should observe that change. This simplifies data-lake pipelines compared with older patterns that inserted sleep intervals or extra metadata stores purely to compensate for eventual list behavior.

Tip

Strong consistency simplifies pipeline correctness, but it does not make multi-object transactions automatic. If a job writes 10,000 objects and fails halfway, S3 will consistently show the partial result. Table formats, manifests, job bookmarks, and idempotent orchestration still matter.

1.3 Choosing a Storage Class for Data Engineering

Storage class is a lifecycle attribute. It should follow the data’s access pattern and risk profile over time. A common analytical object might begin in S3 Standard while ingestion, validation, profiling, and downstream consumption are active. After 30 or 90 days, it might transition to Standard-IA or Glacier Instant Retrieval. After a legal or business retention threshold, it might move to Glacier Flexible Retrieval or Deep Archive. For a reproducible intermediate dataset, One Zone-IA may be acceptable; for medical records or financial records, it usually is not.



Cost decreases left-to-right, but retrieval latency, retrieval fees, and operational planning generally increase.

Figure 1.1: A storage-class lifecycle path for analytical and archival objects. Actual thresholds must be justified by access data, compliance needs, and retrieval objectives.

1.4 Tuesday: Lifecycle, Versioning, Object Lock, and Replication

1.4.1 Lifecycle Policies

Lifecycle rules automate transitions and expirations. A rule can target a bucket, prefix, tag, object size, current versions, noncurrent versions, incomplete multipart uploads, or delete markers. In a data lake, lifecycle rules are often applied by zone. Raw data might be retained longer than curated derivatives. Temporary job output might expire after a few days. Audit logs might transition to archive but never be deleted until the retention period ends.

Note

Lifecycle is not a backup strategy by itself. Expiration rules delete data intentionally. Use versioning, replication, backup policy, and retention controls for recovery from accidental deletion or malicious overwrite.

1.4.2 Versioning

S3 Versioning keeps multiple variants of an object under the same key. A normal delete creates a delete marker rather than immediately destroying prior versions. Versioning is a foundation for recovery, replication, and Object Lock. It is also a cost multiplier when pipelines rewrite large partitions repeatedly. Data engineers should pair versioning with lifecycle rules for noncurrent versions, monitoring, and explicit restore procedures.

Common Pitfall

Enabling versioning without noncurrent-version lifecycle. A job that repeatedly overwrites a 500 GB partition can silently retain every historical version. This may be exactly what compliance requires, but if it is accidental, storage cost will grow quickly.

1.4.3 Object Lock: Governance, Compliance, and Legal Hold

S3 Object Lock provides write-once-read-many controls for object versions when the bucket is created with Object Lock support and versioning is enabled [24]. In governance mode, privileged

users with specific permissions can bypass retention. In compliance mode, protected object versions cannot be overwritten or deleted by any user, including the root account, until retention expires. Legal hold is an independent hold flag that prevents overwrite or deletion until removed by an authorized principal.

Definition

Governance mode protects objects while allowing explicitly authorized bypass for administrative workflows. **Compliance mode** prevents deletion or overwrite until the retention date expires. **Legal hold** has no fixed expiration date; it remains until an authorized user removes it.

Tip

For regulated datasets, design Object Lock before ingestion. Some settings must be enabled at bucket creation, and compliance mode is intentionally difficult to undo. Test policies in a non-production account before applying retention to irreplaceable data.

1.4.4 S3 Replication: SRR and CRR

S3 Replication copies objects automatically according to rules [25]. Same-Region Replication (SRR) is useful for account separation, log aggregation, data sovereignty within a Region, and different ownership or encryption boundaries. Cross-Region Replication (CRR) is used for disaster recovery, lower-latency regional access, regulatory isolation, or multi-region analytical platforms. Replication is asynchronous, so designs must define a recovery point objective (RPO), monitor replication lag, and decide whether delete markers and existing objects are included.

Feature	Use it when	Question to ask in review
Lifecycle transition	Access declines predictably	What retrieval objective and minimum-duration charge apply?
Lifecycle expiration	Data has a clear disposal date	Who approved deletion and how is it audited?
Versioning	Accidental overwrite or delete must be recoverable	How are noncurrent versions aged and monitored?
Object Lock	WORM retention is required	Governance or compliance mode, and who can bypass?
Legal hold	Litigation or investigation requires indefinite hold	What process removes the hold and records approval?
SRR	Same-region separation is needed	Is the target account, owner, and KMS key correct?
CRR	Regional disaster recovery is needed	What RPO, RTO, encryption, and failover process are tested?

Table 1.2: S3 management features mapped to operational review questions.

1.5 Wednesday: Migration and Hybrid Storage

1.5.1 AWS DataSync

AWS DataSync is a managed online transfer service for moving data between on-premises storage, edge locations, other clouds, and AWS storage services such as S3, EFS, and FSx [13]. It is appropriate when a network path exists and ongoing or incremental synchronization is needed. DataSync handles parallelization, verification, scheduling, bandwidth limits, and task

reporting. For data engineers, it is often the cleanest path for moving existing file shares into S3 landing zones while preserving metadata required by downstream processing.

1.5.2 AWS Storage Gateway

AWS Storage Gateway is a hybrid cloud storage service. File Gateway presents file protocols backed by S3. Volume Gateway presents block storage with cloud-backed snapshots. Tape Gateway presents a virtual tape library for backup applications [16]. Storage Gateway is not merely a migration tool; it is a bridge for workloads that cannot immediately be rewritten to native object APIs.

Definition

File Gateway maps NFS or SMB file access to S3 objects. **Volume Gateway** provides iSCSI block volumes with snapshots. **Tape Gateway** emulates tape workflows for backup software while storing virtual tapes in AWS.

1.5.3 AWS Snow Family

The AWS Snow Family supports offline or edge transfer when networks are too slow, too expensive, unreliable, or unavailable [15]. Snowcone is a small rugged device for edge and limited transfer. Snowball Edge supports larger transfer and edge compute. Snowmobile is a truck-scale option for extremely large migrations. Offline migration does not remove data-engineering responsibilities: you still need inventory, chain of custody, encryption, verification, cutover planning, and reconciliation.

Migration option	Best fit	Trade-off
DataSync	Online, repeatable transfer with available bandwidth	Requires network connectivity; large first loads may still take time
File Gateway	Applications need NFS/SMB while AWS stores objects	File semantics can hide object-layout choices; not a substitute for native data-lake design
Volume Gateway	Block workloads and snapshot workflows	Less natural for analytical object processing
Tape Gateway	Existing backup software expects tapes	Good transition path; not an analytical ingestion pattern
Snowcone	Edge sites, small rugged transfers, constrained environments	Operational handling and shipping time
Snowball Edge	Large offline transfer or edge processing	Requires device logistics, job planning, validation
Snowmobile	Massive data-center-scale migration	Specialized engagement; heavy operational planning

Table 1.3: Migration and hybrid-storage options for AWS data engineering foundations.

Tip

Estimate migration time before choosing a service. A 10 PB migration over a 1 Gbps link can take years after overhead and contention. Offline transfer may be the only practical first-load path, followed by DataSync or application replication for deltas.

1.6 Thursday Hands-on Project: Bucket, Lifecycle, and Events

This lab creates an S3 bucket, enables versioning, applies a 30-day transition to S3 Glacier Flexible Retrieval, creates an SQS queue, and configures S3 event notifications. It is intentionally CLI-first so that the same actions can be automated later in CloudFormation, CDK, Terraform, or CI/CD. Replace names, account IDs, and Regions before running.

Note

The commands below create billable AWS resources. Use a sandbox account, set a unique bucket name, and delete resources after testing. Bucket names are globally unique; append your initials, account alias, or a random suffix.

```

1 # PowerShell users can run these commands from a shell with AWS CLI v2
   configured.
2 # Replace the values before running.
3 REGION="us-east-1"
4 ACCOUNT_ID="123456789012"
5 BUCKET="aws-master-data-engineer-week1-123456789012"
6 QUEUE="s3-ingest-events-week1"
7
8 aws s3api create-bucket \
9   --bucket "$BUCKET" \
10  --region "$REGION"
11
12 aws s3api put-bucket-versioning \
13   --bucket "$BUCKET" \
14   --versioning-configuration Status=Enabled
15
16 cat > lifecycle.json <<'JSON'
17 {
18   "Rules": [
19     {
20       "ID": "transition-raw-objects-to-glacier-after-30-days",
21       "Status": "Enabled",
22       "Filter": { "Prefix": "raw/" },
23       "Transitions": [
24         { "Days": 30, "StorageClass": "GLACIER" }
25       ],
26       "NoncurrentVersionTransitions": [
27         { "NoncurrentDays": 30, "StorageClass": "GLACIER" }
28       ],
29       "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
30     }
31   ]
32 }
33 JSON
34
35 aws s3api put-bucket-lifecycle-configuration \
36   --bucket "$BUCKET" \
37   --lifecycle-configuration file:///lifecycle.json
38
39 QUEUE_URL=$(aws sqs create-queue \
40   --queue-name "$QUEUE" \
41   --attributes VisibilityTimeout=60 \
42   --query QueueUrl \
43   --output text)
44
45 QUEUE_ARN=$(aws sqs get-queue-attributes \
46   --queue-url "$QUEUE_URL" \
47   --attribute-names QueueArn \
48   --query 'Attributes.QueueArn' \

```

```
49 --output text)
```

Listing 1.1: AWS CLI setup for an S3 bucket with versioning, lifecycle transition, SQS queue, and S3 event notification.

```
1 cat > sqs-policy.json <<JSON
2 {
3   "Version": "2012-10-17",
4   "Statement": [
5     {
6       "Sid": "AllowS3ToSendObjectCreatedEvents",
7       "Effect": "Allow",
8       "Principal": { "Service": "s3.amazonaws.com" },
9       "Action": "sqs:SendMessage",
10      "Resource": "$QUEUE_ARN",
11      "Condition": {
12        "ArnLike": { "aws:SourceArn": "arn:aws:s3:::$BUCKET" },
13        "StringEquals": { "aws:SourceAccount": "$ACCOUNT_ID" }
14      }
15    }
16  ]
17 }
18 JSON
19
20 aws sqs set-queue-attributes \
21   --queue-url "$QUEUE_URL" \
22   --attributes Policy="$(cat sqs-policy.json)"
23
24 cat > notification.json <<JSON
25 {
26   "QueueConfigurations": [
27     {
28       "Id": "send-raw-object-created-events-to-sqs",
29       "QueueArn": "$QUEUE_ARN",
30       "Events": ["s3:ObjectCreated:*"],
31       "Filter": {
32         "Key": {
33           "FilterRules": [
34             { "Name": "prefix", "Value": "raw/" }
35           ]
36         }
37       }
38     }
39   ]
40 }
41 JSON
42
43 aws s3api put-bucket-notification-configuration \
44   --bucket "$BUCKET" \
45   --notification-configuration file://notification.json
46
47 printf 'study_id,modality\n001,XRAY\n' > sample.csv
48 aws s3 cp sample.csv "s3://$BUCKET/raw/xray/sample.csv"
49 aws sqs receive-message --queue-url "$QUEUE_URL" --max-number-of-messages 1
```

Listing 1.2: Continuing the AWS CLI event-notification configuration.

Common Pitfall

Copying Bash syntax directly into PowerShell. The listing uses portable shell conventions for compactness. In native PowerShell, assign variables with `$REGION = 'us-east-1'` and create JSON files with here-strings. The AWS CLI calls are the important part.

The lifecycle JSON in listing 1.3 is the minimal policy extracted from the lab. It transitions current and noncurrent objects under `raw/` after 30 days and aborts incomplete multipart uploads after seven days.

```

1 {
2   "Rules": [
3     {
4       "ID": "transition-raw-objects-to-glacier-after-30-days",
5       "Status": "Enabled",
6       "Filter": { "Prefix": "raw/" },
7       "Transitions": [
8         { "Days": 30, "StorageClass": "GLACIER" }
9       ],
10      "NoncurrentVersionTransitions": [
11        { "NoncurrentDays": 30, "StorageClass": "GLACIER" }
12      ],
13      "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
14    }
15  ]
16 }

```

Listing 1.3: Lifecycle configuration for a 30-day transition to S3 Glacier Flexible Retrieval.

1.6.1 Boto3 Verification and Event Reader

The Python snippet below checks versioning, lifecycle rules, and SQS events. In production, this logic would belong in integration tests or deployment verification, not in an ad hoc notebook.

```

1 import json
2 import os
3
4 import boto3
5
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 bucket = os.environ["BUCKET"]
8 queue_url = os.environ["QUEUE_URL"]
9
10 s3 = boto3.client("s3", region_name=region)
11 sqs = boto3.client("sqs", region_name=region)
12
13 versioning = s3.get_bucket_versioning(Bucket=bucket)
14 assert versioning.get("Status") == "Enabled", versioning
15
16 lifecycle = s3.get_bucket_lifecycle_configuration(Bucket=bucket)
17 rules = {rule["ID"]: rule for rule in lifecycle["Rules"]}
18 rule = rules["transition-raw-objects-to-glacier-after-30-days"]
19 assert rule["Transitions"][0]["Days"] == 30
20 assert rule["Transitions"][0]["StorageClass"] == "GLACIER"
21
22 s3.put_object(
23     Bucket=bucket,
24     Key="raw/xray/boto3-smoke-test.txt",
25     Body=b"week 1 smoke test\n",
26     ContentType="text/plain",
27 )
28
29 messages = sqs.receive_message(
30     QueueUrl=queue_url,
31     MaxNumberOfMessages=1,
32     WaitTimeSeconds=10,
33 ).get("Messages", [])
34
35 if not messages:

```

```

36     raise RuntimeError("No S3 event arrived in SQS within 10 seconds")
37
38 body = json.loads(messages[0]["Body"])
39 record = body["Records"][0]
40 print(record["eventName"], record["s3"]["object"]["key"])

```

Listing 1.4: Boto3 verification for S3 lifecycle and SQS event delivery.

1.7 Friday Use Case: Healthcare Disaster-Recovery Storage Architecture

A healthcare provider processes 10 PB of X-ray images. The platform must ingest studies from hospitals, preserve original DICOM objects, support analytics and ML labeling, survive a regional outage, and maintain HIPAA-style controls for encryption, access, audit, retention, and deletion. The storage architecture should be conservative: raw medical images are irreplaceable, regulated, and expensive to reacquire.

1.7.1 Requirements and Assumptions

Assume the primary Region is `us-east-1` and the recovery Region is `us-west-2`. Images arrive from hospitals through private network links, DataSync agents, and occasional Snowball Edge jobs for backfills. The provider requires server-side encryption with customer-managed KMS keys, least-privilege access, audit logs, Object Lock for raw images, CRR for regional recovery, lifecycle transition for cost control, and a tested failover runbook. The target RPO is measured in minutes for online ingestion and in the shipping interval for offline backfills. The RTO depends on downstream processing, not only S3 object availability.

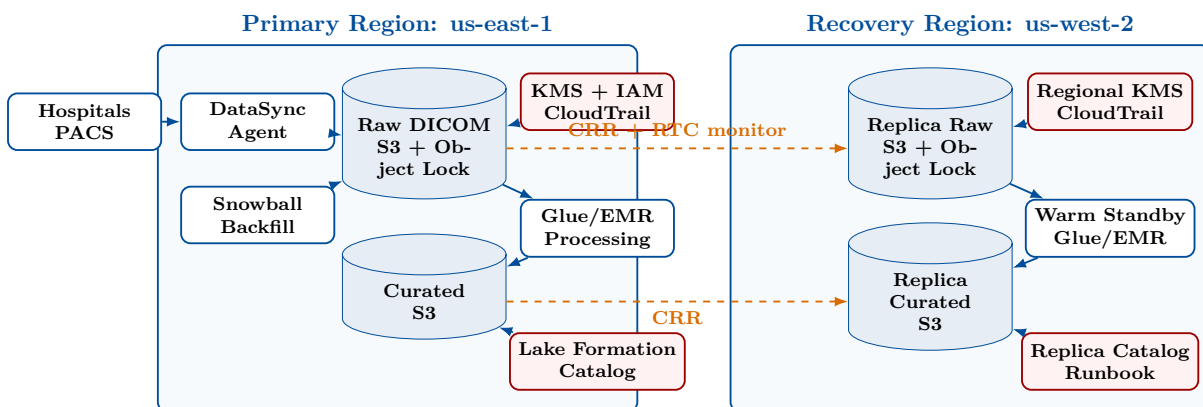


Figure 1.2: A multi-region compliant disaster-recovery storage architecture for 10 PB of healthcare imaging data. Replication protects regional availability; Object Lock, KMS, IAM, Lake Formation, and CloudTrail support compliance controls.

1.7.2 Architecture Decisions

Landing and retention. Raw DICOM images land in a primary S3 bucket with versioning and Object Lock enabled. Compliance mode may be justified for legally mandated immutable retention, but governance mode may be safer during early rollout if the organization still needs an authorized break-glass path. Legal holds are reserved for investigations or litigation.

Encryption and key management. Buckets require server-side encryption with AWS KMS customer-managed keys. Primary and replica Regions use separate regional keys. Key policies

are limited to ingestion roles, processing roles, security administrators, and replication roles. CloudTrail data events are enabled for sensitive buckets and delivered to a separate log archive account.

Replication and recovery. CRR copies raw and curated objects to the recovery Region. The replication role must be permitted to use both source and destination KMS keys. Replication metrics and alarms track lag. The failover runbook includes DNS or application endpoint changes, catalog validation, processing job activation, and a reconciliation process for objects written near the outage window.

Lifecycle and cost. Raw images remain in S3 Standard during active diagnosis, annotation, and model training windows. After an access-based threshold, they transition to Glacier Instant Retrieval or Glacier Flexible Retrieval depending on clinical retrieval requirements. Deep Archive is considered only for records with hours-scale retrieval objectives. Curated analytical derivatives may have shorter retention because they can be regenerated from locked raw data.

Ingestion at 10 PB scale. Online hospital feeds use DataSync when network connectivity and scheduling are feasible. Historical backfills use Snowball Edge jobs to avoid impractical transfer windows. All transfer paths write manifests, checksums, and reconciliation reports to S3 so missing studies can be identified before source systems are decommissioned.

Common Pitfall

Replicating data without replicating operations. A second-region bucket is not a disaster-recovery plan by itself. You must also replicate or recreate KMS access, IAM roles, Glue Catalog metadata, Lake Formation grants, monitoring, event routing, runbooks, and operator training.

1.8 Interview Q&A

1.8.1 Concepts and Trade-offs

- Q: Why is S3 a better foundation for a data lake than local disks attached to a cluster?**
A: S3 decouples durable storage from ephemeral compute, allows multiple engines to read the same data, and supports lifecycle, replication, encryption, and policy controls independently of a cluster.
- Q: When would you avoid One Zone-IA?**
A: Avoid it for the only copy of critical or regulated data because it stores data in one Availability Zone. It is better for re-creatable derivatives or secondary copies.
- Q: What does strong consistency solve, and what does it not solve?**
A: It ensures completed object writes and lists are visible immediately. It does not create multi-object transactions or repair failed jobs that produce partial datasets.
- Q: Governance mode or compliance mode for Object Lock?**
A: Governance mode allows authorized bypass and is useful when administrative exception processes are required. Compliance mode is stronger WORM retention and should be used only when the retention policy is certain.
- Q: DataSync or Snowball for 10 PB?**
A: Snowball or a specialized Snow Family approach is usually required for initial bulk transfer unless dedicated bandwidth is enormous. DataSync can handle ongoing deltas afterward.

1.9 Further Reading

Start with the Amazon S3 User Guide for bucket, object, lifecycle, encryption, event, and replication behavior [10]. Review AWS guidance on S3 storage classes before setting transitions [28], and study strong consistency semantics for pipeline correctness [1]. For retention design, read S3 Object Lock documentation [24]; for disaster-recovery copying, read S3 Replication documentation [25]. For migration planning, compare DataSync [13], Storage Gateway [16], and the Snow Family [15]. The AWS Well-Architected Analytics Lens gives broader review questions for reliability, operational excellence, security, performance, and cost [2]. For distributed-system background, Dynamo remains a classic paper on highly available storage design [31]; for the lakehouse direction of modern analytics platforms, see Armbrust et al. [30].

Key Takeaways

- S3 is the durable object-storage foundation for many AWS data engineering architectures, but it is not a POSIX file system.
- Storage-class choice must balance access frequency, retrieval latency, retrieval fees, minimum duration, resilience, and compliance.
- Strong consistency simplifies ingestion and listing logic, but multi-object correctness still requires idempotent jobs and table-aware commit patterns.
- Lifecycle rules control cost and disposal; versioning, Object Lock, legal hold, and replication control recovery and retention.
- DataSync, Storage Gateway, and Snow Family solve different migration problems: online sync, hybrid protocol bridge, and offline or edge transfer.
- A compliant disaster-recovery design includes data, keys, policies, metadata, monitoring, runbooks, and tested failover operations.

1.10 Exercises

1. **(Conceptual)** Explain the difference between durability and availability. Give one S3 design decision that improves each.
2. **(Conceptual)** A team wants all objects to transition to Glacier Deep Archive after seven days. List three questions you would ask before approving the rule.
3. **(Hands-on)** Run the Thursday lab in a sandbox account. Capture the bucket versioning status, lifecycle configuration, and one SQS event message as evidence.
4. **(Hands-on)** Modify the lifecycle rule so objects under `tmp/` expire after seven days while objects under `raw/` transition after 30 days. Validate with `aws s3api get-bucket-lifecycle-configuration`.
5. **(Design)** Design prefixes and lifecycle rules for a lake with `raw/`, `validated/`, `curated/`, and `sandbox/` zones. State retention, storage class, and deletion policy for each.
6. **(Design)** Choose DataSync, File Gateway, Snowball Edge, or a combination for a 600 TB on-premises radiology archive with a 500 Mbps link. Justify the migration timeline and cutover plan.
7. **(Governance)** Compare Object Lock governance mode, compliance mode, and legal hold for medical-image retention. Include the operational risk of choosing the wrong mode.

8. **(Architecture)** Extend fig. 1.2 with Athena, Redshift Spectrum, or SageMaker consumers. Describe how access would be governed without exposing raw images broadly.

1.11 Capstone Project: A Compliant Multi-Region Medical Image Data Lake on S3

Capstone Project

Mission. Design, implement, and verify a compliant multi-region S3 data lake for medical X-ray images. Your platform must ingest raw images, preserve immutable versions, encrypt data with customer-managed keys, transition objects through cost-optimized storage classes, replicate protected data to a disaster-recovery Region, and publish ingest events to Amazon SQS.

Estimated time. 4–6 hours in a sandbox AWS account, plus optional time for cost modeling and security review.

What you'll practice.

- Selecting S3 storage classes and lifecycle transitions for hot, warm, archive, and deep-archive medical-image data.
- Enabling versioning, S3 Object Lock in compliance mode, and legal holds for WORM retention.
- Configuring SSE-KMS encryption, least-privilege IAM, Cross-Region Replication, and S3 event notifications to SQS.
- Verifying the design with AWS CLI commands, lifecycle JSON, replication configuration, and a Python/Boto3 smoke test.

Scenario

A regional healthcare provider is consolidating 10 PB of historical and newly generated X-ray images into an S3-backed data lake. The source systems include hospital PACS archives, imaging-center uploads, radiology workflow applications, and periodic research exports. Raw files are treated as regulated medical records: they may contain protected health information, must be encrypted at rest, must be retained without tampering, and must remain recoverable after operator error or a regional service disruption.

The business also faces cost pressure. Keeping all 10 PB in S3 Standard forever is simple but wasteful once studies age out of active clinical workflows. At the same time, moving records too quickly into deep archive can violate retrieval expectations for patient care, audits, research reproducibility, or litigation. Your design must therefore combine durability, compliance, disaster recovery, observability, and lifecycle economics instead of optimizing only for the lowest storage price.

Assume the primary Region is `us-east-1`, the disaster-recovery Region is `us-west-2`, and the organization already has AWS CLI v2, Python, Boto3, and appropriate sandbox permissions. Use globally unique bucket names, separate customer-managed KMS keys per Region, and non-production retention values while testing. For a real compliance deployment, retention periods and legal-hold procedures must be approved by legal, security, privacy, and records-management teams.

Requirements

Functional requirements.

- Create a primary S3 bucket for raw X-ray images and a disaster-recovery bucket in a second Region.
- Enable versioning on both buckets because S3 Object Lock and replication depend on versioned object history.
- Enable S3 Object Lock on the primary bucket at creation time, configure default retention in compliance mode, and demonstrate a legal hold on at least one test object.
- Require server-side encryption with AWS KMS customer-managed keys for all objects. Use a primary-region key for the source bucket and a separate recovery-region key for the replica bucket.
- Apply lifecycle transitions for objects under `raw/xray/`: S3 Standard for active ingestion, S3 Standard-IA after 30 days, and S3 Glacier Deep Archive after 365 days. Include noncurrent-version transitions so overwritten versions do not remain indefinitely in hot storage.
- Configure Cross-Region Replication from the primary bucket to the disaster-recovery bucket, including encrypted objects and Object Lock metadata where applicable [25, 24].
- Configure S3 event notifications so every `s3:ObjectCreated:*` event under `raw/xray/` sends a message to an SQS queue.
- Provide a least-privilege IAM design for ingestion, verification, replication, security administration, and read-only audit roles.

Non-functional requirements.

- Durability must rely on S3 multi-AZ storage in each Region plus CRR for regional disaster recovery.
- Recovery objectives must be explicit: target a minutes-scale RPO for online ingestion, document that CRR is asynchronous, and define an RTO that includes KMS, IAM, catalog, application, and queue reconfiguration.
- Compliance controls must prevent destructive changes to protected object versions during the retention period. Compliance mode is intentionally irreversible until the retention date expires, so testing must use short-lived, non-production retention only.
- Access must follow least privilege. Ingestion can write but not delete protected records; audit can list and read metadata; replication can read source versions and write replicas; security administrators can manage policies without broad data access.
- Operations must include verification evidence: bucket versioning status, Object Lock configuration, lifecycle rules, replication status, KMS encryption, legal-hold status, and one SQS event message.
- Cost controls must explain why each lifecycle threshold exists and must identify retrieval-latency risk before using Glacier Deep Archive [28].

Reference Architecture

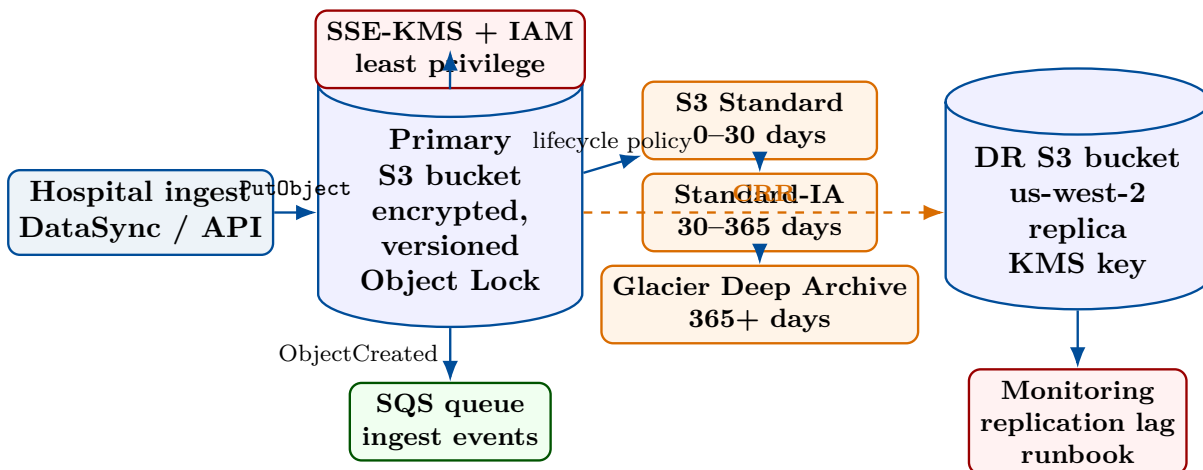


Figure 1.3: Capstone reference architecture for a compliant multi-region medical-image data lake on S3. The primary bucket receives encrypted, versioned, immutable objects; lifecycle rules manage cost; CRR creates a disaster-recovery copy; S3 event notifications feed SQS for downstream processing.

Implementation Steps

1. **Prepare names and Regions.** Choose globally unique bucket names and record your AWS account ID. In PowerShell, set variables for the primary Region, recovery Region, source bucket, destination bucket, queue name, retention days, and IAM role names. Keep all names free of patient identifiers.
2. **Create KMS keys.** Create one customer-managed KMS key in each Region. The source key must allow the ingestion and replication roles to encrypt and decrypt as required. The destination key must allow the replication role to encrypt replicas. In production, key policies should separate key administration from data use.
3. **Create Object-Lock-enabled buckets.** Object Lock must be enabled when the bucket is created. Enable versioning on both buckets immediately, then configure default retention on the primary bucket in compliance mode. Use a short test retention period in a sandbox.
4. **Apply encryption and lifecycle.** Configure default SSE-KMS encryption and apply lifecycle JSON for current and noncurrent versions under `raw/xray/`. The example transitions current objects to Standard-IA after 30 days and Glacier Deep Archive after 365 days, while moving noncurrent versions on their own schedule.
5. **Configure replication.** Create an IAM role trusted by S3, attach a scoped replication policy, and apply a replication configuration that handles encrypted objects. Verify that the destination bucket uses the recovery-region KMS key.
6. **Configure SQS ingest events.** Create an SQS queue, attach a queue policy allowing only the source bucket to send messages, and configure S3 event notifications for object-created events under `raw/xray/`.
7. **Ingest and verify.** Upload a sample object with KMS encryption, set a legal hold on that object version, confirm replication status, and consume the SQS notification. Record screenshots or command output as evidence.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $PrimaryRegion = "us-east-1"
3 $DrRegion = "us-west-2"
4 $AccountId = "123456789012"
5 $SourceBucket = "medical-xray-primary-123456789012"
6 $DestBucket = "medical-xray-dr-123456789012"
7 $QueueName = "medical-xray-ingest-events"
8 $ReplicationRoleName = "s3-medical-xray-replication-role"
9 $SourceKmsKeyArn = "arn:aws:kms:us-east-1:123456789012:key/source-key-id"
10 $DestKmsKeyArn = "arn:aws:kms:us-west-2:123456789012:key/dest-key-id"
11
12 aws s3api create-bucket '
13   --bucket $SourceBucket '
14   --region $PrimaryRegion '
15   --object-lock-enabled-for-bucket
16
17 aws s3api create-bucket '
18   --bucket $DestBucket '
19   --region $DrRegion '
20   --create-bucket-configuration LocationConstraint=$DrRegion '
21   --object-lock-enabled-for-bucket
22
23 aws s3api put-bucket-versioning '
24   --bucket $SourceBucket '
25   --versioning-configuration Status=Enabled
26
27 aws s3api put-bucket-versioning '
28   --bucket $DestBucket '
29   --region $DrRegion '
30   --versioning-configuration Status=Enabled
31
32 $ObjectLockJson = @"
33 {
34   "ObjectLockEnabled": "Enabled",
35   "Rule": {
36     "DefaultRetention": {
37       "Mode": "COMPLIANCE",
38       "Days": 30
39     }
40   }
41 }
42 "@
43 $ObjectLockJson | Set-Content -Path object-lock.json -Encoding ascii
44
45 aws s3api put-object-lock-configuration '
46   --bucket $SourceBucket '
47   --object-lock-configuration file://object-lock.json
48
49 $LifecycleJson = @"
50 {
51   "Rules": [
52     {
53       "ID": "raw-xray-standard-to-ia-to-deep-archive",
54       "Status": "Enabled",
55       "Filter": { "Prefix": "raw/xray/" },
56       "Transitions": [
57         { "Days": 30, "StorageClass": "STANDARD_IA" },
58         { "Days": 365, "StorageClass": "DEEP_ARCHIVE" }
59       ],
60       "NoncurrentVersionTransitions": [
61         { "NoncurrentDays": 30, "StorageClass": "STANDARD_IA" },
62         { "NoncurrentDays": 365, "StorageClass": "DEEP_ARCHIVE" }
63       ],

```

```

64     "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
65   }
66 ]
67 }
68 @"
69 $LifecycleJson | Set-Content -Path lifecycle-capstone.json -Encoding ascii
70
71 aws s3api put-bucket-lifecycle-configuration '
72   --bucket $SourceBucket '
73   --lifecycle-configuration file://lifecycle-capstone.json
74
75 $SourceEncryptionJson = @"
76 {
77   "Rules": [
78     {
79       "ApplyServerSideEncryptionByDefault": {
80         "SSEAlgorithm": "aws:kms",
81         "KMSEMasterKeyID": "$SourceKmsKeyArn"
82       },
83       "BucketKeyEnabled": true
84     }
85   ]
86 }
87 @"
88 $SourceEncryptionJson | Set-Content -Path source-encryption.json -Encoding
89   ascii
90
91 aws s3api put-bucket-encryption '
92   --bucket $SourceBucket '
93   --server-side-encryption-configuration file://source-encryption.json
94
95 $QueueUrl = aws sqs create-queue '
96   --queue-name $QueueName '
97   --region $PrimaryRegion '
98   --query QueueUrl '
99   --output text
100
101 $QueueArn = aws sqs get-queue-attributes '
102   --queue-url $QueueUrl '
103   --attribute-names QueueArn '
104   --query 'Attributes.QueueArn' '
105   --output text
106
107 $SqsPolicyJson = @"
108 {
109   "Version": "2012-10-17",
110   "Statement": [
111     {
112       "Sid": "AllowOnlyPrimaryBucketEvents",
113       "Effect": "Allow",
114       "Principal": { "Service": "s3.amazonaws.com" },
115       "Action": "sqs:SendMessage",
116       "Resource": "$QueueArn",
117       "Condition": {
118         "ArnLike": { "aws:SourceArn": "arn:aws:s3:::$SourceBucket" },
119         "StringEquals": { "aws:SourceAccount": "$AccountId" }
120       }
121     }
122   ]
123 }
124 @"
125 $SqsPolicyJson | Set-Content -Path sqs-policy-capstone.json -Encoding ascii

```

```

126 aws sqs set-queue-attributes '
127   --queue-url $QueueUrl '
128   --attributes Policy="$(Get-Content -Raw sqs-policy-capstone.json)"
129
130 $NotificationJson = @"
131 {
132   "QueueConfigurations": [
133     {
134       "Id": "raw-xray-object-created-to-sqs",
135       "QueueArn": "$QueueArn",
136       "Events": ["s3:ObjectCreated:*"],
137       "Filter": {
138         "Key": {
139           "FilterRules": [
140             { "Name": "prefix", "Value": "raw/xray/" }
141           ]
142         }
143       }
144     }
145   ]
146 }
147 "@
148 $NotificationJson | Set-Content -Path notification-capstone.json -Encoding
    ascii
149
150 aws s3api put-bucket-notification-configuration '
151   --bucket $SourceBucket '
152   --notification-configuration file://notification-capstone.json

```

Listing 1.5: PowerShell-oriented AWS CLI commands for the capstone S3 data lake. Replace every placeholder before running.

The replication policy is intentionally separated from the first listing because it is the easiest place to over-permission a role. The policy must allow the role to read source versions, read Object Lock metadata, decrypt source objects when required, replicate objects and tags, and encrypt destination replicas with the destination KMS key.

```

1 {
2   "Role": "arn:aws:iam::123456789012:role/s3-medical-xray-replication-role",
3   "Rules": [
4     {
5       "ID": "replicate-raw-xray-to-dr-region",
6       "Status": "Enabled",
7       "Priority": 1,
8       "DeleteMarkerReplication": { "Status": "Disabled" },
9       "Filter": { "Prefix": "raw/xray/" },
10      "SourceSelectionCriteria": {
11        "SseKmsEncryptedObjects": { "Status": "Enabled" }
12      },
13      "Destination": {
14        "Bucket": "arn:aws:s3:::medical-xray-dr-123456789012",
15        "StorageClass": "STANDARD",
16        "EncryptionConfiguration": {
17          "ReplicaKmsKeyID": "arn:aws:kms:us-west-2:123456789012:key/dest-key-
id"
18        },
19        "Metrics": {
20          "Status": "Enabled",
21          "EventThreshold": { "Minutes": 15 }
22        },
23        "ReplicationTime": {
24          "Status": "Enabled",
25          "Time": { "Minutes": 15 }

```

```

26     }
27     }
28     }
29 ]
30 }

```

Listing 1.6: Skeleton replication configuration for encrypted medical-image objects. Replace role and KMS values before use.

After creating and attaching the replication role policy, apply the replication configuration with the AWS CLI. Then run a Boto3 smoke test that uploads a sample object, records its version ID, applies a legal hold, verifies encryption and Object Lock metadata, checks for a replication status value, and reads the SQS event.

```

1  import json
2  import os
3  import time
4  from pathlib import Path
5
6  import boto3
7
8  primary_region = os.environ.get("PRIMARY_REGION", "us-east-1")
9  dr_region = os.environ.get("DR_REGION", "us-west-2")
10 source_bucket = os.environ["SOURCE_BUCKET"]
11 dest_bucket = os.environ["DEST_BUCKET"]
12 queue_url = os.environ["QUEUE_URL"]
13 kms_key_arn = os.environ["SOURCE_KMS_KEY_ARN"]
14
15 s3 = boto3.client("s3", region_name=primary_region)
16 s3_dr = boto3.client("s3", region_name=dr_region)
17 sqs = boto3.client("sqs", region_name=primary_region)
18
19 key = "raw/xray/capstone-study-0001.dcm"
20 body = b"synthetic DICOM placeholder for capstone validation\n"
21
22 put_response = s3.put_object(
23     Bucket=source_bucket,
24     Key=key,
25     Body=body,
26     ServerSideEncryption="aws:kms",
27     SSEKMSKeyId=kms_key_arn,
28     ContentType="application/dicom",
29     Metadata={"study-id": "capstone-study-0001", "modality": "xray"},
30 )
31 version_id = put_response["VersionId"]
32
33 s3.put_object_legal_hold(
34     Bucket=source_bucket,
35     Key=key,
36     VersionId=version_id,
37     LegalHold={"Status": "ON"},
38 )
39
40 head = s3.head_object(Bucket=source_bucket, Key=key, VersionId=version_id)
41 assert head["ServerSideEncryption"] == "aws:kms"
42 assert head["SSEKMSKeyId"] == kms_key_arn
43
44 lock = s3.get_object_legal_hold(
45     Bucket=source_bucket,
46     Key=key,
47     VersionId=version_id,
48 )
49 assert lock["LegalHold"]["Status"] == "ON"

```

```

50
51 versioning = s3.get_bucket_versioning(Bucket=source_bucket)
52 assert versioning.get("Status") == "Enabled"
53
54 object_lock = s3.get_object_lock_configuration(Bucket=source_bucket)
55 mode = object_lock["ObjectLockConfiguration"]["Rule"]["DefaultRetention"]["Mode"]
56 assert mode == "COMPLIANCE"
57
58 for attempt in range(12):
59     replica = s3.head_object(Bucket=source_bucket, Key=key, VersionId=
        version_id)
60     status = replica.get("ReplicationStatus", "PENDING")
61     if status in {"COMPLETED", "FAILED", "REPLICA"}:
62         print(f"replication status: {status}")
63         break
64     time.sleep(10)
65 else:
66     print("replication status still pending after smoke-test wait")
67
68 try:
69     s3_dr.head_object(Bucket=dest_bucket, Key=key)
70     print("destination object is visible")
71 except s3_dr.exceptions.ClientError as exc:
72     print(f"destination check not complete yet: {exc.response['Error']['Code']}")
73
74 messages = sqs.receive_message(
75     QueueUrl=queue_url,
76     MaxNumberOfMessages=1,
77     WaitTimeSeconds=10,
78 ).get("Messages", [])
79 if not messages:
80     raise RuntimeError("No S3 ObjectCreated notification arrived in SQS")
81
82 payload = json.loads(messages[0]["Body"])
83 record = payload["Records"][0]
84 assert record["s3"]["bucket"]["name"] == source_bucket
85 assert record["s3"]["object"]["key"] == key
86 print("verified event:", record["eventName"], record["s3"]["object"]["key"])
87
88 Path("capstone-verification.json").write_text(
89     json.dumps(
90         {
91             "bucket": source_bucket,
92             "key": key,
93             "version_id": version_id,
94             "legal_hold": "ON",
95             "object_lock_mode": mode,
96             "event_name": record["eventName"],
97         },
98         indent=2,
99     ),
100     encoding="utf-8",
101 )

```

Listing 1.7: Boto3 ingestion and verification snippet for the capstone.

Deliverables

Submit a concise engineering packet containing the following artifacts:

- A one-page architecture summary with primary Region, disaster-recovery Region, bucket

names, KMS-key strategy, RPO, RTO, and lifecycle thresholds.

- The final lifecycle JSON, Object Lock configuration, replication configuration, SQS queue policy, and S3 notification configuration.
- IAM policy excerpts for ingestion, replication, audit, and security administration roles, with a short least-privilege explanation for each role.
- Command output or screenshots showing versioning enabled, Object Lock compliance retention configured, legal hold enabled on one object version, lifecycle rules present, and default SSE-KMS encryption active.
- Evidence of successful ingestion: object key, version ID, KMS key ID, metadata, SQS event message, and replication status or replication-lag observation.
- A cleanup plan for sandbox resources, including queues, IAM roles, temporary JSON files, test objects after retention expires, and KMS key deletion scheduling where appropriate.

Acceptance Criteria

Use this checklist to self-grade the capstone before submitting:

- The source bucket was created with Object Lock support and has versioning enabled.
- The destination bucket is in a different Region and has versioning enabled before replication is configured.
- Default bucket encryption uses SSE-KMS with a customer-managed key, not unencrypted objects or ad hoc client-side encryption.
- Object Lock default retention is configured in compliance mode for the test prefix or bucket, and the submission explains why production retention values require formal approval.
- At least one uploaded object version has legal hold enabled and the evidence includes its version ID.
- Lifecycle rules transition current versions from S3 Standard to Standard-IA and then to Glacier Deep Archive, with noncurrent-version handling and incomplete multipart upload cleanup.
- CRR is configured for `raw/xray/`, includes encrypted-object handling, and uses a destination KMS key in the recovery Region.
- S3 event notifications send `s3:ObjectCreated:*` events for `raw/xray/` to SQS, and the queue policy restricts sends to the source bucket and account.
- IAM policies are scoped by bucket ARN, prefix, KMS key, and role purpose. Broad administrative wildcards are justified only for temporary sandbox setup roles, not steady-state data access.
- Verification evidence includes AWS CLI or Boto3 output for versioning, lifecycle, Object Lock, legal hold, encryption, SQS event receipt, and replication status.
- The architecture explains retrieval-latency trade-offs for Standard-IA and Glacier Deep Archive instead of treating lifecycle as a pure cost-saving feature.
- The disaster-recovery plan covers data, keys, permissions, monitoring, catalog dependencies, application cutover, and post-failover reconciliation.

Stretch Goals

- Use S3 Batch Operations to apply tags, legal holds, or retention settings to a manifest of historical objects after validating the workflow on a small sample.
- Run Amazon Macie discovery jobs on a controlled prefix to identify possible protected health information or other sensitive fields in metadata exports before broad analytics access is granted.
- Build a cost model for 10 PB across S3 Standard, Standard-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive. Include request charges, retrieval charges, minimum storage duration, replication storage, KMS requests, and data transfer assumptions.
- Enable S3 Storage Lens and define dashboard metrics for object count, bytes by storage class, incomplete multipart uploads, noncurrent-version growth, encrypted-object coverage, and replication health.
- Add EventBridge or Lambda processing downstream of SQS to write an immutable ingest manifest containing object key, version ID, checksum, KMS key ID, study metadata, and replication status.
- Prototype a failover drill: disable writes to the primary application path, validate replica availability in the recovery Region, switch a read-only consumer to the DR bucket, and document the reconciliation steps required before failing back.

Relational Databases: RDS and Aurora

Learning Objectives

- Explain the managed relational database model behind Amazon RDS, including engines, instance classes, storage, backups, maintenance windows, and operational responsibility boundaries.
- Design highly available RDS deployments with Multi-AZ DB instances or Multi-AZ DB clusters, and distinguish failover capacity from horizontal read scaling.
- Compare RDS Read Replicas, Aurora Replicas, Aurora Global Database, and cross-region replica patterns for reporting, disaster recovery, and low-latency reads.
- Describe Aurora's cloud-native distributed storage architecture, quorum writes, cache-aware failover, Global Database replication, and Aurora Serverless v2 scaling model.
- Use AWS Database Migration Service and AWS Schema Conversion Tool to plan homogeneous and heterogeneous database migrations with full-load and change data capture phases.
- Deploy an Aurora Serverless v2 PostgreSQL cluster, add cross-region read capacity, simulate regional failover, and design a zero-downtime Oracle ERP cutover to Aurora PostgreSQL.

Cross-Cloud Equivalence

Managed relational platforms use different product names, but the architectural choices are familiar: engine compatibility, high availability, read scale, global reach, migration tooling, and operational automation.

Capability	AWS	Azure	GCP
Managed relational	RDS / Aurora	Azure SQL / Azure Database for PostgreSQL Hyperscale	Cloud SQL / AlloyDB
Cloud-native scale-out	Aurora		AlloyDB
High availability	Multi-AZ	Zone-redundant HA / auto-failover groups	Regional HA
Read replicas	RDS Read Replicas / Aurora Replicas	Read replicas	Read replicas
Global or geo-replication	Aurora Global Database	Active geo-replication	Cross-region replicas
Migration	DMS + SCT	Azure DMS	Database Migration Service
Operational automation	Backups, patching, monitoring, Performance Insights	Automated backups, maintenance, Query Performance Insight	Backups, maintenance, Query Insights

Adjacent services follow the same mental model. MongoDB Atlas and Amazon DocumentDB are managed document-database analogs; Microsoft Fabric and BigQuery are lakehouse or warehouse platforms rather than OLTP systems; Snowflake separates compute and storage for analytical workloads, while RDS and Aurora optimize transactional consistency and operational database availability.

2.1 Week 2 Roadmap and Mental Model

Relational databases remain central to data engineering because most enterprise facts begin as transactions: orders, invoices, shipments, journal entries, patient encounters, payments, and inventory movements. A data engineer must understand not only how to extract those records, but also how the source database preserves consistency, scales reads, fails over, and migrates safely. Week 2 therefore focuses on managed relational infrastructure: Amazon RDS for familiar engines, Amazon Aurora for cloud-native relational scale, and AWS DMS plus AWS SCT for migration and replication workflows [9, 5, 12].

Amazon RDS removes much of the undifferentiated database administration work: provisioning, backups, patching, monitoring hooks, storage management, and high-availability automation. It does not remove schema design, transaction design, index selection, query tuning, capacity planning, security boundaries, or migration accountability. Aurora goes further by redesigning the storage layer for distributed cloud infrastructure, but applications still experience it through MySQL-compatible or PostgreSQL-compatible SQL endpoints.

Definition

Amazon RDS is a managed relational database service for engines such as PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, and Db2. **Amazon Aurora** is an AWS-designed, MySQL-compatible and PostgreSQL-compatible relational database that separates compute from a distributed storage volume.

Key Terms

Primary instance: the writer for a database. **Standby:** synchronous failover target in a Multi-AZ deployment. **Read replica:** asynchronous read-scaling copy. **Cluster endpoint:** Aurora endpoint that tracks the current writer. **CDC:** change data capture used to keep a migration target current.

2.2 Monday: Amazon RDS Architecture, Multi-AZ, and Read Replicas

2.2.1 Managed RDS Architecture

An RDS database is a managed combination of engine software, compute, storage, networking, backup automation, monitoring, and control-plane operations. You choose an engine, version, instance class, allocated storage, subnet group, security groups, parameter groups, option groups, backup retention, maintenance window, encryption configuration, and high-availability topology. AWS operates the underlying infrastructure and service automation; the customer owns data model, credentials, network exposure, query behavior, and application compatibility.

For a data engineering team, the most important RDS design boundary is the split between transactional workloads and analytical workloads. RDS can support operational reporting, but it is not a replacement for S3 data lakes, Redshift, Athena, or specialized analytical stores. Extracting from production RDS requires workload isolation, replica usage where appropriate, and a clear understanding of transaction logs, snapshots, and business cutover windows.

Note

RDS automation is not the same as database tuning. A poorly indexed query, chatty transaction pattern, or long-running report can still saturate CPU, memory, locks, connections, IOPS, or log volume on a fully managed database.

2.2.2 Backups, Maintenance, and Observability

RDS automated backups combine daily snapshots with transaction logs so a database can be restored to a point within the retention period. Manual snapshots persist until deleted. Maintenance windows let AWS apply engine or OS updates; some changes are online, while others require a reboot or failover. CloudWatch metrics, Enhanced Monitoring, Performance Insights, database logs, and events are the operational evidence used to decide whether a system is healthy.

Tip

For production databases, set backup retention, deletion protection, encryption, monitoring, and maintenance windows intentionally at creation time. Retrofitting these controls during an incident is slower and riskier than declaring them in infrastructure as code.

2.2.3 Multi-AZ Deployments

Multi-AZ is the core RDS high-availability pattern. In a traditional Multi-AZ DB instance deployment, RDS synchronously replicates data to a standby in a different Availability Zone and automatically fails over when the primary becomes unavailable. The standby is not used for read traffic. Newer Multi-AZ DB clusters provide a writer and readable standby instances with lower commit latency and faster failover for supported engines [22].

Definition

RDS Multi-AZ is a high-availability configuration that maintains a synchronous standby in another Availability Zone and uses automatic failover to preserve database availability during infrastructure, instance, or maintenance events.

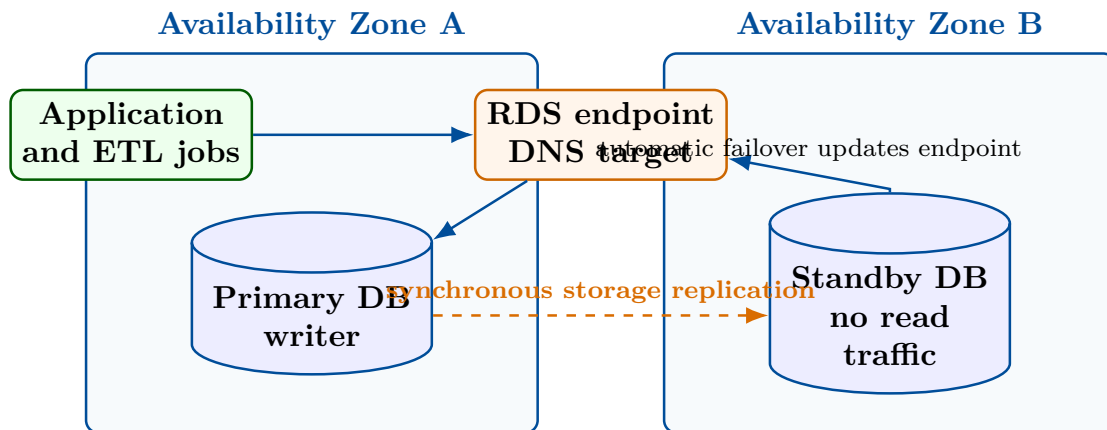


Figure 2.1: RDS Multi-AZ protects availability by synchronously maintaining a standby in another Availability Zone and moving the endpoint during failover. It is not a read-scaling feature in the traditional DB instance model.

Common Pitfall

Confusing Multi-AZ with read scaling. Multi-AZ is primarily for availability and durability of service during failures or maintenance. Use Read Replicas, Aurora Replicas, caching, or analytical offload when the problem is read throughput.

2.2.4 Read Replicas

RDS Read Replicas use asynchronous replication to offload read-heavy workloads, support reporting, test upgrades, or create cross-region disaster-recovery candidates for supported engines [29]. Because replication is asynchronous, replicas can lag. Applications must tolerate stale reads, and data engineers must not point critical reconciliation jobs at a lagging replica without checking freshness.

Pattern	Best fit	Design caution
Single-AZ RDS	Development, tests, noncritical workloads	No AZ-level database failover target
Multi-AZ DB instance	Production OLTP availability	Standby is not a reporting target
Multi-AZ DB cluster	Production engines needing faster failover and readable standbys	Engine and feature support differ from classic deployments
Read Replica	Read scaling, reporting, migration staging	Asynchronous lag and replica-specific capacity planning
Cross-region replica	Regional DR candidate or local reads	Higher cost, lag, egress, and failover runbook complexity

Table 2.1: RDS deployment patterns mapped to common data engineering decisions.

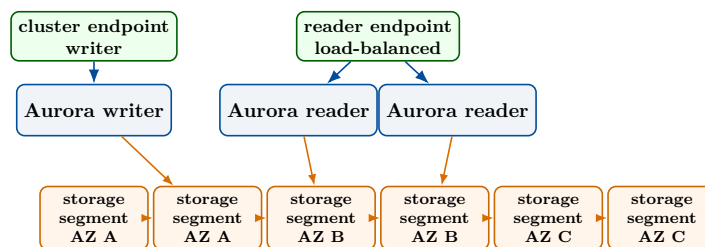
2.3 Tuesday: Amazon Aurora Cloud-Native Relational Architecture

2.3.1 Distributed Storage and Compute Separation

Aurora keeps the SQL engine familiar while changing the storage architecture. An Aurora cluster has one writer instance, zero or more reader instances, and a distributed cluster volume that automatically grows as data increases. The storage layer replicates data across multiple Availability Zones, and database instances attach to that shared volume rather than each owning a complete independent storage copy. This separation enables fast replica creation, faster failover, and storage durability that is not tied to one compute node [4].

Definition

Aurora cluster volume is a distributed, replicated storage volume shared by the DB instances in an Aurora cluster. Compute nodes can fail or be replaced while the storage service preserves the database volume independently.



The storage volume is replicated across Availability Zones; readers share the same logical volume rather than receiving separate full database copies.

Figure 2.2: Aurora separates compute instances from a distributed cluster volume. Writer and reader endpoints route application traffic while the storage service handles replication and durability.

Note

Aurora replicas can serve reads and participate in failover promotion, but they still share the cluster's operational fate more closely than independent analytical systems. Heavy reporting can still affect cache, connections, locks, and engine resources.

2.3.2 Aurora Global Database

Aurora Global Database replicates an Aurora cluster to secondary Regions using storage-level replication designed for low-latency cross-region propagation [26]. The primary Region accepts writes. Secondary Regions can serve low-latency reads and can be promoted during a regional disaster. A global database improves recovery posture, but it requires a failover plan for application endpoints, secrets, DNS, parameter differences, monitoring, and downstream integrations.

Tip

Write down the difference between *planned switchover* and *unplanned failover*. A planned switchover can coordinate writes and reduce data loss. An unplanned failover must prioritize recovery time while accepting that asynchronous replication may have an RPO window.

2.3.3 Aurora Serverless v2

Aurora Serverless v2 scales database capacity in Aurora Capacity Units without managing fixed DB instance sizes [27]. It is useful for variable workloads, development and testing, SaaS tenants with uneven activity, and operational systems that need finer-grained scaling than scheduled instance changes. Unlike a stateless compute service, database scaling still interacts with connections, transactions, locks, memory pressure, and query plans.

Common Pitfall

Assuming serverless means unlimited. Aurora Serverless v2 scales within configured minimum and maximum ACUs. Connection storms, unbounded queries, missing indexes, and long transactions can still produce throttling, poor latency, or cost surprises.

2.4 Wednesday: AWS DMS and Schema Conversion Tool

2.4.1 Migration Phases

AWS Database Migration Service moves data from supported sources to supported targets with full-load, change data capture, or combined full-load-plus-CDC tasks [12]. Homogeneous migrations, such as PostgreSQL to PostgreSQL, can often use native schema tools plus DMS for replication. Heterogeneous migrations, such as Oracle to Aurora PostgreSQL, require schema assessment, code conversion, data-type mapping, and extensive application testing. AWS Schema Conversion Tool analyzes and converts schema objects where possible, then flags objects that require manual work [14].

Definition

Full load copies existing rows from the source to the target. **Change data capture** reads ongoing changes from database logs and applies them to the target so the migration can approach cutover with a small lag window.

2.4.2 Replication Instances, Endpoints, and Tasks

A DMS migration uses a replication instance or serverless replication configuration, source and target endpoints, table mappings, task settings, and validation options. The replication resource must reach both databases over the network and must have credentials with enough rights to read source logs and apply target changes. For Oracle sources, supplemental logging, archive log retention, LOB handling, and privileges are frequent planning items.

Note

DMS is a migration and replication service, not a magic compatibility layer. Stored procedures, triggers, packages, optimizer hints, scheduler jobs, security models, and application SQL must be assessed separately.

2.4.3 Validation and Cutover

A zero-downtime or near-zero-downtime migration depends on controlling writes. The usual pattern is: assess schema, provision target, convert schema, run full load, keep CDC running, validate row counts and checksums, rehearse cutover, freeze or dual-write source transactions, let lag reach an approved threshold, switch applications, and keep rollback evidence until the business accepts the new system. DMS validation helps, but business reconciliation remains essential [17].

Tip

Always define who can approve cutover and rollback before the migration weekend. Technical success is insufficient if finance, supply chain, or operations teams cannot reconcile critical balances after the switch.

2.5 Thursday Hands-on Project: Aurora Serverless v2 and Regional Failover

This lab provisions an Aurora PostgreSQL Serverless v2 cluster, creates a secondary-region read path using Aurora Global Database, and rehearses a failover workflow. Replace every account-specific value before running. The commands are written for Windows PowerShell using AWS CLI v2; line continuations use backticks.

Note

The lab creates billable database resources. Use a sandbox account, small ACU ranges, restrictive security groups, and short-lived clusters. Clean up global clusters, DB clusters, DB instances, subnet groups, and secrets after testing.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $PrimaryRegion = "us-east-1"
3 $SecondaryRegion = "us-west-2"
4 $GlobalClusterId = "erp-aurora-global-demo"
5 $PrimaryClusterId = "erp-aurora-primary-demo"
6 $SecondaryClusterId = "erp-aurora-secondary-demo"
7 $DbName = "erpdb"
8 $MasterUser = "erp_admin"
9 $PrimarySubnetGroup = "aurora-private-us-east-1"
10 $SecondarySubnetGroup = "aurora-private-us-west-2"
11 $PrimarySg = "sg-0123456789abcdef0"
12 $SecondarySg = "sg-0fedcba9876543210"
13
14 aws rds create-global-cluster `
15     --global-cluster-identifier $GlobalClusterId `
16     --engine aurora-postgresql `
17     --engine-version 16.4 `
18     --region $PrimaryRegion
19
20 aws rds create-db-cluster `
21     --db-cluster-identifier $PrimaryClusterId `
22     --global-cluster-identifier $GlobalClusterId `
23     --engine aurora-postgresql `
24     --engine-version 16.4 `
25     --database-name $DbName `
26     --master-username $MasterUser `
27     --manage-master-user-password `
28     --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=4 `
29     --db-subnet-group-name $PrimarySubnetGroup `
30     --vpc-security-group-ids $PrimarySg `
31     --backup-retention-period 7 `
32     --deletion-protection `
33     --region $PrimaryRegion
34
35 aws rds create-db-instance `
36     --db-instance-identifier "$PrimaryClusterId-writer-1" `
37     --db-cluster-identifier $PrimaryClusterId `
38     --engine aurora-postgresql `
39     --db-instance-class db.serverless `

```

```
40 --region $PrimaryRegion
```

Listing 2.1: AWS CLI commands to create an Aurora PostgreSQL Serverless v2 cluster and attach it to a global database.

```
1 aws rds wait db-cluster-available '
2   --db-cluster-identifier $PrimaryClusterId '
3   --region $PrimaryRegion
4
5 aws rds create-db-cluster '
6   --db-cluster-identifier $SecondaryClusterId '
7   --global-cluster-identifier $GlobalClusterId '
8   --engine aurora-postgresql '
9   --engine-version 16.4 '
10  --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=4 '
11  --db-subnet-group-name $SecondarySubnetGroup '
12  --vpc-security-group-ids $SecondarySg '
13  --region $SecondaryRegion
14
15 aws rds create-db-instance '
16   --db-instance-identifier "$SecondaryClusterId-reader-1" '
17   --db-cluster-identifier $SecondaryClusterId '
18   --engine aurora-postgresql '
19   --db-instance-class db.serverless '
20   --region $SecondaryRegion
21
22 aws rds describe-global-clusters '
23   --global-cluster-identifier $GlobalClusterId '
24   --query "GlobalClusters[0].GlobalClusterMembers[*].[DBClusterArn,IsWriter]" '
25   --output table '
26   --region $PrimaryRegion
27
28 # Planned switchover is preferred for drills because it coordinates roles.
29 aws rds switchover-global-cluster '
30   --global-cluster-identifier $GlobalClusterId '
31   --target-db-cluster-identifier "arn:aws:rds:us-west-2:123456789012:cluster:
32     erp-aurora-secondary-demo" '
33   --region $PrimaryRegion
```

Listing 2.2: AWS CLI commands to add a secondary Aurora Global Database cluster and rehearse failover.

2.5.1 Boto3 Verification

The Boto3 snippet verifies that the global cluster has one writer, collects endpoints, and records failover evidence. In a real test, pair this with SQL-level checks using a PostgreSQL client from a host inside the VPC.

```
1 import json
2 import os
3 from pathlib import Path
4
5 import boto3
6
7 primary_region = os.environ.get("PRIMARY_REGION", "us-east-1")
8 secondary_region = os.environ.get("SECONDARY_REGION", "us-west-2")
9 global_cluster_id = os.environ["GLOBAL_CLUSTER_ID"]
10 primary_cluster_id = os.environ["PRIMARY_CLUSTER_ID"]
11 secondary_cluster_id = os.environ["SECONDARY_CLUSTER_ID"]
12
13 rds_primary = boto3.client("rds", region_name=primary_region)
14 rds_secondary = boto3.client("rds", region_name=secondary_region)
```

```

15
16 global_cluster = rds_primary.describe_global_clusters(
17     GlobalClusterIdentifier=global_cluster_id
18 )["GlobalClusters"][0]
19 members = global_cluster["GlobalClusterMembers"]
20 writers = [member for member in members if member["IsWriter"]]
21 assert len(writers) == 1, writers
22
23 primary = rds_primary.describe_db_clusters(
24     DBClusterIdentifier=primary_cluster_id
25 )["DBClusters"][0]
26 secondary = rds_secondary.describe_db_clusters(
27     DBClusterIdentifier=secondary_cluster_id
28 )["DBClusters"][0]
29
30 evidence = {
31     "global_cluster": global_cluster_id,
32     "writer_cluster_arn": writers[0]["DBClusterArn"],
33     "primary_endpoint": primary.get("Endpoint"),
34     "primary_reader_endpoint": primary.get("ReaderEndpoint"),
35     "secondary_endpoint": secondary.get("Endpoint"),
36     "secondary_reader_endpoint": secondary.get("ReaderEndpoint"),
37     "members": [
38         {"arn": member["DBClusterArn"], "is_writer": member["IsWriter"]}
39         for member in members
40     ],
41 }
42
43 Path("chapter2-aurora-global-evidence.json").write_text(
44     json.dumps(evidence, indent=2), encoding="utf-8"
45 )
46 print(json.dumps(evidence, indent=2))

```

Listing 2.3: Boto3 verification for Aurora Global Database membership and endpoints.

Common Pitfall

Failing over only the database. A regional failover also needs secrets, DNS, application configuration, security groups, IAM, monitoring alarms, DMS tasks, job schedulers, and downstream consumers to point at the promoted Region.

2.6 Friday Use Case: Zero-Downtime Oracle ERP Migration to Aurora PostgreSQL

A manufacturing company runs a monolithic on-premises Oracle ERP that supports purchasing, inventory, order management, finance, and shop-floor execution. The database is expensive to operate, hard to scale for reporting, and tightly coupled to legacy stored procedures. The target state is Aurora PostgreSQL for transactional workloads, S3 and Redshift for analytics, and DMS-based replication during the migration window.

2.6.1 Architecture Narrative

The migration begins with discovery. SCT assesses schemas, packages, triggers, sequences, materialized views, synonyms, partitioning, data types, and SQL constructs. Objects that convert cleanly become Aurora PostgreSQL DDL. Objects that do not convert become remediation backlog for database developers and application teams. Meanwhile, the platform team provisions Aurora PostgreSQL with Multi-AZ storage, at least one reader, encryption, backups, parameter groups, monitoring, and a cross-region recovery pattern.

DMS then performs a full load from Oracle to Aurora PostgreSQL and continues CDC from Oracle redo or archive logs. During rehearsals, teams compare table counts, financial balances, open orders, inventory quantities, and application smoke tests. The final cutover freezes writes or routes them through a controlled dual-write layer, waits for DMS latency to reach the approved threshold, validates business-critical queries, switches application endpoints, and keeps Oracle read-only until rollback risk has passed.

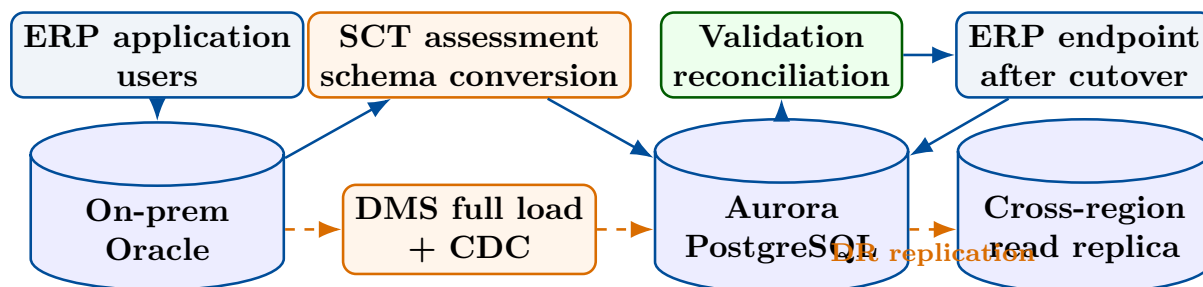


Figure 2.3: Zero-downtime Oracle ERP migration pattern using SCT for schema conversion, DMS for full load and CDC, validation for business reconciliation, and Aurora PostgreSQL with cross-region recovery for the target.

2.6.2 Operational Decisions

Source readiness. Oracle supplemental logging, archive log retention, network connectivity, and read privileges must be in place before DMS starts. The source must keep enough logs to survive DMS pauses without losing CDC continuity.

Target readiness. Aurora PostgreSQL parameter groups, extensions, schemas, roles, vacuum behavior, indexes, and connection pooling must be tested under realistic ERP concurrency. Load indexes strategically; sometimes a full load is faster before building selected secondary indexes, but constraints required for correctness cannot be skipped casually.

Cutover control. The business needs a cutover commander, rollback owner, reconciliation owners, and a communication plan. Technical DMS latency is only one metric; financial totals, order status, inventory balances, and integration queues decide whether the ERP is ready.

Common Pitfall

Underestimating stored procedures and batch jobs. Oracle ERP systems often contain business logic in packages, triggers, scheduler jobs, and ad hoc reports. Schema conversion without application and batch remediation is not a complete migration.

2.7 Interview Q&A

2.7.1 Concepts and Trade-offs

1. **Q: What is the difference between RDS Multi-AZ and a Read Replica?**

A: Multi-AZ provides synchronous standby capacity for high availability and failover. A Read Replica uses asynchronous replication for read scaling, reporting, or migration staging and can lag behind the writer.

2. **Q: Why can Aurora fail over faster than many traditional databases?**

A: Aurora separates compute from a distributed cluster volume. A replica can be promoted

without copying an entire database volume because the storage layer is already shared and replicated.

3. **Q: When would you use Aurora Global Database?**

A: Use it when a workload needs low-latency cross-region reads or a stronger regional disaster-recovery posture than backups alone. It still requires application and operations failover planning.

4. **Q: What does Aurora Serverless v2 solve, and what does it not solve?**

A: It provides fine-grained capacity scaling for variable workloads. It does not eliminate schema tuning, connection management, transaction design, or maximum-capacity planning.

5. **Q: What is the role of SCT in an Oracle to Aurora PostgreSQL migration?**

A: SCT assesses and converts schema objects where possible and identifies manual remediation for incompatible SQL, procedural code, data types, and database features.

6. **Q: Why is CDC important for near-zero-downtime migration?**

A: CDC keeps the target current while the source remains online after the initial full load. Cutover can then happen after replication lag and business validation reach approved thresholds.

2.8 Further Reading

Start with the Amazon RDS User Guide for instance, backup, monitoring, security, Multi-AZ, and replica behavior [9]. Review RDS Multi-AZ and Read Replica documentation before promising RTO or read-scaling outcomes [22, 29]. Study the Aurora User Guide, especially storage reliability, endpoints, Global Database, and Serverless v2 capacity scaling [5, 4, 26, 27]. For migrations, read the AWS DMS User Guide, AWS SCT User Guide, and DMS best-practices guidance before designing an Oracle ERP cutover [12, 14, 17].

Key Takeaways

- RDS is managed infrastructure for familiar relational engines, but customers still own schema design, query behavior, access control, and workload isolation.
- Multi-AZ protects database availability; Read Replicas and Aurora Replicas address read scaling and can introduce asynchronous lag.
- Aurora separates compute from a distributed storage layer, enabling fast replica operations, failover, and cloud-native durability patterns.
- Aurora Global Database supports cross-region reads and disaster recovery, but application failover, secrets, networking, and monitoring must be rehearsed.
- Aurora Serverless v2 scales database capacity within configured ACU limits; it is not a substitute for indexing, pooling, and transaction discipline.
- DMS and SCT can reduce migration downtime and conversion effort, but heterogeneous ERP migrations still require business reconciliation and application remediation.

2.9 Exercises

1. **(Conceptual)** Explain why a traditional RDS Multi-AZ standby is not used for reporting traffic. What problem does it solve instead?

2. **(Conceptual)** Compare RDS Read Replicas and Aurora Replicas. Include replication behavior, failover use, and reporting risk.
3. **(Hands-on)** Use AWS CLI in a sandbox to describe an existing RDS or Aurora cluster and identify engine version, backup retention, deletion protection, endpoints, and Multi-AZ or replica status.
4. **(Hands-on)** Modify listing 2.3 to write CloudWatch-style metrics for global-cluster writer Region and replica count.
5. **(Design)** Design an RDS PostgreSQL deployment for a billing system with strict RTO requirements and moderate reporting load. Choose Multi-AZ, replicas, backups, and monitoring.
6. **(Migration)** List the prerequisites for DMS CDC from Oracle to Aurora PostgreSQL. Include source logging, privileges, network, LOB handling, and validation.
7. **(Governance)** Create a cutover checklist for an ERP migration that includes technical, financial, security, and business sign-offs.
8. **(Architecture)** Extend fig. 2.3 with S3, Glue, and Redshift for analytics offload. Explain how you avoid reporting queries against the OLTP writer.

2.10 Capstone Project: Migrating and Operating a Cross-Region Aurora PostgreSQL Fleet for Oracle ERP Cutover

Capstone Project

Mission. Migrate and operate a highly available, cross-region Aurora PostgreSQL fleet for an Oracle ERP cutover. Your platform must support schema conversion, full-load migration, CDC replication, validation, planned cutover, regional failover, and post-cutover operations.

Estimated time. 6–8 hours in a sandbox AWS account for infrastructure and rehearsal artifacts, plus optional time for application SQL remediation and performance testing.

What you'll practice.

- Translating Oracle ERP migration requirements into Aurora PostgreSQL capacity, HA, backup, monitoring, security, and cross-region recovery decisions.
- Using SCT assessment outputs and DMS full-load-plus-CDC tasks to reduce downtime during heterogeneous migration.
- Verifying Aurora Global Database or cross-region read-replica readiness, failover evidence, replication lag, and application endpoint cutover.
- Producing business-facing acceptance criteria for reconciliation, rollback, and operational handoff.

Scenario

A manufacturing enterprise is replacing an on-premises Oracle ERP database with Aurora PostgreSQL. The ERP supports procurement, inventory, manufacturing execution, sales orders, shipping, accounts payable, and general ledger. Downtime must be minimized because warehouses and production lines operate across time zones. The source database contains decades of transactional history, Oracle packages, triggers, sequences, materialized views, and nightly batch jobs.

The target platform must run in a primary AWS Region with high availability across Availability

Zones and a secondary Region for disaster recovery. The migration team must prove that DMS can keep the target current, that SCT conversion gaps are remediated or accepted, that critical business totals reconcile, and that operators can promote the secondary Region if the primary Region is unavailable.

Requirements

Functional requirements.

- Produce an SCT assessment summary that classifies schema objects as automatically converted, manually remediated, retired, or deferred.
- Provision Aurora PostgreSQL with encryption, backup retention, deletion protection, at least one reader, Performance Insights, database logs, and restricted network access.
- Configure cross-region recovery using Aurora Global Database or a documented cross-region replica pattern with explicit RPO and RTO targets.
- Create DMS source and target endpoints, table mappings, and a full-load-plus-CDC task for ERP schemas selected for the cutover rehearsal.
- Validate row counts, sample checksums, sequence alignment, referential integrity, and business totals for orders, inventory, invoices, and ledger balances.
- Define cutover steps that freeze or control writes, drain replication lag, switch application endpoints, verify smoke tests, and preserve rollback evidence.

Non-functional requirements.

- The target database must be highly available within the primary Region and recoverable in the secondary Region.
- Secrets must be stored outside scripts, preferably in AWS Secrets Manager, and database access must follow least privilege.
- DMS replication lag, task errors, table errors, database CPU, memory, storage, connections, deadlocks, and slow queries must be observable.
- The migration must have a named cutover owner, rollback owner, validation owners, and business sign-off process.
- The final design must avoid analytical reporting against the Aurora writer by using replicas, S3 exports, or downstream analytical stores.

Reference Architecture

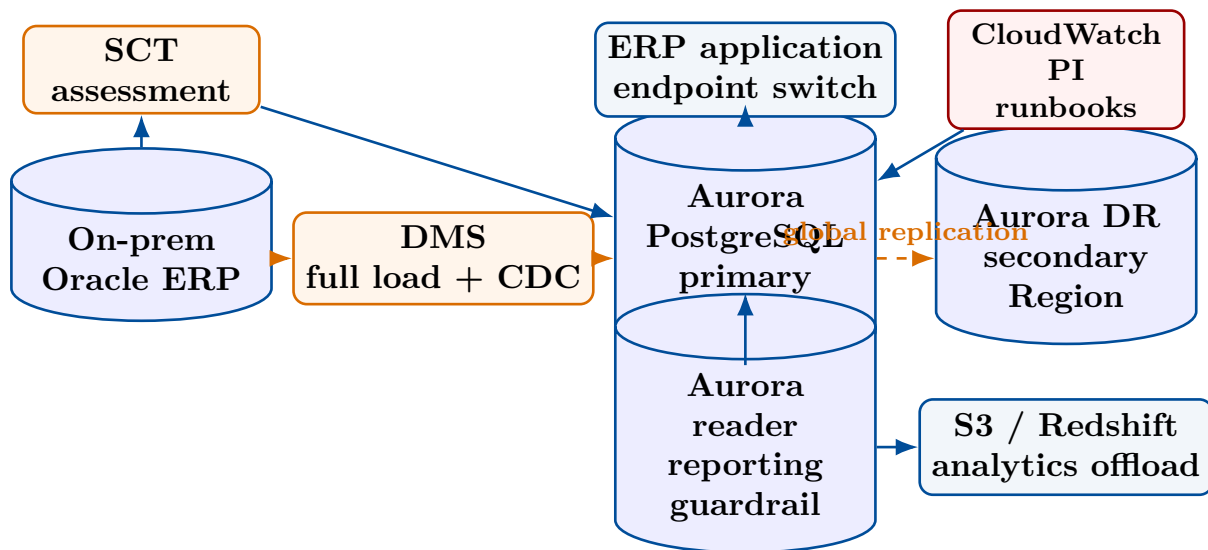


Figure 2.4: Capstone reference architecture for an Oracle ERP migration to Aurora PostgreSQL with DMS CDC, SCT conversion, high availability, cross-region recovery, observability, and analytics offload.

Implementation Steps

1. **Assess and classify.** Run SCT against the Oracle source, export the assessment, and classify incompatible objects by business criticality and remediation owner.
2. **Provision Aurora.** Create the Aurora PostgreSQL cluster, readers, subnet groups, security groups, parameter groups, Secrets Manager entries, backups, logs, Performance Insights, and deletion protection.
3. **Configure regional recovery.** Create a global database or cross-region replica, document RPO and RTO, and run a planned switchover or failover rehearsal.
4. **Create DMS migration tasks.** Configure source and target endpoints, table mappings, full-load-plus-CDC settings, validation, logging, and CloudWatch alarms.
5. **Reconcile data.** Compare row counts, checksums, sequence values, totals, and application smoke-test outputs. Record exceptions and owners.
6. **Execute cutover rehearsal.** Freeze writes or enable controlled dual-write, wait for CDC lag to drain, switch a test application endpoint, validate transactions, and document rollback steps.

```

1 $Region = "us-east-1"
2 $DrRegion = "us-west-2"
3 $ClusterId = "erp-prod-aurora-pg"
4 $GlobalClusterId = "erp-prod-global"
5 $DbName = "erp"
6 $SubnetGroup = "erp-private-db-subnets"
7 $SecurityGroup = "sg-0123456789abcdef0"
8 $KmsKeyId = "arn:aws:kms:us-east-1:123456789012:key/aurora-key"
9
10 aws rds create-global-cluster '
11   --global-cluster-identifier $GlobalClusterId '
12   --engine aurora-postgresql '

```

```

13 --engine-version 16.4 '
14 --region $Region
15
16 aws rds create-db-cluster '
17 --db-cluster-identifier $ClusterId '
18 --global-cluster-identifier $GlobalClusterId '
19 --engine aurora-postgresql '
20 --engine-version 16.4 '
21 --database-name $DbName '
22 --manage-master-user-password '
23 --storage-encrypted '
24 --kms-key-id $KmsKeyId '
25 --backup-retention-period 14 '
26 --deletion-protection '
27 --enable-cloudwatch-logs-exports postgresql '
28 --serverless-v2-scaling-configuration MinCapacity=2,MaxCapacity=32 '
29 --db-subnet-group-name $SubnetGroup '
30 --vpc-security-group-ids $SecurityGroup '
31 --region $Region
32
33 aws rds create-db-instance '
34 --db-instance-identifier "$ClusterId-writer-1" '
35 --db-cluster-identifier $ClusterId '
36 --engine aurora-postgresql '
37 --db-instance-class db.serverless '
38 --enable-performance-insights '
39 --region $Region

```

Listing 2.4: PowerShell-oriented AWS CLI skeleton for Aurora PostgreSQL fleet creation.

```

1 $DmsReplicationInstanceArn = "arn:aws:dms:us-east-1:123456789012:rep:
  ABCDEF123456"
2 $OracleSecretArn = "arn:aws:secretsmanager:us-east-1:123456789012:secret:oracle
  -erp"
3 $AuroraSecretArn = "arn:aws:secretsmanager:us-east-1:123456789012:secret:aurora
  -erp"
4 $OracleEndpointId = "oracle-erp-source"
5 $AuroraEndpointId = "aurora-pg-target"
6 $TaskId = "oracle-erp-to-aurora-cdc"
7
8 aws dms create-endpoint '
9 --endpoint-identifier $OracleEndpointId '
10 --endpoint-type source '
11 --engine-name oracle '
12 --database-name ERPDDB '
13 --secrets-manager-secret-id $OracleSecretArn '
14 --region us-east-1
15
16 aws dms create-endpoint '
17 --endpoint-identifier $AuroraEndpointId '
18 --endpoint-type target '
19 --engine-name aurora-postgresql '
20 --database-name erp '
21 --secrets-manager-secret-id $AuroraSecretArn '
22 --region us-east-1
23
24 $TableMappings = @"
25 {
26   "rules": [
27     {
28       "rule-type": "selection",
29       "rule-id": "include-erp-schema",
30       "rule-name": "include-erp-schema",
31       "object-locator": { "schema-name": "ERP", "table-name": "%" },

```

```

32     "rule-action": "include"
33   }
34 ]
35 }
36 "@
37 $TableMappings | Set-Content -Path ch2-dms-table-mappings.json -Encoding ascii
38
39 aws dms create-replication-task '
40   --replication-task-identifier $TaskId '
41   --source-endpoint-arn "arn:aws:dms:us-east-1:123456789012:endpoint:SOURCE" '
42   --target-endpoint-arn "arn:aws:dms:us-east-1:123456789012:endpoint:TARGET" '
43   --replication-instance-arn $DmsReplicationInstanceArn '
44   --migration-type full-load-and-cdc '
45   --table-mappings file://ch2-dms-table-mappings.json '
46   --replication-task-settings file://ch2-dms-task-settings.json '
47   --region us-east-1

```

Listing 2.5: PowerShell-oriented AWS CLI skeleton for DMS endpoints and a full-load-plus-CDC task.

```

1  import json
2  import os
3  from pathlib import Path
4
5  import boto3
6
7  region = os.environ.get("AWS_REGION", "us-east-1")
8  cluster_id = os.environ["AURORA_CLUSTER_ID"]
9  task_id = os.environ["DMS_TASK_ID"]
10
11 rds = boto3.client("rds", region_name=region)
12 dms = boto3.client("dms", region_name=region)
13
14 cluster = rds.describe_db_clusters(DBClusterIdentifier=cluster_id)["DBClusters"]
15             [0]
16 instances = rds.describe_db_instances()["DBInstances"]
17 cluster_instances = [
18     instance for instance in instances
19     if instance.get("DBClusterIdentifier") == cluster_id
20 ]
21 tasks = dms.describe_replication_tasks(
22     Filters=[{"Name": "replication-task-id", "Values": [task_id]}]
23 )["ReplicationTasks"]
24 assert tasks, f"DMS task {task_id} not found"
25 task = tasks[0]
26
27 summary = {
28     "cluster_id": cluster_id,
29     "engine": cluster["Engine"],
30     "engine_version": cluster["EngineVersion"],
31     "multi_az_members": cluster.get("AvailabilityZones", []),
32     "backup_retention_days": cluster["BackupRetentionPeriod"],
33     "deletion_protection": cluster.get("DeletionProtection"),
34     "cluster_endpoint": cluster.get("Endpoint"),
35     "reader_endpoint": cluster.get("ReaderEndpoint"),
36     "instance_count": len(cluster_instances),
37     "dms_task_status": task["Status"],
38     "dms_migration_type": task["MigrationType"],
39     "dms_stop_reason": task.get("StopReason"),
40 }
41
42 Path("ch2-capstone-evidence.json").write_text(
43     json.dumps(summary, indent=2), encoding="utf-8"

```

```
44 )  
45 print(json.dumps(summary, indent=2))
```

Listing 2.6: Boto3 evidence collector for DMS task status and Aurora cluster posture.

Deliverables

- Architecture summary with primary Region, secondary Region, RPO, RTO, Aurora sizing, backup policy, security posture, and failover plan.
- SCT assessment summary showing automatic conversions, manual remediations, retired objects, and owners for unresolved issues.
- Aurora configuration evidence: engine version, endpoints, readers, backup retention, encryption, deletion protection, Performance Insights, logs, and parameter groups.
- DMS artifacts: endpoint definitions, table mappings, task settings, validation settings, CloudWatch alarms, and replication-lag evidence.
- Reconciliation packet with row counts, checksums or sample comparisons, sequence validation, and business totals for critical ERP domains.
- Cutover and rollback runbook with command sequence, approval gates, communications, smoke tests, and rollback criteria.

Acceptance Criteria

- Aurora PostgreSQL is deployed with encryption, backups, deletion protection, monitoring, and at least one reader in the primary Region.
- Cross-region recovery is configured and the team has documented whether failover is planned switchover, unplanned failover, or restore from backup.
- DMS full-load-plus-CDC task reaches running or load-complete status in rehearsal, and replication lag is measured before cutover.
- SCT findings are triaged; no critical Oracle package, trigger, or scheduler dependency is left without an owner or disposition.
- Validation covers both technical counts and business reconciliation for orders, inventory, invoices, and ledger balances.
- Application endpoint switching is rehearsed without hardcoded database hostnames in application code or batch jobs.
- Rollback criteria are explicit, time-bound, and approved by business and technical owners before the cutover window.
- Analytical workloads are routed away from the Aurora writer through readers, S3 exports, Redshift, or another downstream store.

Stretch Goals

- Add AWS DMS premigration assessment and validation reports to the acceptance packet, including table-level error thresholds.
- Use AWS Secrets Manager rotation for Aurora credentials and document how applications refresh connections safely.

- Add RDS Proxy or PgBouncer-style pooling for application connection storms, then test failover behavior with pooled connections.
- Export selected Aurora tables to S3 and build a Redshift or Athena reporting path that eliminates ad hoc reporting against the writer.
- Build CloudWatch dashboards for ACU usage, CPU, database connections, deadlocks, commit latency, replica lag, DMS CDCLatencySource, and CDCLatencyTarget.
- Run a game day that simulates primary-region unavailability, promotes the secondary database, switches a read-only application, and documents failback reconciliation.

High-Scale NoSQL: Amazon DynamoDB

Learning Objectives

- Explain DynamoDB core concepts: tables, items, attributes, partitions, request routing, read capacity units, write capacity units, and capacity modes.
- Model high-scale access patterns with partition keys, sort keys, sparse indexes, local secondary indexes, and global secondary indexes.
- Describe DynamoDB Streams, time to live, and Global Tables for event-driven integration, lifecycle management, and multi-region active-active designs.
- Create a DynamoDB table, design a single-table schema for a forum workload, insert items with Boto3, and configure a GSI for query access.
- Design an ingestion and serving layer for a global gaming leaderboard that handles one million reads and writes per second with predictable hot-key controls.
- Build a capstone data layer using single-table design, GSIs, Streams, Global Tables, and on-demand scaling.

Cross-Cloud Equivalence

NoSQL services differ in APIs and consistency models, but the recurring design vocabulary is familiar: key distribution, item or document shape, secondary access paths, change feeds, expiration, and multi-region replication.

Concept	AWS DynamoDB	Dy-	Azure DB	Cosmos	GCP Firestore / Bigtable	MongoDB
Key-value / document store	Table of items with scalar, map, list, set, and document attributes		Container with JSON documents	items with JSON documents	Firestore documents or Bigtable rows and column families	Collections of BSON documents
Partition key	Hash key distributes items across physical partitions		Partition key routes items to logical partitions		Firestore document path or Bigtable row-key prefix design	Shard key when sharding is enabled
Sort key	Optional range key orders related items within one partition key		Composite keys are modeled in item fields and indexed		Firestore indexed fields; Bigtable row-key suffix patterns	Compound index fields or embedded sortable attributes
Secondary indexes	GSI for alternate partition or sort keys; LSI for alternate sort key on same partition key		Automatic or defined indexes with partition-aware query design		Firestore composite indexes; Bigtable secondary indexes are limited and often modeled manually	Secondary and compound indexes
Streams / change feed	DynamoDB Streams with Lambda, Kinesis, or consumers		Change feed processor		Firestore triggers; Bigtable change streams	Change streams on replica sets or sharded clusters
TTL	Per-item TTL attribute deletes expired items asynchronously		Time to live on containers or items		Firestore TTL policies; Bigtable garbage-collection rules	TTL indexes on date fields
Global tables / multi-region	Global replicate active tables across Regions	Tables multi-	Multi-region writes with tunable consistency		Firestore multi-region locations; Bigtable replication	Atlas global clusters and zone sharding

The transferable skill is not memorizing product names. It is learning to state access patterns first, choose keys that distribute traffic, and add only the secondary indexes needed to satisfy known queries.

3.1 Week 3 Roadmap and Mental Model

DynamoDB is AWS's managed, serverless NoSQL database for key-value and document workloads that need single-digit millisecond latency at very high scale [7]. Unlike relational systems, DynamoDB does not reward late query discovery. The data model is intentionally shaped around known access patterns, expected cardinality, item size, consistency needs, and write distribution. A data engineer uses DynamoDB when the workload needs operational serving speed, elastic scale, event-driven integration, or multi-region active-active availability more than ad hoc joins.

Definition

Amazon DynamoDB is a fully managed NoSQL key-value and document database. Applications read and write items in tables using primary keys, optional secondary indexes, capacity modes, and service-managed partitioning.

Key Terms

Item: one record in a table. **Attribute:** one named value on an item. **Partition key:** hash key used to distribute and locate data. **Sort key:** optional range key used to order related items. **GSI:** global secondary index with its own key schema. **LSI:** local secondary index with the table partition key and alternate sort key.

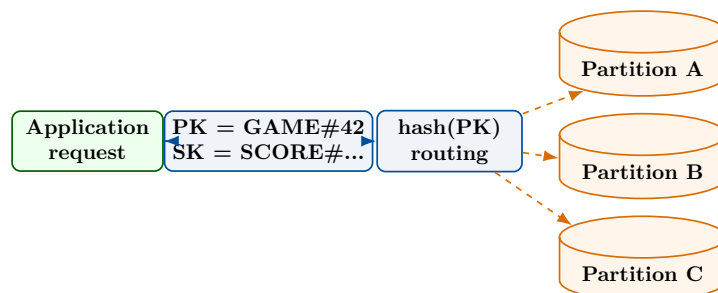
3.2 Monday: DynamoDB Core Concepts, Partitions, and Capacity

3.2.1 Tables, Items, and Physical Partitions

A DynamoDB table stores items. Each item is a set of attributes and can have a different shape from its neighbors. The required key attributes define how the service locates data. With a simple primary key, the partition key uniquely identifies an item. With a composite primary key, the partition key groups related items and the sort key uniquely orders items inside that group. Internally, DynamoDB hashes the partition key to route requests to physical partitions that hold data and serve capacity [18].

Note

DynamoDB is schema-flexible, not schema-free. The table does not enforce every attribute, but the application contract must still define entity types, required attributes, key formats, index projections, and lifecycle rules.



A good partition key spreads writes and reads across many hash values while preserving efficient item collections for known queries.

Figure 3.1: DynamoDB hashes the partition key to route items and requests to physical partitions. Sort keys organize related items after the partition key has chosen the item collection.

3.2.2 Read and Write Capacity

DynamoDB meters throughput in read capacity units and write capacity units for provisioned mode, or request units under on-demand billing. One write capacity unit supports one 1 KB write per second. Strongly consistent reads consume more capacity than eventually consistent reads, and larger items require more units. Capacity is consumed by the table and by indexes because index entries are separate write targets [20].

Definition

RCU and **WCU** are capacity abstractions used by provisioned DynamoDB tables. They make item size, read consistency, and request rate visible so teams can plan cost and throttling risk.

Tip

Estimate capacity from access patterns, not from table size alone. A small table with a celebrity user, global match, or viral post can throttle if the key design concentrates traffic on one partition key.

3.2.3 Provisioned Versus On-Demand Capacity

Provisioned capacity is appropriate when traffic is predictable, budgets require explicit limits, or autoscaling can follow a known pattern. On-demand capacity is appropriate for spiky, unknown, or early-stage workloads where operational simplicity and burst tolerance matter more than the lowest possible steady-state cost. Both modes still require key designs that avoid hot partitions.

Common Pitfall

Assuming on-demand fixes bad keys. On-demand mode removes capacity planning for the table, but it does not make one partition key able to absorb unlimited traffic. Hot keys still need write sharding, bucketing, caching, aggregation, or a different access pattern.

3.3 Tuesday: Data Modeling with Primary Keys, Sort Keys, LSI, and GSI**3.3.1 Access-Pattern-First Modeling**

Relational modeling starts from entities and normalization. DynamoDB modeling starts from questions the application must answer. For a forum, those questions might be: load a forum by slug, list threads in a forum by recent activity, load posts in a thread by creation time, list a user's posts, show moderation queues by status, and retrieve leaderboard summaries. Each access pattern becomes a key choice, index choice, or denormalized item.

Definition

Single-table design stores multiple entity types in one DynamoDB table and uses overloaded key attributes to colocate items that are read together. It trades relational normalization for predictable, low-latency access patterns.

Access pattern	PK	SK	Index
Get forum metadata	FORUM#slug	META	Table primary key
List threads in forum	FORUM#slug	THREAD#lastActivity#threadId	Table primary key
Load posts in thread	THREAD#threadId	POST#createdAt#postId	Table primary key
List posts by user	USER#userId	POST#createdAt#postId	GSI1
Moderation queue	STATUS#pending	REPORTED#createdAt#postId	GSI2, Global secondary index

Table 3.1: Example single-table access patterns for a forum workload.

3.3.2 Primary Keys and Sort-Key Encoding

Partition keys should have enough cardinality to distribute load. Sort keys should encode hierarchy and ordering in a way that supports begins-with, between, forward order, and reverse order queries. Common patterns include prefixes such as **FORUM#**, **THREAD#**, and **POST#**; ISO-8601 timestamps for lexical ordering; and inverted scores or padded numbers when descending order is needed.

Note

DynamoDB query operations require an equality condition on the partition key. Sort-key conditions refine the item collection after the partition key is chosen. A query that starts with only a non-key attribute is not a DynamoDB query; it is usually an index-design problem.

3.3.3 Local and Global Secondary Indexes

An LSI is created with the table, shares the table partition key, and provides an alternate sort key for items in the same item collection. A GSI can be created after table creation and has its own partition key and optional sort key. GSIs are the main tool for alternate access patterns, but they add write cost, storage cost, replication delay, and operational complexity [23].

Tip

Use sparse GSIs deliberately. If only items with GSI1PK and GSI1SK attributes appear in the index, the GSI can become a compact queue, lookup table, or status-specific access path.

Common Pitfall

Adding indexes after every new question. Each GSI duplicates selected attributes and receives writes. Too many indexes turn one application write into many index writes and can make hot access patterns harder to diagnose.

3.4 Wednesday: Streams, TTL, and Global Tables

3.4.1 DynamoDB Streams

DynamoDB Streams capture item-level changes in near real time. A stream record can contain keys only, new image, old image, or both images. Lambda functions, stream consumers, and Kinesis integrations use these records to update search indexes, maintain materialized views, publish events, enrich lake ingestion, or invalidate caches [21].

Definition

DynamoDB Streams is a time-ordered change log for item modifications in a table. It enables event-driven processing without polling the table for recently changed items.

Note

Stream consumers must be idempotent. Lambda retries, partial failures, and duplicate event processing are normal distributed-systems concerns, so downstream writes should use deterministic keys or conditional updates.

3.4.2 Time to Live

TTL marks items for asynchronous expiration using an epoch-time attribute. It is useful for sessions, temporary leaderboards, deduplication records, invitations, cached computations, and retention controls. TTL deletion is not immediate and should not be used when exact deletion time is a hard compliance boundary [21].

Common Pitfall

Treating TTL as a scheduler. TTL is a lifecycle hint, not a cron system. If business logic must execute exactly at expiration, use EventBridge, Step Functions, or a workflow service and treat TTL as cleanup.

3.4.3 Global Tables

Global Tables replicate DynamoDB tables across AWS Regions for multi-active applications. Applications can read and write locally in each participating Region, and DynamoDB replicates changes. This reduces latency for global users and improves regional resilience, but it also requires conflict-aware design, deterministic writes, and careful observability [19].

Tip

Global tables work best when each write has a clear owner or monotonic version strategy. For leaderboards, make the score update conditional on a higher score or newer version so replicated writes do not accidentally move a player backward.

3.5 Thursday Hands-on Project: Forum Single-Table Schema and GSI

This lab creates a DynamoDB table for a forum, inserts representative single-table items, and configures a GSI for user-centric queries. The commands assume Windows PowerShell with AWS CLI v2 and a sandbox AWS account. Replace account-specific values and clean up resources after testing.

Note

The lab uses on-demand capacity for simplicity. In production, choose on-demand or provisioned capacity from measured access patterns, expected peak traffic, budget controls, and throttling tolerance.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $Region = "us-east-1"
3 $TableName = "forum-single-table-demo"
4
5 aws dynamodb create-table '
6   --table-name $TableName '
7   --attribute-definitions '
8     AttributeName=PK,AttributeType=S '
9     AttributeName=SK,AttributeType=S '
10    AttributeName=GSI1PK,AttributeType=S '
11    AttributeName=GSI1SK,AttributeType=S '
12   --key-schema '
13     AttributeName=PK,KeyType=HASH '
14     AttributeName=SK,KeyType=RANGE '
15   --global-secondary-indexes "[
16     {
17       \"IndexName\": \"GSI1\",
18       \"KeySchema\": [
19         {\"AttributeName\": \"GSI1PK\", \"KeyType\": \"HASH\"},
20         {\"AttributeName\": \"GSI1SK\", \"KeyType\": \"RANGE\"}
21       ],
22       \"Projection\": {\"ProjectionType\": \"ALL\"}
23     }
24   ]" '

```

```

25 --billing-mode PAY_PER_REQUEST '
26 --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
27 --region $Region
28
29 aws dynamodb wait table-exists --table-name $TableName --region $Region
30
31 aws dynamodb update-time-to-live '
32 --table-name $TableName '
33 --time-to-live-specification "Enabled=true,AttributeName=expiresAt" '
34 --region $Region

```

Listing 3.1: PowerShell-oriented AWS CLI commands to create a DynamoDB table with a GSI.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os
4 import boto3
5
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 table_name = os.environ.get("TABLE_NAME", "forum-single-table-demo")
8 table = boto3.resource("dynamodb", region_name=region).Table(table_name)
9 now = datetime.now(timezone.utc).isoformat()
10 forum_slug, thread_id, user_id = "data-engineering", "thread-1001", "user-42"
11
12 items = [
13     {"PK": f"FORUM#{forum_slug}", "SK": "META", "entityType": "Forum", "title":
14       "Data Engineering", "createdAt": now},
15     {"PK": f"FORUM#{forum_slug}", "SK": f"THREAD#{now}#{thread_id}",
16       "entityType": "Thread", "threadId": thread_id, "title": "DynamoDB modeling
17       patterns", "GSI1PK": f"USER#{user_id}", "GSI1SK": f"THREAD#{now}#{thread_id}"
18     },
19     {"PK": f"THREAD#{thread_id}", "SK": f"POST#{now}#post-1", "entityType": "
20       Post", "postId": "post-1", "authorUserId": user_id, "body": "Start with
21       access patterns.", "score": Decimal("1"), "GSI1PK": f"USER#{user_id}", "
22       GSI1SK": f"POST#{now}#post-1", "expiresAt": int((datetime.now(timezone.utc)
23       + timedelta(days=365)).timestamp())},
24 ]
25
26 with table.batch_writer() as batch:
27     for item in items:
28         batch.put_item(Item=item)
29
30 response = table.query(IndexName="GSI1", KeyConditionExpression="GSI1PK = :user
31   ", ExpressionAttributeValues={" :user": f"USER#{user_id}"}, ScanIndexForward=
32   False)
33 for item in response["Items"]:
34     print(item["entityType"], item["SK"])

```

Listing 3.2: Boto3 script to insert forum items and query a GSI by user.

3.6 Friday Use Case: Global Gaming Leaderboard at One Million Reads and Writes Per Second

A gaming company runs tournaments in North America, Europe, Asia, and South America. Every match emits score events, profile updates, anti-cheat flags, and session telemetry. Players expect local writes, fresh regional leaderboards, and fast reads from mobile clients. The platform target is one million reads and writes per second globally, using DynamoDB Global Tables for active-active serving, Streams for derived views, and cache layers for extreme fan-out.

3.6.1 Architecture Narrative

The write path accepts score events through regional APIs. Each event uses an idempotency key so retries cannot double-count a match. The score item is written to a regional DynamoDB replica with a partition key that includes game, season, region, and bucket. A conditional expression updates the player's best score only if the new score is higher or has a newer authoritative version. Global Tables replicate the update to peer Regions. Streams publish changes to Lambda or Kinesis consumers that maintain top-N materialized leaderboard items, feed S3 analytics, and invalidate caches.

The read path separates personalized reads from hot global rankings. Player profile and rank lookups go directly to DynamoDB or DAX where appropriate [6]. Extremely hot top-N pages are precomputed from streams and served from a cache or CDN-backed API. This keeps millions of clients from repeatedly querying the same partition key during a tournament final.

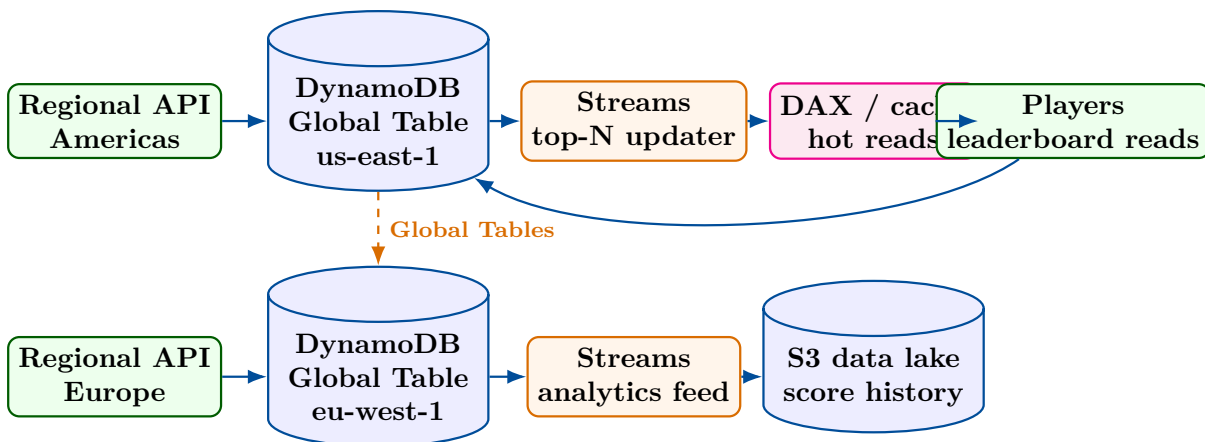


Figure 3.2: Global gaming leaderboard architecture with regional writes to DynamoDB Global Tables, streams for derived top-N views and lake ingestion, and cache or DAX layers for hot read serving.

3.6.2 Design Decisions

Write distribution. Avoid a single partition key such as `GAME#123#SEASON#2026` for all score writes. Add buckets, region segments, or match identifiers so writes spread across many partition keys, then compute aggregate top-N views asynchronously.

Correctness. Use conditional writes for best score, optimistic version attributes for profile changes, and idempotency items for score-event ingestion. Streams consumers must tolerate duplicate records and out-of-order cross-region observations.

Serving. Use DynamoDB for player-specific and partitioned rank queries. Use stream-maintained materialized top-N items plus cache for the hottest pages. DAX can reduce read latency for read-heavy DynamoDB access patterns, but it does not replace good key design.

Common Pitfall

Trying to query the whole leaderboard live. A global top-N query over every player on demand is an analytical operation. Operational leaderboards should maintain precomputed slices for each game, season, region, league, and time window.

3.7 Interview Q&A

3.7.1 Concepts and Trade-offs

1. **Q: How is DynamoDB different from a relational database?**

A: DynamoDB is a managed key-value and document database optimized for predictable access patterns and high scale. It does not provide joins or ad hoc relational querying; the table and indexes are modeled around known queries.

2. **Q: What makes a DynamoDB partition key good?**

A: It has high cardinality, distributes reads and writes evenly, and still groups items that must be queried together. It avoids hot keys created by globally popular users, games, or status values.

3. **Q: When would you choose on-demand capacity?**

A: Choose it for unknown, bursty, or early-stage traffic where operational simplicity matters. For predictable steady traffic, provisioned capacity with autoscaling may be more cost controlled.

4. **Q: What is the difference between an LSI and a GSI?**

A: An LSI shares the table partition key and provides an alternate sort key created with the table. A GSI has its own key schema, can be created later, and supports alternate access patterns.

5. **Q: Why are sparse GSIs useful?**

A: Only items containing the index key attributes appear in the index, so a sparse GSI can efficiently represent queues, status views, or selected entity types without indexing every item.

6. **Q: What are DynamoDB Streams used for?**

A: They capture item changes for event-driven processing such as cache invalidation, materialized views, search indexing, audit trails, and lake ingestion.

7. **Q: What caveat applies to TTL?**

A: TTL expiration is asynchronous. It is useful for cleanup and lifecycle management, but not for exact-time scheduling or strict deletion deadlines.

8. **Q: What extra design work do Global Tables require?**

A: They require conflict-aware write rules, regional observability, deterministic updates, idempotent consumers, and application routing that understands multi-region behavior.

3.8 Further Reading

Start with the DynamoDB Developer Guide for table design, APIs, indexes, streams, TTL, backup, and security behavior [7]. Study capacity mode, adaptive capacity, and throttling guidance before estimating costs [20, 18]. Review secondary index design and projection choices for every non-primary access pattern [23]. For event-driven and multi-region workloads, read the Streams, TTL, Global Tables, and DAX documentation [21, 19, 6].

Key Takeaways

- DynamoDB design starts with access patterns, traffic distribution, item size, consistency, and latency requirements.

- Partition keys distribute data and capacity; sort keys organize related items for efficient range and prefix queries.
- Provisioned capacity gives explicit throughput control, while on-demand capacity simplifies bursty workloads; neither mode fixes a hot key.
- GSIs support alternate access patterns, but each index adds write cost, storage cost, and replication behavior to understand.
- Streams, TTL, and Global Tables extend DynamoDB into event-driven, lifecycle-managed, and multi-region architectures.
- Large-scale leaderboards need write sharding, idempotency, materialized views, and cache-aware read serving rather than live global scans.

3.9 Exercises

1. **(Conceptual)** Explain why a high-cardinality partition key matters for both cost and availability under heavy write traffic.
2. **(Conceptual)** Compare provisioned and on-demand capacity for a workload with predictable weekday peaks and unpredictable weekend tournaments.
3. **(Modeling)** Extend table 3.1 with an access pattern for unread notifications per user.
4. **(Hands-on)** Run listing 3.1 in a sandbox account, then capture table status, stream ARN, TTL status, and GSI status.
5. **(Hands-on)** Modify listing 3.2 to insert three posts by two users and query each user's content through GSI1.
6. **(Design)** Design a sparse GSI for a moderation queue that only includes reported posts with status pending, escalated, or blocked.
7. **(Operations)** Write an alarm plan for throttled requests, system errors, replication latency, stream iterator age, and cache hit ratio.
8. **(Architecture)** Sketch a leaderboard write-sharding strategy for one million global writes per second and explain how top-N views are rebuilt.

3.10 Capstone Project: Building a Global Gaming-Leaderboard Data Layer

Capstone Project

Mission. Build the data layer for a global gaming leaderboard using DynamoDB single-table design, GSIs, Streams, Global Tables, and on-demand scaling. The platform must support regional score writes, player rank lookups, hot top-N serving, analytics export, and operational evidence.

Estimated time. 5–7 hours in a sandbox AWS account for table design, scripted inserts, stream integration stubs, global-table configuration, and acceptance evidence.

What you'll practice.

- Translating leaderboard access patterns into partition keys, sort keys, item collections, and GSIs.
- Designing write-sharded score ingestion with idempotency and conditional updates.

- Using Streams to maintain materialized top-N views and send immutable score history to analytics storage.
- Configuring Global Tables and on-demand scaling for multi-region active-active serving.

Scenario

A competitive gaming company is launching a world championship across four Regions. Players submit scores after every match, mobile clients read personal rank after each game, and spectators refresh top leaderboards during finals. The business requires low-latency regional writes, globally replicated player best scores, and hot read paths that can absorb sudden fan traffic. Data science teams also need score history in S3 for fraud detection and tournament analytics.

The data layer must use DynamoDB as the operational source of truth. It should avoid global scans, use GSIs only where access patterns require them, and produce evidence that streams, TTL, Global Tables, and capacity settings are configured correctly.

Requirements

Functional requirements.

- Design a single-table schema for games, seasons, players, score events, player best scores, regional leaderboard buckets, and top-N materialized views.
- Use a composite primary key with write-sharded leaderboard partitions for high-ingestion score events.
- Create at least one GSI for player-centric queries and one sparse GSI for tournament moderation or anti-cheat review.
- Enable Streams with new and old images so consumers can update top-N views and export score history.
- Enable TTL for temporary idempotency records, session records, and short-lived tournament cache items.
- Configure Global Tables across at least two Regions or document the exact commands and acceptance evidence required when sandbox limits prevent creation.

Non-functional requirements.

- The design must sustain hot tournaments without a single partition key receiving all writes.
- All score updates must be idempotent and conditional so retries or replicated writes do not corrupt best-score state.
- Read paths must separate personal rank lookups, regional top-N queries, and global spectator traffic.
- Monitoring must include throttling, consumed capacity, user errors, system errors, stream iterator age, replication latency, and cache hit ratio where a cache is used.
- Scripts must not contain secrets, account credentials, or hardcoded personal data.

Reference Architecture

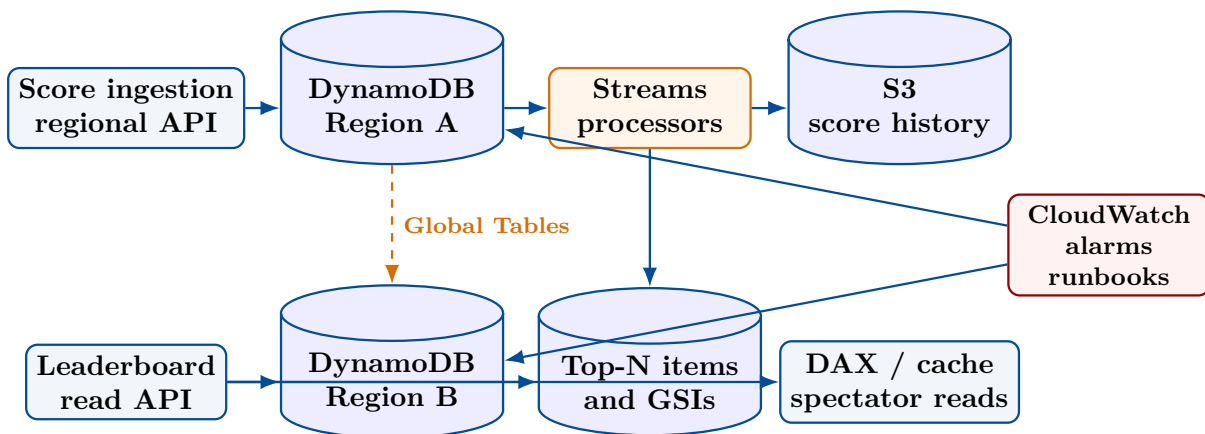


Figure 3.3: Capstone reference architecture for a global gaming-leaderboard data layer with regional APIs, DynamoDB Global Tables, Streams processors, top-N materialized items, cache acceleration, S3 export, and observability.

Implementation Steps

1. **Define access patterns.** List every read and write path: submit score, get player best score, get player rank, list top players by game and season, review flagged events, and export score history.
2. **Design keys.** Choose table PK and SK formats, write-sharded leaderboard bucket keys, GSI keys for player lookups, and sparse GSI keys for anti-cheat queues.
3. **Create the table.** Use on-demand capacity, streams with new and old images, TTL, server-side encryption, point-in-time recovery, and least-privilege IAM.
4. **Write ingestion code.** Insert score events idempotently and update best-score items with conditional expressions.
5. **Build stream consumers.** Maintain top-N materialized items and export immutable events to S3 or a delivery stream.
6. **Configure Global Tables.** Add replica Regions, test regional writes, and record replication status and latency evidence.
7. **Validate scale posture.** Demonstrate write distribution across bucketed keys, confirm hot read paths use materialized views or cache, and document alarms.

```

1 $PrimaryRegion = "us-east-1"
2 $ReplicaRegion = "eu-west-1"
3 $TableName = "global-leaderboard"
4
5 aws dynamodb create-table '
6   --table-name $TableName '
7   --attribute-definitions '
8     AttributeName=PK,AttributeType=S '
9     AttributeName=SK,AttributeType=S '
10    AttributeName=GSI1PK,AttributeType=S '
11    AttributeName=GSI1SK,AttributeType=S '
12    AttributeName=GSI2PK,AttributeType=S '
13    AttributeName=GSI2SK,AttributeType=S '
14    --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE '

```

```

15 --global-secondary-indexes "[
16     {
17         \"IndexName\": \"GSI1PlayerLookup\",
18         \"KeySchema\": [
19             {\"AttributeName\": \"GSI1PK\", \"KeyType\": \"HASH\"},
20             {\"AttributeName\": \"GSI1SK\", \"KeyType\": \"RANGE\"}
21         ],
22         \"Projection\": {\"ProjectionType\": \"ALL\"}
23     },
24     {
25         \"IndexName\": \"GSI2ReviewQueue\",
26         \"KeySchema\": [
27             {\"AttributeName\": \"GSI2PK\", \"KeyType\": \"HASH\"},
28             {\"AttributeName\": \"GSI2SK\", \"KeyType\": \"RANGE\"}
29         ],
30         \"Projection\": {\"ProjectionType\": \"ALL\"}
31     }
32 ]" '
33 --billing-mode PAY_PER_REQUEST '
34 --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
35 --sse-specification Enabled=true '
36 --region $PrimaryRegion
37
38 aws dynamodb wait table-exists --table-name $TableName --region $PrimaryRegion
39
40 aws dynamodb update-continuous-backups '
41     --table-name $TableName '
42     --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true '
43     --region $PrimaryRegion
44
45 aws dynamodb update-table '
46     --table-name $TableName '
47     --replica-updates "Create={RegionName=$ReplicaRegion}" '
48     --region $PrimaryRegion

```

Listing 3.3: PowerShell-oriented AWS CLI skeleton for the leaderboard table and replicas.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os, uuid, boto3
4 from botocore.exceptions import ClientError
5
6 table = boto3.resource("dynamodb", region_name=os.environ.get("AWS_REGION", "us
    -east-1")).Table(os.environ.get("TABLE_NAME", "global-leaderboard"))
7
8 def submit_score(game_id, season_id, player_id, score, event_id=None):
9     now = datetime.now(timezone.utc)
10    event_id = event_id or str(uuid.uuid4())
11    bucket = int(player_id[-2:], 16) % 100
12    try:
13        table.put_item(Item={"PK": f"IDEMPOTENCY#{event_id}", "SK": "EVENT", "
    expiresAt": int((now + timedelta(days=2)).timestamp())}, ConditionExpression
    ="attribute_not_exists(PK)")
14    except ClientError as exc:
15        if exc.response["Error"]["Code"] == "ConditionalCheckFailedException":
16            return {"status": "duplicate", "eventId": event_id}
17        raise
18
19    table.put_item(Item={"PK": f"GAME#{game_id}#SEASON#{season_id}#BUCKET#{
    bucket:02d}", "SK": f"SCORE#{now.isoformat()}#{event_id}", "entityType": "
    ScoreEvent", "playerId": player_id, "score": Decimal(str(score)), "GSI1PK":
    f"PLAYER#{player_id}", "GSI1SK": f"SCORE#{now.isoformat()}#{game_id}"})
20    table.update_item(
21        Key={"PK": f"PLAYER#{player_id}", "SK": f"BEST#{game_id}#{season_id}"},

```

```

22     UpdateExpression="SET entityType=:t, bestScore=:s, updatedAt=:n, GSI1PK
    =:p, GSI1SK=:g",
23     ConditionExpression="attribute_not_exists(bestScore) OR bestScore < :s"
    ,
24     ExpressionAttributeValues={":t": "PlayerBestScore", ":s": Decimal(str(
    score)), ":n": now.isoformat(), ":p": f"PLAYER#{player_id}", ":g": f"BEST#{
    game_id}#{season_id}"},
25     )
26     return {"status": "accepted", "eventId": event_id, "bucket": bucket}
27
28 print(submit_score("space-race", "2026", "af42", 987650))

```

Listing 3.4: Boto3 score ingestion with idempotency and conditional best-score updates.

Deliverables

- Access-pattern matrix with table keys, sort-key formats, GSI keys, expected cardinality, and consistency choice for each query.
- DynamoDB table creation evidence showing billing mode, stream setting, TTL status, point-in-time recovery, encryption, GSIs, and replica Regions.
- Ingestion script with idempotency records, write-sharded score events, conditional best-score updates, and sample output.
- Stream-processing design that explains how top-N views are updated, how duplicate events are handled, and how score history reaches S3.
- Read-serving plan for player rank, regional leaderboard, global top-N, spectator traffic, and cache invalidation.
- Operations packet with CloudWatch alarms, dashboard metrics, failure scenarios, and cleanup steps.

Acceptance Criteria

- The table uses a composite primary key and at least two GSIs: one for player lookups and one sparse index for review or moderation workflows.
- Score writes include idempotency protection and conditional best-score updates.
- Leaderboard writes are distributed across bucketed partition keys rather than concentrated on one game-season key.
- Streams are enabled with new and old images, and the top-N update design is idempotent.
- TTL is enabled for temporary records, and the design does not rely on exact deletion timing.
- Global Tables are configured across at least two Regions or a sandbox exception documents the exact command, required IAM, and expected evidence.
- The read path avoids live global scans and uses materialized views or cache for hot top-N pages.
- Monitoring covers throttles, errors, consumed capacity, stream iterator age, global replication health, and cache behavior.

Stretch Goals

- Add a load-test script that writes synthetic scores across 100 leaderboard buckets and reports key distribution.
- Implement a Lambda stream consumer that maintains top 100 players per game, season, region, and league.
- Add DAX or another cache for player rank reads, then measure hit ratio and stale-read tolerance.

Hybrid Ingestion and Caching

Learning Objectives

- Explain when to accelerate databases with Amazon ElastiCache for Redis or Memcached, and how cache-aside, write-through, TTL, eviction, and invalidation choices affect correctness.
- Describe DynamoDB Accelerator as a managed, API-compatible, in-memory cache for read-heavy DynamoDB workloads.
- Compare AWS AppSync and Amazon API Gateway as managed ingestion front doors for GraphQL, REST, HTTP, and event-driven data APIs.
- Deploy an ElastiCache Redis cluster and write a Python cache-aside script that caches RDS query results to reduce repeated database load.
- Design a high-throughput, low-latency mobile data API that survives live sports traffic spikes with cache-aware serving paths.
- Complete the Part I milestone by integrating S3, Aurora Serverless v2, DynamoDB, ElastiCache, DAX, and AWS DMS into an e-commerce data backend.

Cross-Cloud Equivalence

Caching services vary by API, but the engineering vocabulary transfers across clouds: in-memory acceleration, TTL, eviction, cache-aside, write-through, database-integrated cache, invalidation, and hot-key protection.

Concept	AWS	Azure	GCP	Others
Managed Redis / Memcached	ElastiCache for Redis OSS, Valkey, and Memcached; MemoryDB for durable Redis-compatible workloads	Azure Cache for Redis	Memorystore for Redis and Memorystore for Memcached	Redis Enterprise Cloud, Upstash Redis self-managed Redis or Memcached
Database-integrated cache	DynamoDB Accelerator for API-compatible DynamoDB read caching	Azure Cosmos DB integrated cache	No direct universal equivalent; combine Memorystore with Firestore, Spanner, AlloyDB, or Bigtable patterns	MongoDB Atlas query features plus external Redis caches
Cache-aside	Application checks cache, then queries RDS, Aurora, DynamoDB, or an API and stores the result	Same pattern with Azure SQL, Cosmos DB, or services behind Functions	Same pattern with Cloud SQL, Spanner, Firestore, or Bigtable	Common pattern in open-source Redis, Memcached, and application frameworks
Write-through / write-behind	Application or library writes cache and source together; requires idempotency and failure handling	Same concept with cache plus source write coordination	Same concept; often implemented in service code or stream processors	Available in some cache clients, proxies, and data-grid products
TTL and eviction	Per-key TTL, Redis policies, Memcached LRU behavior, DAX item and query cache TTL	TTL and eviction policies in Azure Cache for Redis; Cosmos DB integrated-cache staleness setting	Memorystore maxmemory policies and TTL support	Product-specific maxmemory, LRU, LFU, TTL, and persistence options
Hot-key control	Shard keys, precompute hot pages, DAX for DynamoDB reads, ElastiCache replicas, API throttling	Redis clustering, replicas, API Management throttles	Memorystore scaling, Cloud CDN, API Gateway quotas	CDN, request coalescing, client backoff, distributed locks, and key bucketing
The transferable skill is not choosing a cache by brand. It is deciding which data can be stale, how stale it may be, who invalidates it, and how the system behaves when the cache is unavailable.				

4.1 Week 4 Roadmap and Mental Model

Weeks 1–3 built the storage foundation: S3 for durable objects, RDS and Aurora for relational records, and DynamoDB for high-scale key-value and document access. Week 4 adds the serving layer that makes those systems usable during real traffic: cache acceleration, managed API ingestion, and hybrid read paths. A cache is not a database replacement. It is a latency and load-management tool placed between clients and a durable source of truth.

Definition

Hybrid ingestion and caching combines durable data stores, in-memory acceleration, and managed API front doors so applications can accept traffic quickly, serve hot reads cheaply, and preserve correct writes in systems of record.

Key Terms

Cache-aside: application reads from cache first, then loads from the database on a miss. **TTL:** time to live for cached values. **Eviction:** removal caused by capacity pressure or policy. **Invalidation:** explicit removal or refresh after source data changes. **Hot key:** cache or database key receiving disproportionate traffic.

4.2 Monday: Amazon ElastiCache for Database Acceleration

4.2.1 Redis, Valkey, and Memcached in the Data Path

Amazon ElastiCache is a managed in-memory service for Redis OSS, Valkey, and Memcached workloads [8]. Redis-compatible engines provide rich data structures, replication, clustering, persistence options, sorted sets, streams, pub/sub, and atomic operations. Memcached is simpler: it is a distributed, in-memory key-value cache optimized for ephemeral objects and horizontal simplicity. A data engineer usually places ElastiCache in front of RDS, Aurora, an API, or a derived view when repeated reads are expensive or latency-sensitive.

Definition

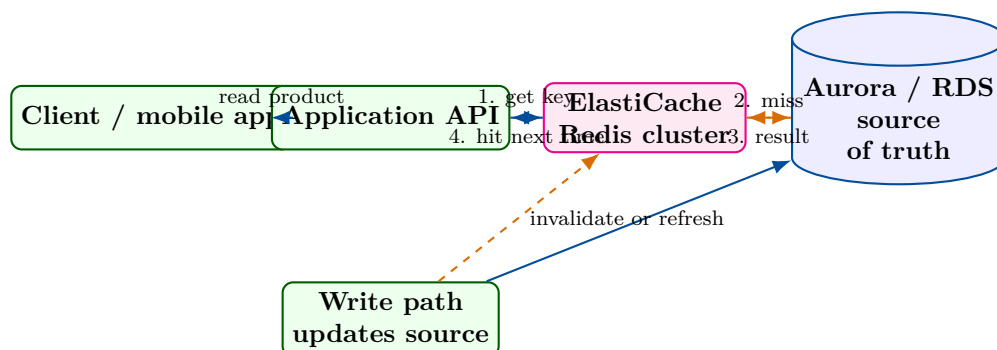
Amazon ElastiCache is a fully managed in-memory cache service. It reduces database load by serving repeated reads from memory and by storing short-lived computations, sessions, counters, and hot aggregates.

Note

ElastiCache improves read latency only when the application repeatedly asks for cacheable data. It will not fix unindexed SQL, missing DynamoDB access patterns, slow network calls, or write bottlenecks unless the architecture changes the path those requests take.

4.2.2 Cache-Aside, Write-Through, TTL, and Invalidation

The most common pattern is cache-aside. The application checks Redis or Memcached. If the key exists, the value is returned. If it misses, the application queries the database, serializes the result, stores it with a TTL, and returns it. This pattern keeps the cache disposable and easy to rebuild. The cost is application complexity: every caller must agree on key names, serialization, TTL, error handling, and invalidation.



Cache-aside keeps Redis disposable, but correctness depends on TTL selection and write-path invalidation.

Figure 4.1: Cache-aside flow between an application, ElastiCache Redis, and an RDS or Aurora source of truth.

Write-through writes data to the cache and database together. It can make read-after-write behavior more predictable, but it creates more failure modes. If the cache write succeeds and the database write fails, the cache may advertise data that is not durable. If the database write succeeds and the cache write fails, reads may stay stale until TTL or invalidation. For most operational systems, cache-aside plus precise invalidation is easier to reason about.

Tip

Start with short TTLs and explicit invalidation for data that users can change. Use longer TTLs for reference data, product catalog fragments, feature flags, and computed summaries where bounded staleness is acceptable.

Common Pitfall

Caching without an ownership rule. Every cached key needs an owner that knows when the source data changes. If multiple services write the same database row but only one service invalidates the cache, stale reads become intermittent and hard to diagnose.

4.2.3 Operational Design

ElastiCache clusters live in VPC subnets and should be reached from private application tiers, not from public clients. Production designs use security groups, encryption in transit where supported, authentication, subnet groups, Multi-AZ replication, automatic failover, metrics, and alarms. Important metrics include cache hit ratio, evictions, CPU, memory, network throughput, replication lag, current connections, and command latency.

Note

A low cache hit ratio is not automatically bad. New workloads, write-heavy workloads, or personalized one-time reads may be poor cache candidates. Measure hit ratio with database load, latency, and stale-read tolerance before declaring success.

4.3 Tuesday: DynamoDB Accelerator

4.3.1 Where DAX Fits

DynamoDB Accelerator is a managed, in-memory cache for DynamoDB. It is API-compatible with DynamoDB client patterns through the DAX SDK, so applications can accelerate eventually consistent read-heavy access patterns without building a separate Redis key naming scheme [6]. DAX keeps item and query results in memory, handles cache population, and invalidates entries for writes that pass through the DAX client.

Definition

DAX is a DynamoDB-compatible caching layer that can reduce read latency from milliseconds to microseconds for suitable read-heavy workloads. It is not a general Redis replacement and it does not support strongly consistent reads.

4.3.2 DAX Versus ElastiCache

Choose DAX when the source is DynamoDB, the application uses key-value or query access patterns, and the team wants the cache to understand DynamoDB operations. Choose ElastiCache when the source is RDS, Aurora, APIs, S3 metadata, computed responses, session data,

counters, sorted sets, rate limits, or any workload requiring custom cache structures. DAX is specialized; ElastiCache is general-purpose.

Decision point	DAX	ElastiCache
Source of truth	DynamoDB tables and indexes	RDS, Aurora, DynamoDB, APIs, files, computed values, and service responses
Client model	DAX-aware DynamoDB client	Redis, Valkey, or Memcached protocol clients
Invalidation	Managed for writes through DAX; TTL for cached items and query results	Application-managed invalidation, TTL, eviction, pub/sub, or stream-driven refresh
Data structures	DynamoDB items and query result sets	Strings, hashes, lists, sets, sorted sets, streams, counters, and arbitrary serialized objects
Consistency	Eventually consistent reads only	Application-defined; cache can be stale unless invalidated or bypassed
Best fit	Read-heavy DynamoDB hot paths	General cache-aside, sessions, rate limits, leaderboards, API response cache, SQL result cache

Table 4.1: DAX and ElastiCache solve different cache problems.

Tip

Use DAX for workloads with repeated `GetItem`, `BatchGetItem`, and predictable `Query` requests. Do not use it to hide poor partition-key design, scans, or analytical access patterns.

Common Pitfall

Expecting strong consistency from DAX. Strongly consistent reads bypass the DAX cache and go to DynamoDB. If the workflow requires read-after-write correctness for every request, design that path explicitly instead of assuming the cache can provide it.

4.3.3 DAX Operations

A DAX cluster is deployed into a VPC with subnet groups and IAM permissions. Applications should set timeouts, retries, and fallback behavior so a cache issue does not become a total outage. Monitor cache hit rate, item-cache hits, query-cache hits, CPU, memory, error counts, and request latency. Because DAX nodes are stateful caches, resizing or failover can temporarily reduce hit ratio while entries warm again.

4.4 Wednesday: AppSync and API Gateway Data Ingestion

4.4.1 Managed API Front Doors

AWS AppSync is a managed GraphQL service for building APIs that connect clients to DynamoDB, Lambda, OpenSearch, HTTP services, RDS through resolvers, and other sources [11]. Amazon API Gateway provides managed REST, HTTP, and WebSocket APIs with authorization, throttling, usage plans, transformations, stages, and integrations [3]. In data engineering systems, these services are the ingestion and serving layer that protects databases from direct client traffic.

Definition

Managed API ingestion uses services such as AppSync and API Gateway to authenticate, validate, throttle, transform, and route incoming data before it reaches storage or processing systems.

4.4.2 Choosing AppSync or API Gateway

AppSync fits applications where clients need a typed graph, flexible selection sets, subscriptions, offline synchronization, or direct resolver integration with DynamoDB. API Gateway fits RESTful ingestion, HTTP proxy patterns, serverless microservices, webhooks, partner APIs, and simple mobile backends. Both can integrate with Lambda for validation and orchestration, and both can sit in front of cached read paths.

Note

GraphQL reduces over-fetching, but it can increase resolver fan-out if each field triggers a separate backend lookup. Use batching, pipeline resolvers, authorization boundaries, and cache-aware resolver design for high-traffic graphs.

Tip

Place throttles and quotas at the API layer before traffic reaches the database. A cache absorbs repeated reads, but API Gateway usage plans, AppSync limits, WAF rules, and client backoff prevent abusive or accidental request storms.

4.4.3 Hybrid Ingestion Patterns

Common ingestion patterns include API Gateway to Lambda to Aurora, AppSync resolvers to DynamoDB, API Gateway to Kinesis for streaming telemetry, and AppSync subscriptions for near-real-time client updates. The serving side may read from DAX for DynamoDB, ElastiCache for RDS query results, or precomputed S3 and CloudFront assets for public content. The best architecture often separates write correctness from read speed: writes go to the durable source, while reads use caches, materialized views, and CDN-friendly objects.

Common Pitfall

Letting clients bypass the ingestion layer. Direct database access from mobile or browser clients exposes credentials, weakens throttling, and removes the chance to validate payloads before writes reach the system of record.

4.5 Thursday Hands-on Project: Redis Cache-Aside for RDS Queries

This lab deploys a small ElastiCache Redis-compatible cluster and uses Python to cache RDS query results. The commands assume Windows PowerShell, AWS CLI v2, a sandbox VPC with at least two private subnets, an application security group, and an existing RDS or Aurora database endpoint. Replace identifiers before running, and clean up resources after testing.

Note

ElastiCache nodes are billed while running. Use the smallest test node type available in your account and delete the replication group after the lab if it is not needed.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $Region = "us-east-1"
3 $VpcId = "vpc-0123456789abcdef0"
4 $SubnetIds = "subnet-aaaa1111 subnet-bbbb2222"
5 $AppSecurityGroupId = "sg-app1234567890"
6 $CacheSecurityGroupId = "sg-cache1234567890"
7 $SubnetGroupName = "ch4-redis-subnets"
8 $ReplicationGroupId = "ch4-rds-query-cache"
9 aws elasticache create-cache-subnet-group '
10     --cache-subnet-group-name $SubnetGroupName '
11     --cache-subnet-group-description "Private subnets for Chapter 4 Redis" '
12     --subnet-ids $SubnetIds '
13     --region $Region
14 aws elasticache create-replication-group '
15     --replication-group-id $ReplicationGroupId '
16     --replication-group-description "Chapter 4 Redis cache-aside demo" '
17     --engine redis '
18     --cache-node-type cache.t4g.micro '
19     --num-cache-clusters 2 '
20     --automatic-failover-enabled '
21     --cache-subnet-group-name $SubnetGroupName '
22     --security-group-ids $CacheSecurityGroupId '
23     --transit-encryption-enabled '
24     --at-rest-encryption-enabled '
25     --region $Region
26 aws elasticache wait replication-group-available '
27     --replication-group-id $ReplicationGroupId '
28     --region $Region

```

Listing 4.1: PowerShell-oriented AWS CLI commands to create an ElastiCache Redis replication group.

```

1 import json
2 import os
3 from decimal import Decimal
4 import psycopg
5 import redis
6 redis_client = redis.Redis(
7     host=os.environ["REDIS_HOST"],
8     port=int(os.environ.get("REDIS_PORT", "6379")),
9     ssl=os.environ.get("REDIS_SSL", "true").lower() == "true",
10    socket_timeout=1.0,
11    decode_responses=True,
12 )
13 connection_string = os.environ["RDS_POSTGRES_DSN"]
14 CACHE_SECONDS = int(os.environ.get("CACHE_SECONDS", "60"))
15 def default_json(value):
16     if isinstance(value, Decimal):
17         return float(value)
18     raise TypeError(f"Cannot serialize {type(value)}")
19 def get_product(product_id):
20     cache_key = f"product:{product_id}"
21     cached = redis_client.get(cache_key)
22     if cached:
23         return {"source": "cache", "item": json.loads(cached)}
24     with psycopg.connect(connection_string) as conn:
25         with conn.cursor() as cur:
26             cur.execute(
27                 """
28                 SELECT product_id, name, price, inventory_count, updated_at
29                 FROM products
30                 WHERE product_id = %s
31                 """,
32                 (product_id,),

```

```

33         )
34         row = cur.fetchone()
35     if row is None:
36         redis_client.setex(cache_key, 15, json.dumps({"missing": True}))
37         return {"source": "database", "item": None}
38     item = {
39         "product_id": row[0],
40         "name": row[1],
41         "price": row[2],
42         "inventory_count": row[3],
43         "updated_at": row[4].isoformat(),
44     }
45     redis_client.setex(cache_key, CACHE_SECONDS, json.dumps(item, default=
default_json))
46     return {"source": "database", "item": item}
47 if __name__ == "__main__":
48     for _ in range(2):
49         print(get_product(os.environ.get("PRODUCT_ID", "SKU-1001")))

```

Listing 4.2: Python cache-aside script using redis-py and pycpg to cache RDS query results.

Tip

Cache negative lookups briefly. A short TTL for missing products, users, or feature flags prevents repeated database hits during typos or crawler traffic while limiting stale absence after a new record appears.

4.6 Friday Use Case: Mobile Sports API with Extreme Traffic Spikes

A sports media company serves a mobile app during championship events. Before kickoff, traffic is moderate. During a goal, trade, injury, or final play, millions of clients refresh the same scorecards, player statistics, odds summaries, and social reactions. The workload combines hot public reads, personalized user reads, bursty clickstream writes, and strict correctness for purchases, subscriptions, and user settings.

4.6.1 Architecture Narrative

The write path accepts clickstream and interaction events through API Gateway and routes them to Kinesis, DynamoDB, or Lambda depending on latency and ordering needs. User profile and billing writes go to Aurora or RDS through a service boundary. Active sessions, shopping carts, live score summaries, and user-specific notification state use DynamoDB where key-value scale is required. The read path uses AppSync for GraphQL mobile screens, API Gateway for REST endpoints, DAX for repeated DynamoDB reads, and ElastiCache for live-score fragments, SQL query results, rate limits, and computed response objects.

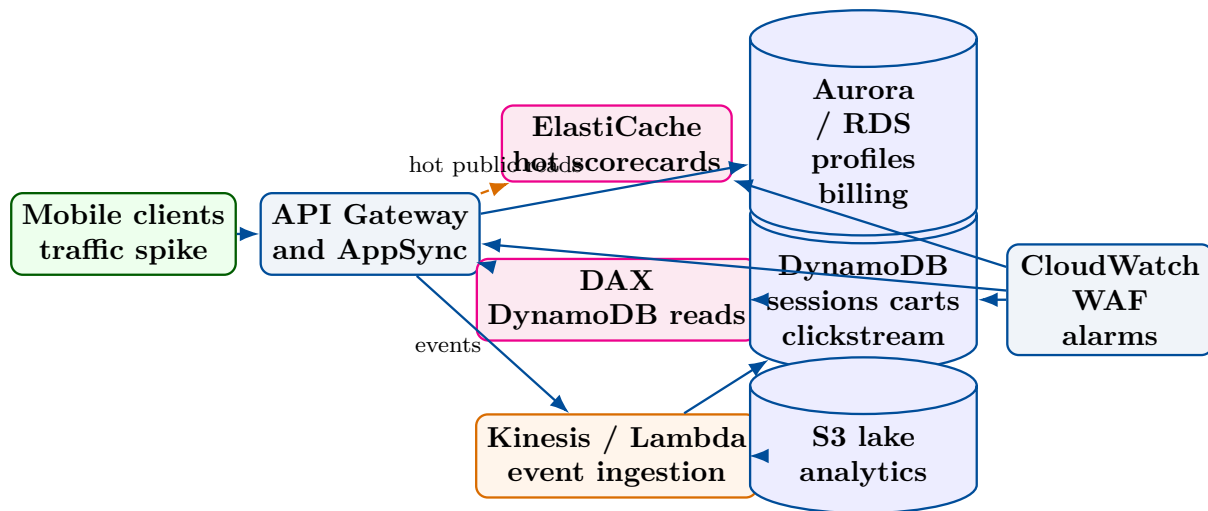


Figure 4.2: Spiky-traffic mobile sports API using API Gateway, AppSync, ElastiCache, DAX, DynamoDB, Aurora or RDS, and streaming ingestion.

4.6.2 Design Decisions

Separate hot public reads from personalized reads. Everyone asks for the same live score after a major play, so cache public scorecards and summary fragments aggressively. Personalized notifications, purchases, and account settings need user-scoped keys and stricter authorization. **Precompute the most popular shapes.** Maintain live scoreboard documents, player stat cards, and top-comment summaries with stream processors or scheduled refresh. Do not ask the database to assemble the same response millions of times during a spike. **Throttle before storage.** API Gateway, AppSync, AWS WAF, quotas, and client retry guidance protect the database from fan-out. Caches reduce load after requests arrive, but throttles prevent overload when the request itself is unnecessary or abusive.

Common Pitfall

Caching the wrong boundary. Caching a full personalized response can leak data if the cache key omits identity, locale, subscription tier, or authorization context. Public and private response fragments should use different key spaces and TTL policies.

4.7 Interview Q&A

4.7.1 Concepts and Trade-offs

1. **Q: What problem does ElastiCache solve?**

A: It serves repeated reads or transient computations from memory so durable databases and APIs handle fewer requests with lower average latency.

2. **Q: When would you choose Redis-compatible ElastiCache over Memcached?**

A: Choose Redis-compatible engines for richer data structures, replication, clustering, atomic operations, streams, sorted sets, and persistence options. Choose Memcached for simple ephemeral object caching with a minimal key-value model.

3. **Q: What is cache-aside?**

A: The application checks the cache first, loads from the database on a miss, stores the result with a TTL, and returns it. The database remains the source of truth.

4. **Q: Why is cache invalidation difficult?**

A: The system must know every write that changes a cached value, remove or refresh all affected keys, and handle races between readers, writers, retries, and TTL expiration.

5. **Q: How is DAX different from ElastiCache?**

A: DAX is a DynamoDB-specific, API-compatible cache for eventually consistent read-heavy access patterns. ElastiCache is a general Redis, Valkey, or Memcached service for many sources and data structures.

6. **Q: Can DAX provide strongly consistent reads?**

A: No. Strongly consistent reads bypass DAX and go to DynamoDB, so workflows requiring strict read-after-write behavior need explicit design.

7. **Q: When is AppSync a better fit than API Gateway?**

A: AppSync is often better for GraphQL mobile or web experiences that need typed schemas, subscriptions, flexible selection sets, and resolver integration. API Gateway is often better for REST, HTTP, webhook, and microservice ingestion patterns.

8. **Q: What metrics indicate cache health?**

A: Cache hit ratio, latency, evictions, memory pressure, CPU, connection count, replication lag, error rate, and downstream database load together show whether the cache is helping.

4.8 Further Reading

Start with the ElastiCache documentation for engine choices, cluster modes, security, scaling, and monitoring [8]. Review DAX behavior before using it with DynamoDB hot paths [6]. Study AppSync and API Gateway ingestion patterns, authorization options, quotas, and resolver or integration models [11, 3]. For the capstone, revisit S3, Aurora Serverless v2, DynamoDB, and AWS DMS documentation from Weeks 1–3 [10, 27, 7, 12].

Key Takeaways

- Caches reduce latency and database load only for cacheable access patterns with repeated reads or reusable computations.
- ElastiCache is general-purpose in-memory acceleration; DAX is specialized DynamoDB read acceleration.
- Cache-aside is simple and resilient, but it requires disciplined key naming, TTLs, serialization, fallback behavior, and invalidation.
- AppSync and API Gateway protect data stores by authenticating, throttling, validating, transforming, and routing client traffic.
- Spiky mobile APIs need precomputed hot responses, API throttles, cache-aware read paths, and durable write paths.
- Part I systems become powerful when S3, Aurora, DynamoDB, caches, and DMS are composed around clear sources of truth.

4.9 Exercises

1. **(Conceptual)** Explain why a cache should usually be treated as disposable rather than as the system of record.

2. **(Conceptual)** Compare cache-aside and write-through for product catalog reads where prices change hourly.
3. **(Design)** Choose Redis-compatible ElastiCache, Memcached, or DAX for sessions, leaderboard sorted sets, DynamoDB product lookups, and RDS report fragments. Justify each choice.
4. **(Hands-on)** Run listing 4.1 in a sandbox VPC and capture the replication group status, primary endpoint, security group rule, and subnet group.
5. **(Hands-on)** Modify listing 4.2 to cache a category query and print whether each response came from the cache or database.
6. **(Operations)** Define CloudWatch alarms for cache evictions, CPU, memory pressure, replication lag, connection count, and RDS read load.
7. **(Security)** Describe how cache key design changes when responses vary by user identity, subscription tier, locale, or authorization scope.
8. **(Architecture)** Sketch a spiky live sports API and identify which responses use AppSync, API Gateway, DAX, ElastiCache, DynamoDB, Aurora, and S3.

4.10 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

Capstone Project

Mission. Build an e-commerce multi-tier data backend that integrates the Part I services: Aurora Serverless v2 for user profiles and billing, DynamoDB for active shopping carts and clickstream tracking, S3 for encrypted product images with lifecycle rules, ElastiCache and DAX for hot-path acceleration, and AWS DMS to continuously replicate Aurora data to S3 for analytics. **Estimated time.** 8–12 hours in a sandbox AWS account, or 4–6 hours for a documented architecture and scripted dry run when account limits prevent full deployment.

What you'll practice.

- Composing relational, NoSQL, object storage, cache, and replication services into one data platform.
- Separating sources of truth from hot read paths and analytical replicas.
- Applying encryption, lifecycle management, least privilege, TTL, capacity planning, and operational evidence.
- Producing implementation artifacts, acceptance criteria, and a cleanup plan suitable for a portfolio project.

Scenario

A fast-growing e-commerce company is replacing a monolithic database with a multi-tier AWS backend. User profiles, billing methods, orders, refunds, and account status require relational transactions and belong in Aurora Serverless v2. Active shopping carts, clickstream events, recommendation impressions, and session metadata need key-value scale and belong in DynamoDB. Product images and exports belong in S3 with encryption and lifecycle controls. Read-heavy product and cart APIs need ElastiCache and DAX. Finance and analytics teams need a continuous copy of Aurora data in S3 for lake queries without impacting production.

Requirements

Functional requirements.

- Create an Aurora Serverless v2 cluster or document the exact cluster, database, subnet group, and security configuration required for profiles and billing.
- Create DynamoDB tables or a single-table design for active carts, cart items, clickstream events, session state, and recommendation impressions.
- Store product images in S3 with server-side encryption, public access blocked, versioning where appropriate, and lifecycle transition or expiration rules.
- Use ElastiCache to cache hot product detail and inventory summary reads sourced from Aurora or API composition.
- Use DAX for repeated DynamoDB cart or session reads where eventual consistency is acceptable.
- Configure AWS DMS to continuously replicate selected Aurora tables to S3 for analytics landing files.

Non-functional requirements.

- No source code, scripts, or documentation may contain secrets, passwords, personal credentials, or private keys.
- User and billing writes must favor correctness over cache freshness. Caches must be invalidated or bypassed for sensitive updates.
- Product image buckets must block public access unless a separate CloudFront or signed URL design is explicitly documented.
- Cart and clickstream keys must distribute traffic across users and time buckets rather than concentrate all writes on one global key.
- DMS replication must have monitoring for task status, latency, errors, and S3 object delivery.

Reference Architecture

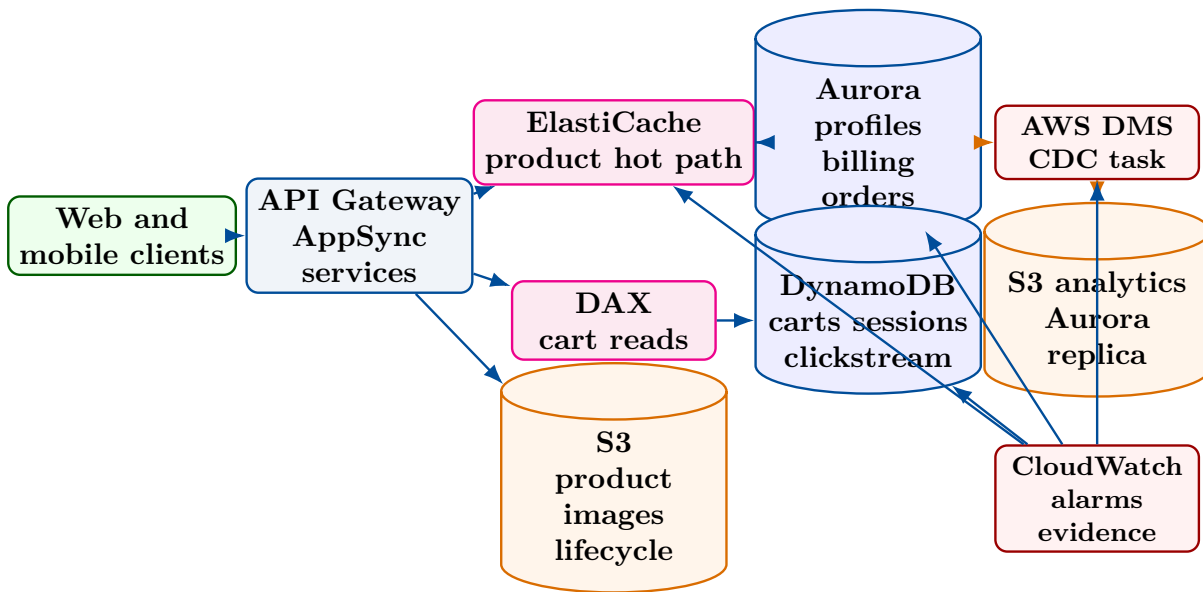


Figure 4.3: Milestone 1 e-commerce reference architecture integrating Aurora Serverless v2, DynamoDB, S3, ElastiCache, DAX, AWS DMS, and observability across all tiers.

Implementation Steps

1. **Define domains and ownership.** Mark Aurora as the source of truth for profiles, billing, orders, and refunds. Mark DynamoDB as the source for active carts, sessions, and clickstream events. Mark S3 as the source for product image objects and analytics landing files.
2. **Create storage foundations.** Configure S3 buckets with encryption, public access block, versioning, and lifecycle rules. Create Aurora Serverless v2 and DynamoDB resources using private networking and least-privilege IAM.
3. **Add acceleration.** Place ElastiCache in private subnets for product detail and inventory summary reads. Add DAX for repeated cart reads if eventual consistency is acceptable.
4. **Implement ingestion APIs.** Use API Gateway or AppSync to validate requests, authorize users, throttle clients, and route writes to Aurora, DynamoDB, or S3 pre-signed URL workflows.
5. **Replicate to analytics.** Configure DMS source and target endpoints, a replication instance or serverless task where available, table mappings, and continuous CDC from Aurora to S3.
6. **Validate failure modes.** Test cache miss behavior, stale cache invalidation, DAX bypass for strict reads, DynamoDB TTL behavior, S3 lifecycle policy syntax, and DMS task recovery.
7. **Capture evidence.** Save architecture diagrams, command output, resource descriptions, sample API calls, metrics screenshots, and cleanup commands.

```

1 $Region = "us-east-1"
2 $AccountSuffix = "123456789012"
3 $ImageBucket = "m1-ecommerce-product-images-$AccountSuffix"

```

```

4 $AnalyticsBucket = "m1-ecommerce-analytics-$AccountSuffix"
5 $CartTable = "m1-active-carts"
6 $SubnetGroupName = "m1-cache-subnets"
7 $ReplicationGroupId = "m1-product-cache"
8 aws s3api create-bucket --bucket $ImageBucket --region $Region
9 aws s3api put-public-access-block '
10   --bucket $ImageBucket '
11   --public-access-block-configuration '
12     BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,
13     RestrictPublicBuckets=true
14 aws s3api put-bucket-encryption '
15   --bucket $ImageBucket '
16   --server-side-encryption-configuration '
17     '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}]}'
18 aws s3api put-bucket-lifecycle-configuration '
19   --bucket $ImageBucket '
20   --lifecycle-configuration '
21     '{"Rules":[{"ID":"transition-old-images","Status":"Enabled","Filter":{"Prefix":"products/"},
22     "Transitions":[{"Days":90,"StorageClass":"STANDARD_IA"}]}'
23 aws s3api create-bucket --bucket $AnalyticsBucket --region $Region
24 aws dynamodb create-table '
25   --table-name $CartTable '
26   --attribute-definitions AttributeName=PK,AttributeType=S AttributeName=SK,
27   AttributeType=S '
28   --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE '
29   --billing-mode PAY_PER_REQUEST '
30   --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
31   --sse-specification Enabled=true '
32   --region $Region
33 aws dynamodb wait table-exists --table-name $CartTable --region $Region
34 aws dynamodb update-time-to-live '
35   --table-name $CartTable '
36   --time-to-live-specification "Enabled=true,AttributeName=expiresAt" '
37   --region $Region
38 aws elasticache create-cache-subnet-group '
39   --cache-subnet-group-name $SubnetGroupName '
40   --cache-subnet-group-description "Milestone 1 cache subnets" '
41   --subnet-ids subnet-aaaa1111 subnet-bbbb2222 '
42   --region $Region
43 aws elasticache create-replication-group '
44   --replication-group-id $ReplicationGroupId '
45   --replication-group-description "Milestone 1 product cache" '
46   --engine redis '
47   --cache-node-type cache.t4g.micro '
48   --num-cache-clusters 2 '
49   --automatic-failover-enabled '
50   --cache-subnet-group-name $SubnetGroupName '
51   --security-group-ids sg-cache1234567890 '
52   --transit-encryption-enabled '
53   --at-rest-encryption-enabled '
54   --region $Region

```

Listing 4.3: PowerShell-oriented AWS CLI skeleton for S3, DynamoDB, and cache resources.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os
4 import uuid
5 import boto3
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 table = boto3.resource("dynamodb", region_name=region).Table(
8     os.environ.get("CART_TABLE", "m1-active-carts")

```

```

9 )
10 s3 = boto3.client("s3", region_name=region)
11 image_bucket = os.environ["IMAGE_BUCKET"]
12 def put_cart_item(user_id, sku, quantity, price):
13     now = datetime.now(timezone.utc)
14     ttl = int((now + timedelta(days=30)).timestamp())
15     table.put_item(
16         Item={
17             "PK": f"USER#{user_id}",
18             "SK": f"CART#{sku}",
19             "entityType": "CartItem",
20             "sku": sku,
21             "quantity": quantity,
22             "price": Decimal(str(price)),
23             "updatedAt": now.isoformat(),
24             "expiresAt": ttl,
25         }
26     )
27     return {"cacheInvalidationKey": f"cart:{user_id}", "sku": sku}
28 def record_click(user_id, sku, action):
29     now = datetime.now(timezone.utc)
30     event_id = str(uuid.uuid4())
31     day = now.strftime("%Y-%m-%d")
32     table.put_item(
33         Item={
34             "PK": f"CLICK#{day}#{user_id[-2:]}",
35             "SK": f"{now.isoformat()}#{event_id}",
36             "entityType": "ClickstreamEvent",
37             "userId": user_id,
38             "sku": sku,
39             "action": action,
40             "expiresAt": int((now + timedelta(days=7)).timestamp()),
41         }
42     )
43     return event_id
44 def create_product_image_upload(sku, content_type="image/jpeg"):
45     key = f"products/{sku}/{uuid.uuid4()}.jpg"
46     return s3.generate_presigned_url(
47         "put_object",
48         Params={"Bucket": image_bucket, "Key": key, "ContentType": content_type},
49         ExpiresIn=900,
50     )

```

Listing 4.4: Boto3 sketch for carts, clickstream events, and cache invalidation metadata.

Deliverables

- Architecture diagram showing clients, API layer, Aurora Serverless v2, DynamoDB, S3 product images, ElastiCache, DAX, DMS, S3 analytics, and monitoring.
- Data ownership matrix that identifies the source of truth, cache policy, TTL, encryption setting, backup or retention rule, and access pattern for each entity.
- AWS CLI or infrastructure scripts for S3 buckets, DynamoDB tables, ElastiCache, DAX or documented DAX plan, Aurora configuration, and DMS replication.
- Boto3 or service code demonstrating cart writes, clickstream ingestion, product image upload workflow, cache invalidation keys, and safe serialization.
- Evidence packet with resource descriptions, table schemas, bucket policies, lifecycle output, cache endpoints, DMS task status, CloudWatch metrics, and cleanup commands.

Acceptance Criteria

- Aurora Serverless v2 is the documented transactional source for profiles, billing, orders, and refunds.
- DynamoDB cart and clickstream keys distribute traffic by user, date, or bucket and include TTL where temporary data is expected.
- S3 product image storage blocks public access, uses server-side encryption, and has a lifecycle policy.
- ElastiCache is placed in private subnets and has a clear cache-aside or invalidation strategy for product and inventory reads.
- DAX is used only for DynamoDB eventually consistent read paths or is explicitly rejected with a reason.
- AWS DMS has a continuous replication design from Aurora to S3 with table mappings, monitoring, and recovery notes.
- API ingestion includes authentication, validation, throttling, and no direct client access to databases.
- The final design includes cleanup steps and cost controls for every billable resource.

Stretch Goals

- Add CloudFront with signed URLs or signed cookies for product image delivery without making the S3 bucket public.
- Implement a Lambda or stream consumer that invalidates ElastiCache keys after Aurora product updates.
- Add a synthetic load script that demonstrates database load reduction after cache warmup.
- Add Athena table definitions over the DMS S3 target and run a sample analytics query.
- Add multi-region disaster recovery notes for Aurora, DynamoDB global tables, S3 replication, and cache warmup.

Appendix A

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-trieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob stor- age	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Database Global	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data (AWS/Azure/GCP)	lake	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS)	prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/ prefix		STAGING schema
Gold (curated)	Gold Lakehouse	curated/ prefix		ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Ice- berg		Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example **raw_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon nomoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + parti- tioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (syn- chronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True- Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

Consistency level A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

Cosmos DB Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

Cross-Region Replication (CRR) Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.

DynamoDB Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

Durability The probability that stored data is not lost, commonly quoted as “eleven nines” (99.99999999%) for cloud object storage.

ElastiCache Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.

Embedding vs referencing The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

Fail-safe Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

Global Secondary Index (GSI) A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.

Hierarchical namespace (HNS) The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

Journal (WAL) The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.

JSON Schema validation MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

Interleaved table A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

Lakehouse An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

Lifecycle policy A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

Medallion architecture A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

Materialized view A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

Memorystore Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Amazon Web Services. Amazon s3 strong consistency. <https://aws.amazon.com/s3/consistency/>, 2020. AWS feature announcement and documentation.
- [2] Amazon Web Services. Aws well-architected framework: Analytics lens. <https://docs.aws.amazon.com/wellarchitected/latest/analytics-lens/analytics-lens.html>, 2024. AWS Well-Architected guidance.
- [3] Amazon Web Services. Amazon api gateway developer guide. <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>, 2026. Accessed 2026-07-31.
- [4] Amazon Web Services. Amazon aurora storage and reliability. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Overview.StorageReliability.html>, 2026. Accessed 2026-07-31.
- [5] Amazon Web Services. Amazon aurora user guide. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html, 2026. Accessed 2026-07-31.
- [6] Amazon Web Services. Amazon dynamodb accelerator developer guide. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html>, 2026. Accessed 2026-07-31.
- [7] Amazon Web Services. Amazon dynamodb developer guide. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>, 2026. Accessed 2026-07-31.
- [8] Amazon Web Services. Amazon elasticache user guide. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>, 2026. Accessed 2026-07-31.
- [9] Amazon Web Services. Amazon relational database service user guide. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>, 2026. Accessed 2026-07-31.
- [10] Amazon Web Services. Amazon simple storage service user guide. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>, 2026. Accessed 2026-07-30.
- [11] Amazon Web Services. Aws appsync developer guide. <https://docs.aws.amazon.com/appsync/latest/devguide/what-is-appsync.html>, 2026. Accessed 2026-07-31.
- [12] Amazon Web Services. Aws database migration service user guide. <https://docs.aws.amazon.com/dms/latest/userguide/Welcome.html>, 2026. Accessed 2026-07-31.
- [13] Amazon Web Services. Aws datasync user guide. <https://docs.aws.amazon.com/datasync/latest/userguide/what-is-datasync.html>, 2026. Accessed 2026-07-30.
- [14] Amazon Web Services. Aws schema conversion tool user guide. https://docs.aws.amazon.com/SchemaConversionTool/latest/userguide/CHAP_Welcome.html, 2026. Accessed 2026-07-31.

- [15] Amazon Web Services. Aws snow family documentation. <https://docs.aws.amazon.com/snowball/latest/developer-guide/whatisedge.html>, 2026. Accessed 2026-07-30.
- [16] Amazon Web Services. Aws storage gateway user guide. <https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html>, 2026. Accessed 2026-07-30.
- [17] Amazon Web Services. Best practices for aws database migration service. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_BestPractices.html, 2026. Accessed 2026-07-31.
- [18] Amazon Web Services. Best practices for designing and architecting with dynamodb. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html>, 2026. Accessed 2026-07-31.
- [19] Amazon Web Services. Dynamodb global tables. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>, 2026. Accessed 2026-07-31.
- [20] Amazon Web Services. Dynamodb read and write capacity mode. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>, 2026. Accessed 2026-07-31.
- [21] Amazon Web Services. Dynamodb streams and time to live. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>, 2026. Accessed 2026-07-31.
- [22] Amazon Web Services. High availability for amazon rds db instances. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>, 2026. Accessed 2026-07-31.
- [23] Amazon Web Services. Improving data access with secondary indexes in dynamodb. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>, 2026. Accessed 2026-07-31.
- [24] Amazon Web Services. Locking objects with s3 object lock. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>, 2026. Accessed 2026-07-30.
- [25] Amazon Web Services. Replicating objects. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html>, 2026. Accessed 2026-07-30.
- [26] Amazon Web Services. Using amazon aurora global database. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database.html>, 2026. Accessed 2026-07-31.
- [27] Amazon Web Services. Using amazon aurora serverless v2. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2.html>, 2026. Accessed 2026-07-31.
- [28] Amazon Web Services. Using amazon s3 storage classes. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-intro.html>, 2026. Accessed 2026-07-30.
- [29] Amazon Web Services. Working with db instance read replicas. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html, 2026. Accessed 2026-07-31.
- [30] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.

- [31] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*, pages 205–220, 2007.

Index

- access pattern, [19](#), [57](#)
- active-active, [61](#)
- adaptive capacity, [59](#)
- Amazon API Gateway, [71](#), [75](#)
- Amazon Aurora, [38](#)
- Amazon DynamoDB, [56](#)
- Amazon ElastiCache, [71](#)
- Amazon RDS, [38](#)
- Amazon S3, [15](#)
- Apache Spark
 - equivalence, [90](#)
- API Gateway cache, [78](#)
- API throttling, [76](#)
- AppSync subscription, [78](#)
- asynchronous replication, [41](#)
- attribute, [58](#)
- Aurora cache, [73](#)
- Aurora Capacity Unit, [43](#)
- Aurora cluster volume, [42](#)
- Aurora Global Database, [38](#), [42](#)
 - capstone, [49](#)
- Aurora PostgreSQL, [44](#), [46](#)
 - capstone, [49](#)
- Aurora Serverless v2, [38](#), [43](#), [81](#)
- automated backups, [40](#)
- availability, [15](#), [18](#)
- Availability Zone, [18](#), [40](#)
- AWS account, [1](#)
- AWS AppSync, [71](#), [75](#)
- AWS Budgets, [1](#)
- AWS CLI, [1](#), [22](#), [44](#), [61](#), [76](#)
- AWS CloudShell, [1](#)
- AWS Database Migration Service, [38](#)
- AWS DataSync, [21](#)
- AWS DMS, [43](#), [81](#)
- AWS Schema Conversion Tool, [38](#)
- AWS SCT, [43](#)
- backup retention, [40](#)
- Bigtable
 - equivalence, [91](#)
- Boto3, [44](#), [61](#)
- bucket, [16](#)
- cache hit ratio, [74](#)
- cache invalidation, [73](#)
- cache warming, [75](#)
- cache-aside, [71](#)
- cache-aside pattern, [73](#)
- caching, [71](#)
 - equivalence, [90](#)
- capstone project, [28](#), [49](#), [65](#)
- change data capture, [43](#), [60](#)
- cloning
 - equivalence, [91](#)
- compliance mode, [20](#)
- composite key, [59](#)
- conflict resolution, [61](#)
- connection pooling, [43](#)
- consistency, [18](#)
- Cost Explorer, [1](#)
- cross-cloud equivalence, [87](#)
- Cross-Region Replication, [20](#)
 - capstone, [28](#)
- cross-region replication, [42](#)
- cutover, [46](#)
- data lake, [17](#)
- data residency, [17](#)
- data retention, [60](#)
- DataSync, [15](#)
- DAX, [62](#), [71](#), [74](#)
 - capstone, [81](#)
- DAX cluster, [75](#)
- DB instance, [40](#)
- delete marker, [19](#)
- denormalization, [59](#)
- DICOM, [25](#)
- disaster recovery, [25](#)
- distributed databases
 - equivalence, [89](#)
- distributed storage, [42](#)
- DMS
 - capstone, [49](#)
- DMS endpoint, [43](#)
- document database, [56](#)
- document modeling

- equivalence, 89
- durability, 15, 18
- DynamoDB
 - capstone, 65
- DynamoDB Accelerator, 71, 74
- DynamoDB cache, 74
- DynamoDB Streams, 56, 60
 - capstone, 65
- DynamoDB table, 58
- e-commerce backend, 81
- edge compute, 21
- elastic compute
 - equivalence, 88
- ElastiCache
 - capstone, 81
- ElastiCache for Redis, 73
- encryption, 25
- Enhanced Monitoring, 40
- event-driven architecture, 57
- eventual consistency, 58, 74
- eviction, 74
- failover, 40
- failover drill, 44
- fallback behavior, 75
- File Gateway, 21
- forum schema, 61
- Free Tier, 1
- full load, 43
- gaming leaderboard, 62
 - capstone, 65
- Glacier Flexible Retrieval, 22
- global secondary index, 56, 60
- Global Tables, 56, 61
 - capstone, 65
- glossary, 92
- governance mode, 20
- GraphQL, 75
- healthcare, 25
- heterogeneous migration, 43
- HIPAA, 25
- hot path, 72
- HTTP API, 75
- hybrid cloud, 21
- hybrid ingestion, 71
- IAM Identity Center, 1
- in-memory cache
 - equivalence, 90
- incremental transfer, 21
- index projection, 60
- ISO-8601 timestamp, 59
- item, 58
- item collection, 59
- key-value database, 56
- KMS, 17
- Lambda trigger, 60
- latency, 72
- leaderboard architecture, 62
- legal hold, 20
- lifecycle policies, 15
- lifecycle policy, 19
- list consistency, 18
- LOB migration, 43
- local secondary index, 56, 60
- managed database, 39
- materialized view, 60, 76
- materialized views
 - equivalence, 90
- medallion architecture
 - equivalence, 88
- medical imaging data lake, 28
- Memcached, 71, 73
- metadata, 16
- Milestone 1 capstone, 81
- minimum storage duration, 19
- mobile data API, 78
- mobile ingestion, 76
- Multi-AZ, 38
- Multi-AZ deployment, 40
- multi-cloud, 87
- multi-factor authentication, 1
- multi-region, 61
- multipart upload, 19
- noncurrent version, 19
- NoSQL, 56
 - equivalence, 89
- object key, 16
- Object Lock, 15, 20
- object storage, 15
 - equivalence, 87
- offline migration, 21
- OLTP, 39
- on-demand capacity, 59
- online migration, 21
- option group, 40
- Oracle ERP, 46
- Oracle ERP migration

- capstone, 49
- Oracle migration, 38
- parameter group, 40
- partition key, 56
- Performance Insights, 40
- physical partition, 58
- point-in-time recovery, 40
 - equivalence, 91
- POSIX, 17
- prefix, 17
- provisioned capacity, 59
- query acceleration
 - equivalence, 90
- query-first design, 59
- quorum, 42
- RDS cache, 73
- RDS query cache, 76
- read amplification, 72
- read capacity unit, 58
- Read Replica, 41
- Read Replicas, 38
- read scaling, 41
- read-after-write consistency, 18
- reader endpoint, 42
- recovery, 19
- Redis, 71
- Redis cluster, 76
- redis-py, 76
- Region, 1, 17
- regional failover, 42
- relational databases, 38
 - equivalence, 88
- replica lag, 41
- replication, 15
- replication instance, 43
- replication lag, 20, 74
- resolver, 76
- REST API, 75
- retrieval fee, 19
- root user, 1
- RPO, 20, 42
- RTO, 42
- S3 encryption, 81
- S3 lifecycle, 81
- S3 Object Lock
 - capstone, 28
- Same-Region Replication, 20
- security group, 74
- serverless database, 43
- service mapping, 87
- single-table design, 57
- single-table schema, 61
- Snow Family, 15
- Snowball, 21
- Snowcone, 21
- Snowmobile, 21
- sort key, 56
- sort-key prefix, 59
- source of truth, 72
- sparse index, 60
- sports analytics, 78
- SQS event notification, 22
 - capstone, 28
- storage classes, 15
 - equivalence, 87
- Storage Gateway, 15, 21
- strong consistency, 58
- supplemental logging, 43
- synchronous replication, 40
- table mapping, 43
- tagging, 1
- tags, 16
- Tape Gateway, 21
- throttling, 59
- time to live, 60
- time travel
 - equivalence, 91
- traffic spike, 78
- transaction processing, 39
- TTL, 60, 73
- Valkey, 73
- versioning, 15, 19
- virtual warehouse
 - equivalence, 88
- Volume Gateway, 21
- WebSocket API, 75
- wide-column
 - equivalence, 91
- WORM, 20
- write capacity unit, 58
- write sharding, 62
- write-through cache, 71, 73
- writer endpoint, 42
- zero-downtime migration, 46